

Otto Väyrynen

UTILIZING DIFFUSION-BASED TEXT-TO-IMAGE MODEL IN OBJECT DETECTION

Master of Science Thesis
Faculty of Information Technology and Communication Sciences
Examiners: Prof. Joni Kämäräinen
October 2025

ABSTRACT

Otto Väyrynen: Utilizing Diffusion-Based Text-To-Image Model In Object Detection
Master of Science Thesis
Tampere University
Master's Degree Programme in Information Technology
October 2025

Training an accurate object detection model requires large amounts of diverse images of the target object. Existing datasets for training are not always available, for example when the target is represented in a non-standard format or is uncommon. In those cases, the images must be captured manually, which is time-consuming and labour-intensive. Traditionally, one way to alleviate this problem has been to increase the number of training images by augmenting existing images.

In the last few years, text-to-image models capable of generating realistic synthetic images have developed rapidly. The SDXL text-to-image model from Stable Diffusion has become popular due to its open-source availability, enabling the creation of custom fine-tuned models that generate synthetic images. With only a small number of images, a generative model can be fine-tuned to produce synthetic images that match the content of the dataset. Ideally, these images are realistic enough to train an object detection model for use with real-world images. This work investigates whether such synthetic images can improve the performance of an object detection model.

In this work, an object detection model was developed to locate people in near-infrared (NIR) images in small indoor environments. To train the model, NIR-like synthetic images were generated using the SDXL text-to-image model, fine-tuned with 31 NIR images and the LoRA technique. These synthetic images were combined with real images to create a set of fine-tuned YOLOv8n object detection models, and they were ranged against each other.

The results showed that models fine-tuned solely with synthetic images outperformed the base model ($AP_{IoU:0.50}$: 0.866-0.908 vs. 0.815). When real-world images were added to make up roughly one-tenth of the large synthetic dataset, the model's performance and stability increased even further. The best result was achieved by combining all 568 real-world images with 2500 synthetic images ($AP_{IoU:0.50}$: 0.938). This work showed that synthetic images generated by a fine-tuned diffusion-based text-to-image model can improve the performance of object detection models to detect objects in real-world images.

Keywords: Object Detection, Computer Vision, Machine Learning, Synthetic Data, Synthetic Data Generation, Syn2Real, Diffusion Model

The originality of this thesis has been checked using the Turnitin OriginalityCheck service.

TIIVISTELMÄ

Otto Väyrynen: Diffuusiopohjaisen tekstistä kuvaksi -mallin hyödyntäminen kohteentunnistuksessa

Diplomityö

Tampereen yliopisto

Tietotekniikan DI-ohjelma

Lokakuu 2025

Tarkan kohteentunnistusmallin opettaminen vaatii suuria määriä erilaisia kuvia tunnistettavasta kohteesta. Välttämättä aina ei ole saatavilla valmiita kuvia mallin opettamista varten, jos kyseessä on esimerkiksi perinteisistä värikuvista poikkeava esitysmuoto tai jos tunnistettava kohde ei ole yleinen. Tällöin mallin opettamiseen on otettava kuvat itse, mikä on työlästä ja aikaavievää. Perinteisesti tämän ongelman helpottamiseksi opetukseen käytettävien kuvien määrää on voitu kasvattaa augmentoimalla jo otettuja kuvia.

Nykyiset realistisia synteettisiä kuvia luovat tekstistä-kuvaksi -mallit ovat kehittyneet vauhdilla viimeisten vuosien aikana. Avoimella lähdekoodillaan suosituksi tullut Stable Diffusionin tekstistä-kuvaksi -malli SDXL mahdollistaa omien hienosäädettyjen synteettisiä kuvia tuottavien mallien luomisen. Jo pienellä määrällä kuvia voidaan generoiva malli hienosäätää tuottamaan kuva-asetin sisältöä vastaavia synteettisiä kuvia. Parhaimmillaan nämä kuvat ovat tarpeeksi realistisia, jotta niillä voidaan kouluttaa kohteentunnistusmalli käytettäväksi reaali maailman kuville. Tässä diplomityössä tutkitaan, voidaanko tuotetuilla synteettisillä kuvilla parantaa kohteentunnistusmallin suorituskykyä.

Työssä tuotettiin kohteentunnistusmalli ihmisten paikantamiseen lähi-infrapunakuvista pienissä sisätiloissa. Mallin kouluttamista varten tuotettiin lähi-infrapunakuvien kaltaisia synteettisiä kuvia SDXL tekstistä-kuvaksi -mallilla hienosäätämällä sitä 31 lähi-infrapunakuvan ja LoRA-tekniikan avulla. Näitä synteettisiä kuvia käytettiin yhdessä aitojen kuvien kanssa luomaan joukko hienosäädettyjä YOLOv8n-kohteentunnistussalleja, joiden suorituskykyä verrattiin keskenään tuloksissa.

Tuloksista voitiin havaita, että jo pelkästään synteettisillä kuvilla hienosäädetyt mallit suoriutuivat paremmin ($AP_{IoU:0.50}$: 0.866-0.908) kuin perusmalli ($AP_{IoU:0.50}$: 0.815). Kun reaali maailman kuvien osuus kasvatettiin noin kymmenesosaan suuresta määrästä synteettisiä kuvia, kasvoi mallin suorituskyky ja stabiilisuus entisestään. Paras tulos saatiin, kun kaikki 568 reaali maailman kuvaa yhdistettiin 2500 synteettiseen kuvaan ($AP_{IoU:0.50}$: 0.938). Tämä työ osoitti, että hienosäädetyn diffuusiopohjaisen tekstistä-kuvaksi -mallin tuottamilla synteettisillä kuvilla voidaan parantaa kohteentunnistusmallin suorituskykyä tunnistamaan kohteita reaali maailman kuvista.

Avainsanat: Kohteentunnistus, Koneäkö, Koneoppiminen, Synteettinen data, Synteettisen datan generointi, Synteettisestä todelliseen, Diffuusiomalli

Tämän julkaisun alkuperäisyys on tarkastettu Turnitin OriginalityCheck -ohjelmalla.

USE OF ARTIFICIAL INTELLIGENCE IN THIS WORK

Artificial intelligence (AI) has been used in generating this work:

- ☐ No
☒ Yes

I hereby declare, that the AI-based applications used in generating this work are as follows:

Application	Version
Copilot	GPT-4o, GPT-5

Purpose of the use of AI

In this work, AI was used to correct spelling and grammatical errors and to improve readability.

Parts of this work, where AI was used

All parts of the work were checked and, where necessary, fixed and improved by using AI.

Acknowledgement of risks

I hereby acknowledge, that as the author of this work, I am fully responsible for the contents presented in this thesis. This includes the parts that were generated by an AI, in part or in their entirety. I therefore also acknowledge my responsibility in the case, where use of AI has resulted in ethical guidelines being breached.

CONTENTS

1.	Introduction	1
2.	Theoretical Background	3
2.1	Object Detection	3
2.1.1	Object Detection Methods	5
2.1.2	Object Detection Evaluation Metrics	7
2.2	Deep Learning Architectures	11
2.2.1	Convolutional Neural Networks	12
2.2.2	U-net	14
2.2.3	Transformers	15
2.3	Synthetic Data in Object Detection	18
2.4	Text-To-Image Models	22
2.5	Diffusion Models and Stable Diffusion	24
2.5.1	Diffusion Models	24
2.5.2	Conditioning	31
2.5.3	Stable Diffusion	33
2.5.4	SDXL	35
2.5.5	LoRA	36
3.	Implementation	38
3.1	Workflow	39
3.2	Scoping	39
3.3	Data Management	41
3.4	Synthetic Image Generation	43
3.4.1	Upscaling	43
3.4.2	Image Captioning	44
3.4.3	LoRA Training	46
3.4.4	Image Generation	48
3.5	Object Detection Model Training	50
4.	Results	52
4.1	Testing setup and Impact of Synthetic Images	52
4.2	Confidence Threshold Optimization and Qualitative Analysis	55
5.	Discussion	61
5.1	Discussion of Implementation and Results	61
5.2	Practical Recommendations	62
5.3	Future Works	63

6. Conclusion	65
References.	67
Appendix A: Effects of Image Generation Parameters	79
Appendix B: Additional Results	81

LIST OF SYMBOLS AND ABBREVIATIONS

AI	Artificial Intelligence
AP	Average Precision
CFG	Classifier-Free Guidance
CLIP	Contrastive Language–Image Pretraining
CNN	Convolutional Neural Network
COCO	Common Objects in Context
CV	Computer Vision
DDPM	Denoising Diffusion Probabilistic Model
DETR	Detection Transformer
DM	Diffusion Model
ELBO	Evidence Lower Bound
FN	False Negative
FP	False Positive
GAN	Generative Adversarial Network
GPT	Generative Pre-trained Transformer
IoU	Intersection over Union
KL	Kullback-Leibler divergence
LDM	Latent Diffusion Model
LLM	Large Language Model
LoRA	Low-Rank Adaptation
mAP	mean Average Precision
MLP	Multilayer Perceptron
NIR	Near-Infrared
PEFT	Parameter-Efficient Fine-Tuning
R-CNN	Regions with CNN features
ReLU	Rectified Linear Unit
SDXL	Stable Diffusion XL

Syn2Real	Synthetic-to-Real
T2I	Text-to-Image
TP	True Positive
VAE	Variational Autoencoder
ViT	Vision Transformer
YOLO	You Only Look Once
E	Embedding matrix
I	Identity matrix
L	Loss function
Q, K, V	Query, Key, and Value matrices in attention mechanisms
W	Weight matrix
$\Sigma_{\theta}(x_t, t)$	Covariance of the reverse process at step t
α_t	Signal retention rate $\alpha_t = 1 - \beta_t$
α	LoRA scaling factor
$\bar{\alpha}_t$	Cumulative product of the signal retention rate over time steps, $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$
β_t	Forward process variance at step t
$\epsilon_{\theta}(x_t, t)$	Neural network's predicted noise in x_t at timestep t
ϵ	Standard Gaussian noise
$\mathbb{E}[\cdot]$	Expectation operator
$\mathcal{E}(\cdot), \mathcal{D}(\cdot)$	Encoder and decoder of the variational autoencoder (VAE)
$\mathcal{N}(z; \mu, \Sigma)$	Normal distribution of random variable z with mean μ and covariance Σ
$\mu_{\theta}(x_t, t)$	Mean of the reverse process at step t
$\phi_i(z_t)$	Flattened U-Net intermediate representation
$\tau_{\theta}(y)$	Encoded text prompt for conditioning in latent diffusion
$\tilde{\beta}_t$	Forward process posterior variance
$\tilde{\mu}_t(x_t, x_0)$	Forward process posterior mean
$p_{\theta}(\cdot)$	Learned reverse process distribution
$q(\cdot)$	Forward process distribution

r	LoRA rank
s	Guidance scale for classifier or classifier-free guidance
x_0	Data sample at the initial forward diffusion step
x_T	Data sample at the final forward diffusion step
x_t	Data sample at an intermediate forward diffusion step
y, y_{neg}	Conditional input embedding or negative prompt embedding
z	Latent representation of the data sample x in latent diffusion

1. INTRODUCTION

Object detection has become an essential part of many fields such as medical imaging, transportation, or industrial safety monitoring. Despite its wide range of applications, object detection is still one of the most challenging and fundamental problems in computer vision [1]. Modern deep learning-based object detectors that automatically localize and classify objects of interest in images are only as good as the data used for training the model. Depending on the application, training an accurate model might require thousands to even millions of diverse images [2, 3]. As collecting large-scale datasets can be time-consuming and expensive, data scarcity might become a bottleneck in training accurate object detection models if no public datasets are available. Especially in fields such as military, medical, or surveillance imaging, data gathering can be difficult due to privacy concerns, rare occurring events, or the sheer difficulties of capturing images [4–7].

The traditional approach to overcome data scarcity is to use data augmentation techniques, where existing images are modified using operations such as rotation, scaling, or flipping to create new training images [8]. While traditional data augmentation methods can help improve model robustness, they are limited to producing variations of the original image, bringing nothing new. In this work, methods are utilized that the author refers to as "augmentation with steroids," as they can produce images with new content through synthetic data generation from fine-tuned text-to-image models.

The release of ChatGPT at the end of 2022 started a surge of AI hype. Around the same time, generative text-to-image models such as DALL-E 2, Midjourney, and Stable Diffusion also gained a lot of attention for their ability to generate high-quality images that mimic various artistic styles and create realistic pictures from text prompts [9, 10]. The quality of these models has developed to the point that it can be very difficult to distinguish them from real-world photographs [11]. In addition, there are open-source models that have enabled users to fine-tune large-scale generative models to produce images that reflect the content of their training datasets. While text-to-image generative models have raised ethical concerns, including privacy violations, unauthorized use of artistic styles, and the spread of misinformation, they also present promising opportunities in many fields such as education, healthcare, and object detection [12, 13]. By applying these models to object detection, it is possible to synthesize diverse datasets that generalize to different content while capturing visual similarities to the target images, even creating new sce-

narios that were not in the original dataset. This work will investigate the potential of the text-to-image model to address data scarcity by increasing both the size and diversity of the dataset and how it may help with class imbalance in edge cases.

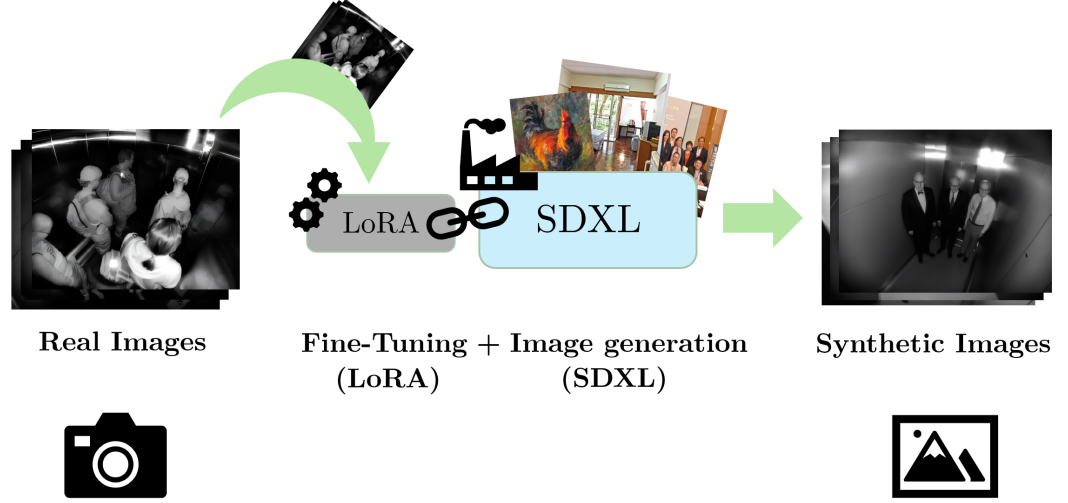


Figure 1.1. Illustration of the synthetic image generation process. Captured sensor images are used to train a LoRA model, which is then used to fine-tune the SDXL text-to-image model. Because SDXL is trained on large-scale datasets of captioned images, the fine-tuned model can generate diverse images from text prompts that resemble the sensor images when the LoRA is applied.

The research question of this work is formulated as follows: Can synthetic images generated from a fine-tuned diffusion-based text-to-image model improve the performance of an object detection model? To answer this question, an end-to-end pipeline for synthetic data generation and object detection is implemented from scratch. The work focuses on detecting people in near-infrared sensor images captured in small indoor environments. By using 31 images captured from the sensor, the Stable Diffusion XL (SDXL) text-to-image model is fine-tuned using low-rank adaptation (LoRA) to generate synthetic images that resemble the sensor images. To evaluate the effectiveness of synthetic images in improving object detection performance, multiple YOLOv8n object detection models are fine-tuned using different combinations of real and synthetic images.

This work is structured as follows: Chapter 2 provides an overview of the related work in the field of object detection and synthetic data generation. Chapter 3 details the scope of this work and the implemented end-to-end pipeline for synthetic data generation and object detection. Chapter 4 presents the results of the trained fine-tuned object detection models. Chapter 5 discusses the implementation and results, and suggests recommendations for implementations and future work. Finally, Chapter 6 concludes the work with a summary of implementations and results.

2. THEORETICAL BACKGROUND

Computer vision is a field of artificial intelligence that studies how computers can extract data from visual information, analyze it, modify it, and understand it at a high level [14, 15]. For example, common computer vision tasks include image classification, object tracking, semantic segmentation, video action detection, and image inpainting [14, 16]. In this work, the focus of computer vision tasks will be on object detection and image generation.

2.1 Object Detection

Object detection is the process by which objects of certain categories are identified from data, combining object classification and localization [1, 17]. It answers the questions of what the object is and where it is located in the data. In object detection, the data comes from visual sources such as images, videos, or even point clouds [17, 18].

Most object detection algorithms are based on machine learning and deep learning algorithms [17]. These algorithms can learn patterns and acquire knowledge from training data to make predictions on future data [19, 20]. Training data is typically annotated data; in this work it consists of images where objects of interest are annotated with bounding boxes and class types. As training data guides how algorithms learn to make predictions, these algorithms are heavily affected by the quality and quantity of the training data [21–23]. The number of images needed to train an accurate object detection model depends highly on the context and ranges from hundreds to hundreds of thousands of images, with varying numbers of object instances. For example, 500 images can be enough for a simple fruit detector, about 2500 images for medical imaging with human-level accuracy, and over a million instances for a complex multi-class detector for 600 object classes [2, 3, 24]. It also appears that complex models benefit more from larger datasets than from smaller ones [3]. As a general rule, model performance increases logarithmically as the training dataset increases, so the training dataset might grow very large [22]. However, more data does not always mean better performance, and higher quality data or using better learning algorithms can give better results even with a much smaller training dataset [25]. That's why it is important to take into account how objects of interest are represented in the training dataset, as a central task for machine learning is extracting features from data

to learn and understand what is in the input data [14, 23]. For example, features of the object can be seen differently when the object is viewed from different perspectives or when the object is being deformed. Collecting a diverse training dataset is challenging, as there are various object-related factors:

- Variations in size, shape, pose, deformability, texture, or color of objects [26, 27]
- Partial visibility due to occlusion [28]
- Different angles due to camera placement, affecting viewpoint and viewing distance [29, 30]
- Differences in resolution, focus, and unwanted effects such as artifacts or noise from different cameras [31, 32]
- Environmental factors such as background clutter and changing illumination conditions [33]
- False-positive objects caused by mirror reflections [34–36]

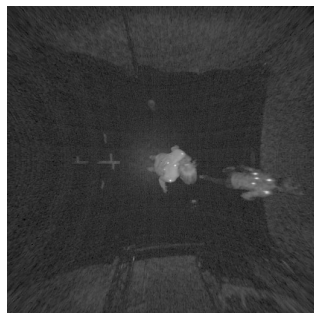


Figure 2.1. *People can appear differently in various contexts [37–39]*

What makes a good object detection model generally depends on its intended use case. For example, a model deployed in a surveillance camera may require high speed and low computational cost, whereas in medical imaging, diagnostic precision is more critical. When comparing object detection algorithms, they are usually evaluated with classification metrics like average precision (AP), but also with other metrics such as inference speed and model size, using benchmark datasets like COCO (Common Objects in Context) [40–43]. There can also be several case-dependent criteria for evaluation that need

to be taken into account when training a model, such as scalability for retraining or the ability to handle certain edge cases. In the end, training a good object detection model requires handling several factors, which usually include selecting a suitable object detection model, providing a training dataset of appropriate size and quality, and applying preprocessing techniques.

Next, subsections 2.1.1 and 2.1.2 will discuss different object detection methods with a brief historical overview and introduce the most important object detection metrics used in this work. Later, chapter 3.1 will present a general perspective on object detection workflows.

2.1.1 Object Detection Methods

Object detection techniques have improved a lot over the last decades. It could be said that the history of object detection can be roughly divided into two historical periods: the period of traditional object detection, which is the time before the year 2014, and the period of deep learning-based detection, which is after that [1]. Correspondingly, object detection methods can be divided into traditional and deep learning-based approaches [17]. Next, traditional and deep learning-based¹ object detection methods are introduced in the context of these two historical periods.

Traditional object detection algorithms were mostly created on the basis of handcrafted features. Usually, the problem was the lack of advanced image representation and computational resources. Methods like the Viola-Jones detector (VJ detector) from 2001, Histogram of Oriented Gradients (HOG), and Deformable Part Models (DPM) were popular in traditional object detection. [1, 17] Traditional object detection has three basic steps: informative region selection, feature extraction, and classification. As these steps included handcrafted features, generally traditional object detection models had difficulties with different domains and detections with complex backgrounds and diverse objects. [44] These problems became easier to handle later with deep learning-based object detection models.

The second turning point in object detection was initiated in 2012 when the deep learning model AlexNet [45] was introduced in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC). AlexNet included, for example, convolutional neural network (CNN) architecture and the ReLU activation function. The achieved results got a lot of attention for using CNNs in object detection, and after 2014, when Regions with CNN features (RCNNs) were proposed by Girshick et al. [46], object detection algorithms started to evolve faster than ever [1]. After 2017, when Vaswani et al. [47] introduced the transformer architecture, it became very popular in the field of natural language processing

¹This section discusses different types of deep learning architectures and related terms, such as CNNs and transformers. These topics are introduced in more detail later in Section 2.2.

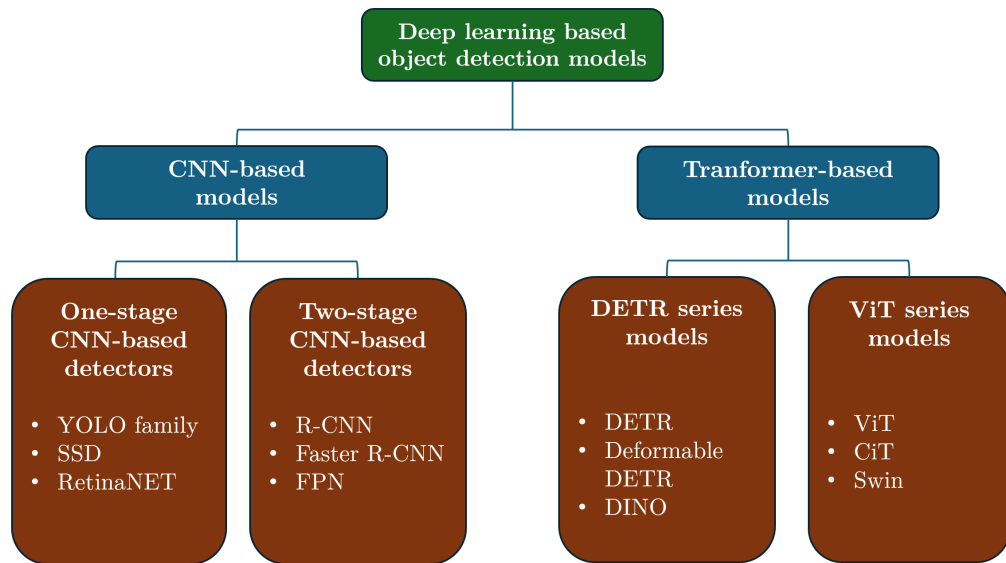


Figure 2.2. Deep learning based object detection classification diagram

(NLP) and was applied very well also in other tasks like object detection. This becomes a point where modern deep learning-based object detection methods can be divided into two main categories: CNN-based and transformer-based object detection methods. [44]

CNN-based deep learning object detection methods are usually divided into two main categories: one-stage detectors and two-stage detectors [17]. The most popular one-stage detectors are particularly the YOLO (You Only Look Once) family. For two-stage detectors, there are various popular options like R-CNN, Faster R-CNN, and feature pyramid network (FPN). These two methods differ from each other in how they detect objects from the data. One-stage detectors can classify and localize objects in one-step inference by extracting features with the CNN directly. Two-stage detectors first search for potential objects using region proposal algorithms like Selective Search or Region Proposal Networks (RPNs) and then classify and localize each proposed region by using CNN. [44] Due to the complexity of two-stage detectors, compared to one-stage detectors they are usually more accurate, precise at localization, robust to occlusions, and better at handling complex scenes and small objects. However, two-stage detectors' inference speed is slower, they are computationally expensive, and complex to train [33, 48]. These advantages and disadvantages suggest that one-stage detectors are better suited for real-time applications such as surveillance cameras and two-stage detectors may be more appropriate for accuracy-sensitive cases like medical image analysis. However, these differences are not so big nowadays as models are improving continuously. For example, [49] reports that YOLOv8 achieved higher results than Faster RCNN in both accuracy and inference time.

Transformer-based object detection methods are quite new compared to CNN-based methods, as before 2020 there were no transformer-based object detection models until

Carion et al. detection transformers (DETR) [50] and Dosovitskiy et al. Vision transformers [51]. These methods can be divided into two categories: DETR series models and ViT series models. The idea of transformers lies in the self-attention mechanism, which makes the model able to pay more attention to the position of sequence features and the relationship between them. DETR models focus on end-to-end object detection by using a transformer encoder-decoder architecture, while ViT models are designed for general vision tasks like image classification by processing images as sequences of patches, and so ViT series models are usually used as backbones in object detection models. [44] The performance of these models is sometimes said to be better than CNN-based models in larger datasets, but they are generally slower, larger, and more memory-intensive than traditional models [33, 52, 53]. Recently, there has been research that tries to improve the performance by combining CNN and transformer models like DEYOv2 [54], but in this work it will be left out of discussion.

2.1.2 Object Detection Evaluation Metrics

Object detection performance is mainly evaluated by localization and classification tasks. When comparing different object detection models, many other metrics related to model properties, like inference time, model size, and floating-point operations (FLOPs), can be included [55]. In this work, the focus is on detection rather than model computational performance; therefore, only the most common metrics used to evaluate the performance of localization and classification will be introduced, starting with localization.

A common way to measure localization performance is to evaluate the overlapped area between predicted and ground truth bounding boxes. This measure is called Intersection over Union (IoU), and it's defined as

$$IoU = \frac{A_{Overlap}}{A_{Union}}. \quad (2.1)$$

In the formula, $A_{Overlap}$ is the overlapping area between the detected bounding box and ground truth bounding box. A_{Union} is the combined area of detected and ground truth bounding box. [56] The closer the IoU value is to 1, the better the localization is.

In practice, IoU is used to classify a detection as correct or incorrect with a given threshold. If IoU is greater or equal to the given threshold, it is classified as correct; otherwise, incorrect. [56] IoU can also be used for removing duplicate bounding boxes in detection. Non-maximum suppression (NMS) is an algorithm used to eliminate overlapping bounding boxes and reduce duplicate detections. First, the algorithm selects the bounding box with the highest confidence score. Then IoU is calculated between this box and all other boxes with lower confidence. Any boxes with an IoU greater than a predefined threshold are discarded. This process is repeated: the next highest-confidence box among

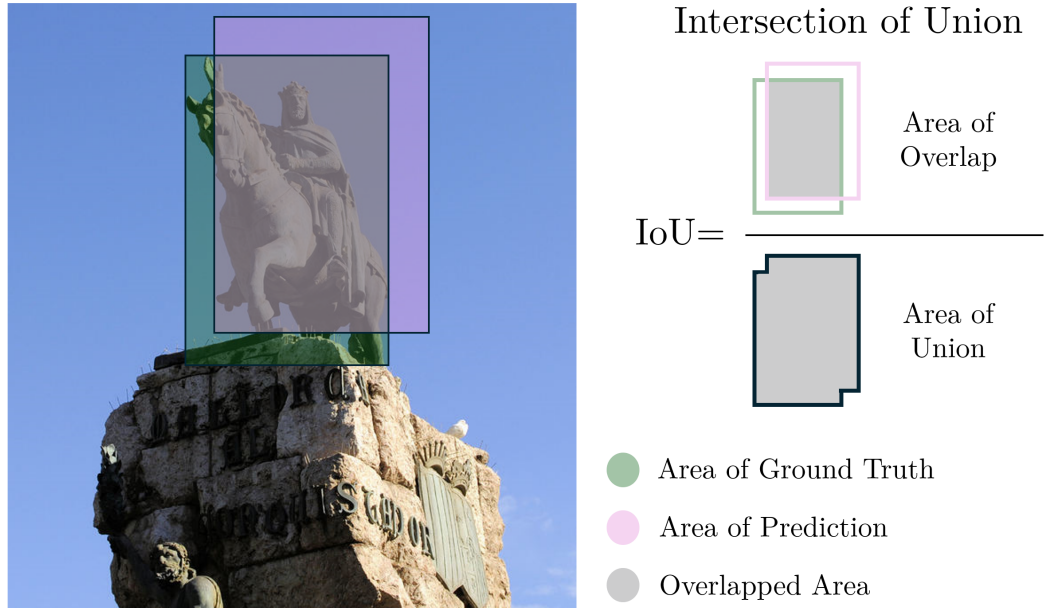


Figure 2.3. IoU is calculated as the area of overlap between the predicted and the ground truth bounding box divided by the area of union.

the remaining boxes is selected, and this process continues until only boxes with locally maximum confidence scores remain. [57]

There are several different metrics to evaluate classification performance. These metrics are derived from the confusion matrix, which is a table that describes different outcomes of classification. In a binary classification task, the confusion matrix has four different outcomes: True positive (TP), True negative (TN), False positive (FP), and False negative (FN). [58] The confusion matrix for a binary classification task is defined as:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP) Object detected correctly	False Negative (FN) Object not detected, but exists
Actual Negative	False Positive (FP) Object detected, but does not exist	True Negative (TN) Object not detected, and does not exist

In this matrix, TP and TN represent correct predictions and likewise FP and FN represent incorrect predictions. It is good to note that TN is not generally used in object detection, because all images can have infinite number of bounding boxes that should not be detected [56]. Therefore TP, FP and FN are used to derive classification metrics precision, recall, average precision and F1-score [58].

Precision is the ratio of correctly detected positive cases to all made detections. It is defined as:

$$Precision = \frac{TP}{TP + FP} . \quad (2.2)$$

A low precision value indicates a high number of falsely detected objects in predicted data. Because of that, precision is an important metric, especially when false alarms are costly. However, since precision does not take into account missed detections, recall is also needed. [58]

Recall is the ratio of correctly detected positive cases to all positive ground truth cases, and it measures the model's ability to detect relevant instances. Recall is defined as:

$$Recall = \frac{TP}{TP + FN} \quad (2.3)$$

A low recall value indicates there is a high number of cases where objects are not detected even though they exist. [58]

Between precision and recall there is a trade-off: If precision increases, recall tends to decrease and vice versa [58]. This trade-off is generally influenced by the confidence score threshold: A higher confidence score threshold means more predictions are discarded due to a strict threshold and so there are more FN cases, meaning higher precision. A lower confidence score threshold means even uncertain predictions are accepted, which leads to greater FP cases and recall. This trade-off can be illustrated in a precision-recall curve (see Figure 2.4), where values are plotted at confidence score threshold values [58].

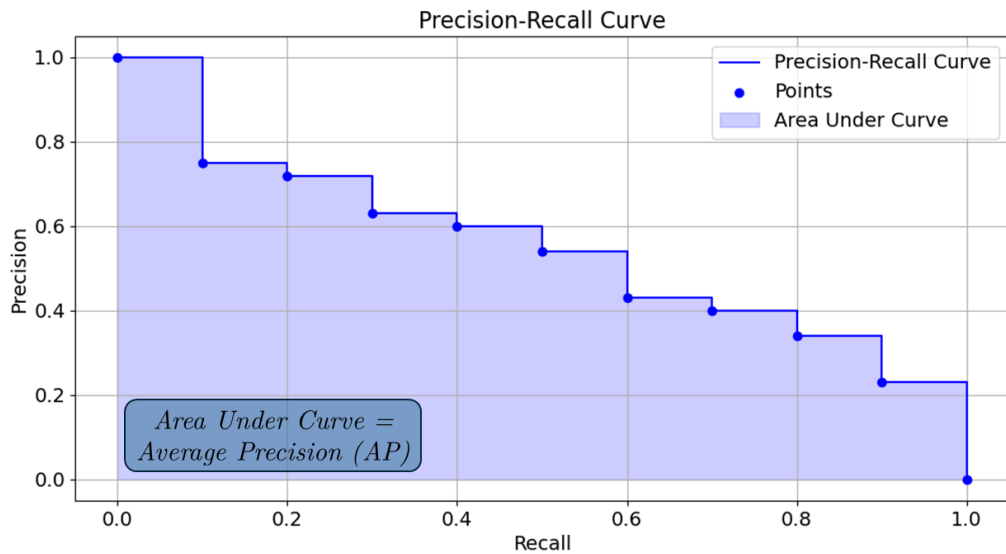


Figure 2.4. Example of a precision-recall curve using the 11-point interpolation approach.

In practice, models are often compared based on how well they maintain high recall before precision starts to drop fast. To represent this overall performance as a single numeric value, the information of the precision-recall curve is expressed as Average Precision (AP). Average Precision and its aggregate form, mean Average Precision (mAP), are very popular metrics to evaluate object detection models. AP is defined as the area under the precision-recall curve and can be expressed as:

$$AP_{\tau} = \int_0^1 p(r)dr, \quad (2.4)$$

where AP_{τ} is AP at IOU threshold τ and $p(r)$ is the precision at recall r . [58] Mean Average Precision is the mean of the AP values of all object classes. mAP is defined as

$$mAP = \frac{1}{n} \sum_{i=1}^n AP_i \quad (2.5)$$

where n is the number of classes and AP_i is the AP of class i . [56] If there is only one class, mAP is equal to AP. It is important to note that there are different ways to calculate AP. For example, two widely used evaluation standards, Microsoft COCO and Pascal VOC, compute AP differently [58]. In this work, the Open-Source Visual Interface for Object Detection Metrics [59] is used to calculate AP using Pascal VOC AP computation. Often in literature, there are notations $AP@0.50$ or $AP@[.50:.05:.95]$, which means that AP is calculated using different IoU thresholds. $AP@50$ refers to AP calculated using IoU threshold 0.5 and $AP@[.50:.05:.95]$ means the average of AP values calculated using IoU thresholds from 0.5 to 0.95 with step size 0.05. [58]

As AP summarizes performance across all confidence score thresholds, it is usually necessary in practice to find a single confidence score threshold that gives the best balance between precision and recall. For this, the F1-score is used. F1-score is the harmonic mean of precision and recall:

$$F1 = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}} = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}. \quad (2.6)$$

One feature of the harmonic mean is that it penalizes large differences between precision and recall, so a high F1-score value indicates a good balance between those two metrics. [58]

In this work, AP is used as the main evaluation metric to measure overall performance for trained object detection models, while F1-score is used to find the optimal confidence score threshold for the models.

2.2 Deep Learning Architectures

Deep learning is a subset of machine learning that utilizes neural networks to solve complex tasks like object detection, image generation and natural language processing [20]. This section will, after a short refresher about neural networks, introduce the main information about convolutional neural networks (CNNs), U-Net, and transformers as they are related to object detection and text-to-image generation models used in this work.

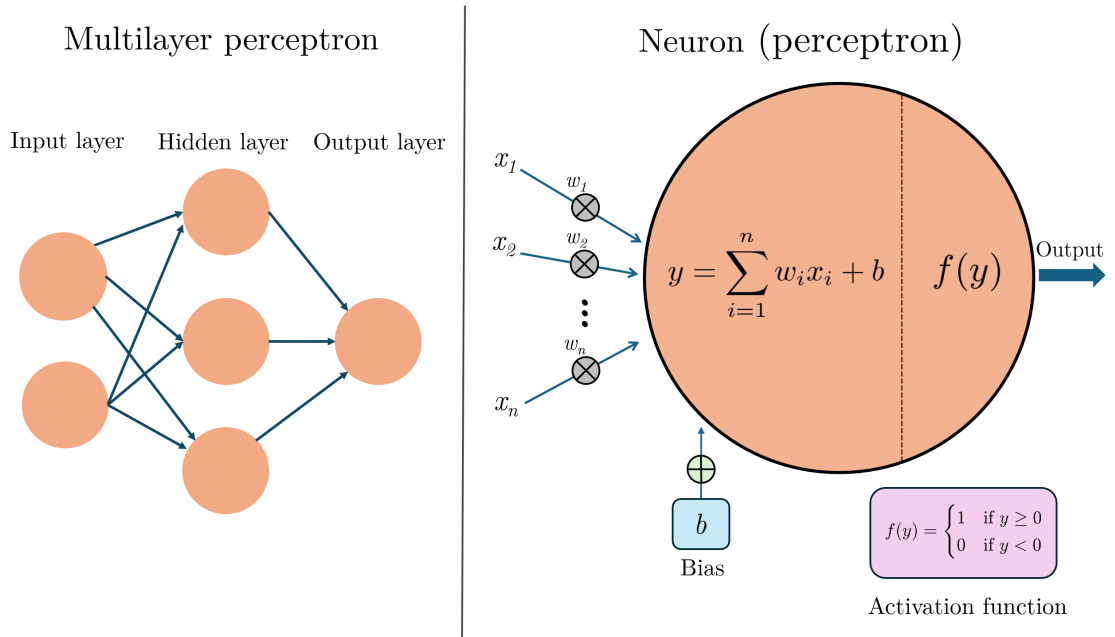


Figure 2.5. Example of a simple multilayer perceptron and a neuron. The activation function is chosen to be the Heaviside step function in this example.

Neural networks are collections of neurons, or computing units, that are connected to each other [60]. They are designed to be universal predictors² that can be trained to learn complex relationships between input and output [60, 63]. While a single neuron, also called a perceptron, can mainly be used for binary classification, learning more complicated features requires several layers of neurons. A multilayer perceptron (MLP) is a type of neural network that consists of an input layer, an output layer, and possibly one or multiple hidden layers. In the MLP, each neuron in a layer is connected to all neurons in the next layer, creating a fully connected neural network. The input layer simply receives and passes forward the input data. In the hidden layers, all incoming input data to a neuron are multiplied by the weights assigned to those inputs, summed together, and then a single bias is added (compare to the equation of a line $y = ax + b$). The result of the summation is then passed through a mathematical function called an activation function, which gives an output based on the function used. Nowadays, activation functions are usually monotone increasing functions in practice, such as the logistic function, hyperbolic tangent function, or ReLU, which serve modern neural network architectures better.

²More information about the universal approximation theorem [61, 62].

At the end of the network is the output layer, which is similar to a hidden layer but gives the final output and may use a different activation function compared to the hidden layers. [60, 64]

The goal of the neural network is to learn relationships between input and output, which can be achieved by learning the weights and biases that minimize the loss function. The loss function, also known as the cost or error function, measures the error between the predicted output and the ground truth output. During the training process, the weights and biases are usually adjusted using backpropagation and the gradient descent algorithm. The backpropagation algorithm computes gradients of the loss with respect to the weights and biases. The gradient descent optimization algorithm, or its popular variant stochastic gradient descent, is used to update the weights and biases in the direction that minimizes the loss function using gradients calculated from backpropagation. This iterative process is repeated at each step, with the step size determined by the learning rate, until the stopping criteria are met. [60, 64]

2.2.1 Convolutional Neural Networks

Convolutional neural networks (CNNs) are deep learning architectures that are commonly used for different computer vision problems, including object detection, image segmentation, and image classification [65]. A CNN is a type of neural network that uses convolution kernels for feature extraction. A typical CNN architecture consists of convolutional layers, pooling layers, and fully connected layers, as seen in Figure 2.6 [66]. Convolutional and pooling layers perform feature extraction to learn relevant features for the fully connected layers, which map those features into the final output such as probabilities in classification tasks [67].

In a convolutional layer, features are extracted from the input image using convolution kernels. Kernels are small weighted filters that are slid over the input image, computing at each step a convolution: a dot product between the kernel and an image patch of the same size. [68] This produces a new image called a feature map. Each feature map highlights different features, such as edges, corners, or textures, depending on the kernel's weights. As the kernel weights do not change while sliding over the image (also known as weight sharing), it allows the detection of the same features across the image while reducing the number of parameters, which is a key feature of the convolution operation. In a convolutional layer, there are multiple different kernels that extract different features, as their weights are different. [67] For example, in Figure 2.6, the first convolution layer of the CNN has 96 kernels. Kernels are learnable parameters that are adjusted during the training process so they can extract meaningful features from the input image [68]. Other important parts of the convolutional layer are zero padding and the activation function. Zero padding is a technique where the input image is padded with zeros on each side

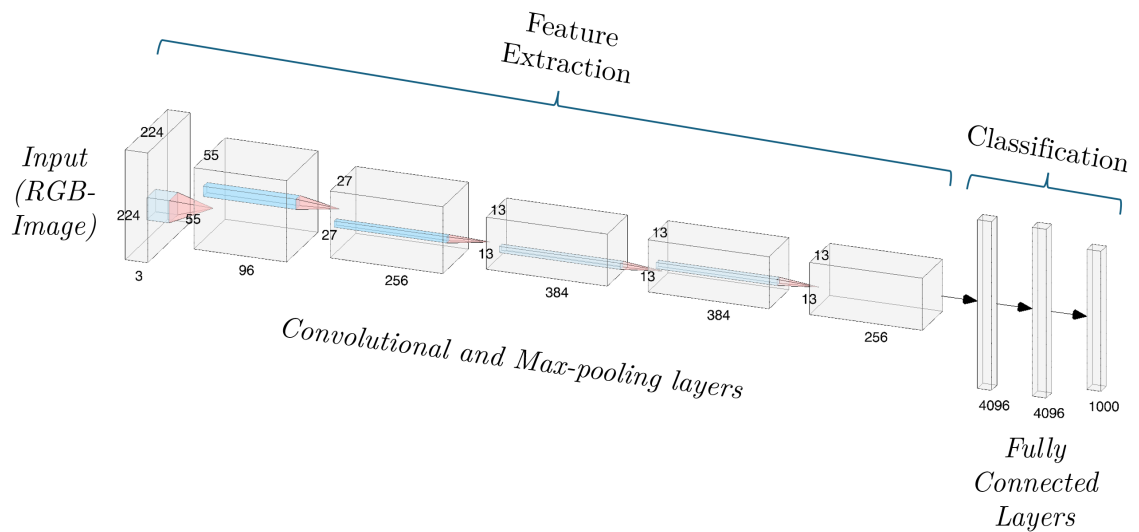


Figure 2.6. Example of a CNN architecture.

so the feature map will not get smaller with each convolution operation. At the end of the convolutional layer, the rectified linear unit (ReLU) activation function is usually used. ReLU is a nonlinear activation function that returns 0 if the input is negative and otherwise returns the input value. [67] The idea of using ReLU is to avoid vanishing gradients and to keep the activation function computationally cheap [64].

In pooling layers, feature maps are downsampled. This reduces the number of subsequent learnable parameters and increases the field of view of the feature maps, which enables the next convolutional layers to capture more complex features while enhancing earlier features. Downsampling is usually done by max pooling, where a small sliding window returns only the maximum value from a feature map patch, or by average pooling, where it returns the average value from a feature map patch. [67]

At the end of the CNN, after the features are extracted, feature maps are flattened and passed to fully connected layers, also called dense layers. This is similar to the MLP, where each neuron in the layer is connected to all neurons in the next layer, also having learnable parameters like weights. Fully connected layers are used to classify the features extracted from the image. In image classification, the output layer gives a vector of probabilities for each possible class. To scale probabilities between 0 and 1 while ensuring their sum equals 1, the softmax activation function is used in the output layer. [67] Comparing a generic CNN architecture to the first YOLO object detection model, which is also a CNN, the output layer predicts four coordinates of the bounding box, a confidence score, and the class. In YOLO, the output layer is a $7 \times 7 \times 30$ tensor, where 7×7 comes from the original image being divided into 7×7 grid cells. Each grid cell predicts 30 values, which are probabilities for 20 detectable objects and 2 bounding boxes, each having four

coordinates and a confidence score. [69]

From the perspective of feature extraction in CNNs, as data goes through deeper layers, the field of view increases, spatial information decreases, and features become hierarchically and progressively more complex and abstract [67, 70]. These layers extract different features from low- to high-level patterns that construct spatial hierarchies of features [67]. In the first layers, the extracted features from convolution are low-level features such as edges and colors. As the layers go deeper, the extracted features become mid-level features such as textures, shapes, and parts of objects, and in the end, high-level features like entire objects and structures. [71] To understand how a CNN makes decisions based on extracted features, tools such Grad-CAM [72], Feature CAM [73], or Integrative CAM [74] can be used. These tools are used to visualize which features are important for decisions in different CNN layers, helping to better understand how a CNN works. One good use case is to improve the model by examining which features of a false positive detection caused a wrong decision.

2.2.2 U-net

U-Net is a CNN architecture that was first introduced in 2015 for biomedical image segmentation [75]. Due to its ability to capture semantic information while preserving spatial information, it has become a popular architecture not only for image segmentation but also for image generation tasks. The Figure 2.7 shows the U-Net architecture.

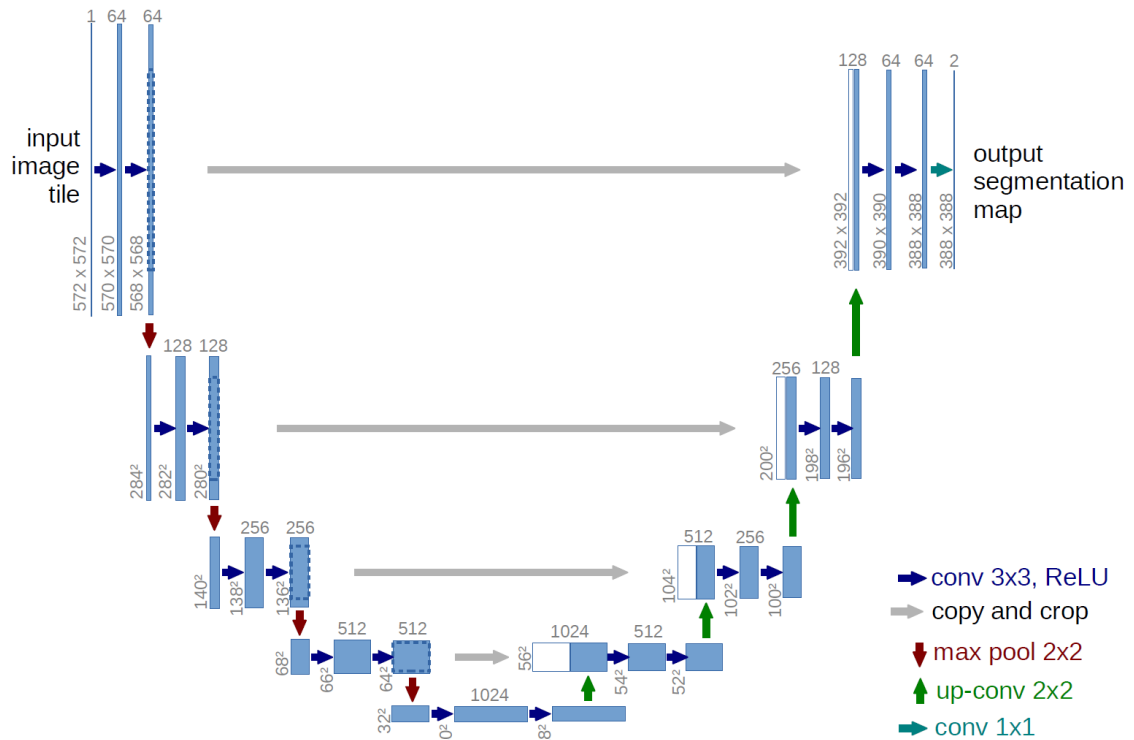


Figure 2.7. Architecture of U-Net as introduced in the paper [75].

U-Net consists of four components: encoder, bottleneck, decoder, and skip-connections [76]. The encoder is used to extract features, and its architecture is topologically identical to convolutional layers [77]. As max-pooling downsampling reduces the spatial dimensions of the image, the model can capture detailed and broad patterns, which are stored in feature maps for later use in the decoder. The bottleneck is the bottom layer of the U-shape that connects the encoder and decoder to transfer extracted features in their compressed representation. [78] The decoder reconstructs that compressed data by upsampling and using skip-connections. Skip-connections are used after each up-sampling step to concatenate the deconvolved feature maps with feature maps from the corresponding part of the encoder. [76, 78] In this way, U-Net can restore lost spatial information from downsampling and combine semantic information from the decoder with spatial information from the encoder, helping to connect features to the context [79].

2.2.3 Transformers

Several popular modern AI applications, such as the large language model (LLM) ChatGPT and text-to-image model DALL-E, utilize transformer architecture. Transformers were proposed in 2017 by Vaswani et al. [47] to address the limitations of traditional sequence-to-sequence models, whose operations are slow due to limited parallelization and which cannot capture relationships between elements in long-range dependencies, also known as the vanishing gradient problem. The main idea of transformers is to use a self-attention mechanism to capture relationships between all elements in the input sequence, which also enables parallel training. [80] In this section, the focus will be on embedding vectors, self-attention, and cross-attention mechanisms, as they are important for understanding how they are used later in the chapter discussing latent diffusion models.

A standard transformer consists of two parts: an encoder and a decoder. The encoder processes and encodes the input sequence into a hidden space, while the decoder generates the target output based on the encoded representation. These two parts can also be used independently, resulting in encoder-only and decoder-only models. Encoder-only models, like BERT, are usually applied to tasks such as classification, regression, and ranking, while decoder-only models, like ChatGPT, are used for generative tasks and language modeling. [81, 82]

At the beginning of the transformer, input data is tokenized, meaning that the input data is divided into smaller parts. Depending on the type of input data, these can be patches of images, segments of an audio signal, or parts of words [83, 84]. Each token is then converted to a numerical vector called an embedding. These vectors are denoted as $E_i \in \mathbb{R}^{1 \times D}$, where E is the index of a certain embedding and D is the dimension of the embedding vector. The dimension D is usually on the order of hundreds, but can be

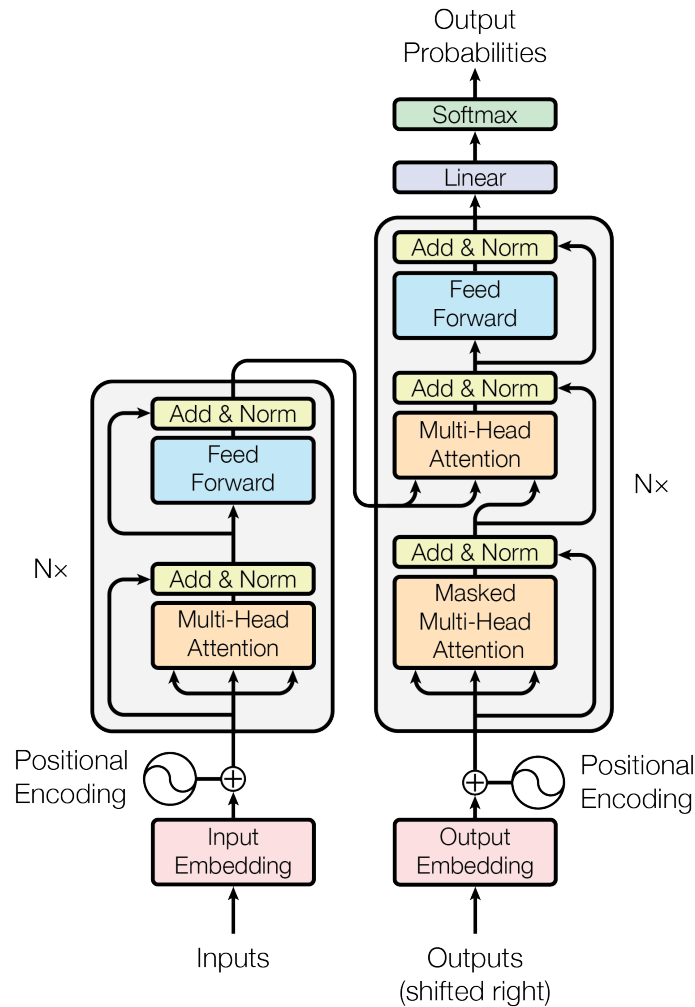


Figure 2.8. Transformer architecture introduced in by Vaswani et al. 2017 [47].

over ten thousand in large models like GPT-3. Embedding vectors capture the semantic meaning and relationships of tokens in the embedding space, which can be measured with cosine similarity. In textual data, this means that words with similar meanings, like "car" and "motorcycle," are close to each other in the embedding space. Also, the vector distance between two embeddings can capture semantic information. For example, the distance between "France" and "Paris" is very similar to the distance between "Finland" and "Helsinki," encoding the "capital of" relationship. [85] However, polysemous words like "rock," which can mean stone or music genre depending on the surrounding words, can get their contextual meaning through the attention mechanism, where the initial embedding value is adjusted based on other embeddings in the sequence.

As embeddings themselves do not have any information about their relative position in the sequence, it is necessary to insert that information into the embedding by positional encoding. In [47], a sinusoidal function is used to capture positional information, which is injected into all elements of the embedding vector. Even though this information is added together with the embedding vector, the transformer will learn what is positional

and what is semantic information in the embedding vector. There are many other ways to implement positional encoding, such as fixed or learned, and addition or concatenation, but they are outside the scope of this work. [86, 87]

The attention mechanism determines, for each embedding in the sequence, its relative importance to the other embeddings. When attention is applied to an embedding, it can increase or decrease the influence of the other embeddings, giving it contextual meaning over large distances in the sequence. In literature, it is said that the attention mechanism "attends to" other elements in the sequence, meaning it focuses on the most relevant parts of the input. [47, 88, 89] The main idea behind computing attention is based on three different vectors that are distinct representations of a single embedding vector: the query vector Q_i , the key vector K_i , and the value vector V_i [90]. In plain language, these three vectors work together to produce attention as follows: the query asks which keys have important information for it; the keys contain information that is stored in the corresponding value vector and respond to the query about how important their information is. The attention mechanism computes weights that represent the importance of each value for the query. After the query receives these importance weights, it takes a weighted average of the information from the values, producing the attention output. This basic idea is behind many different types of attention mechanisms, such as self-attention and cross-attention. Next, it will be explained how self-attention computes attention and how it differs from cross-attention.

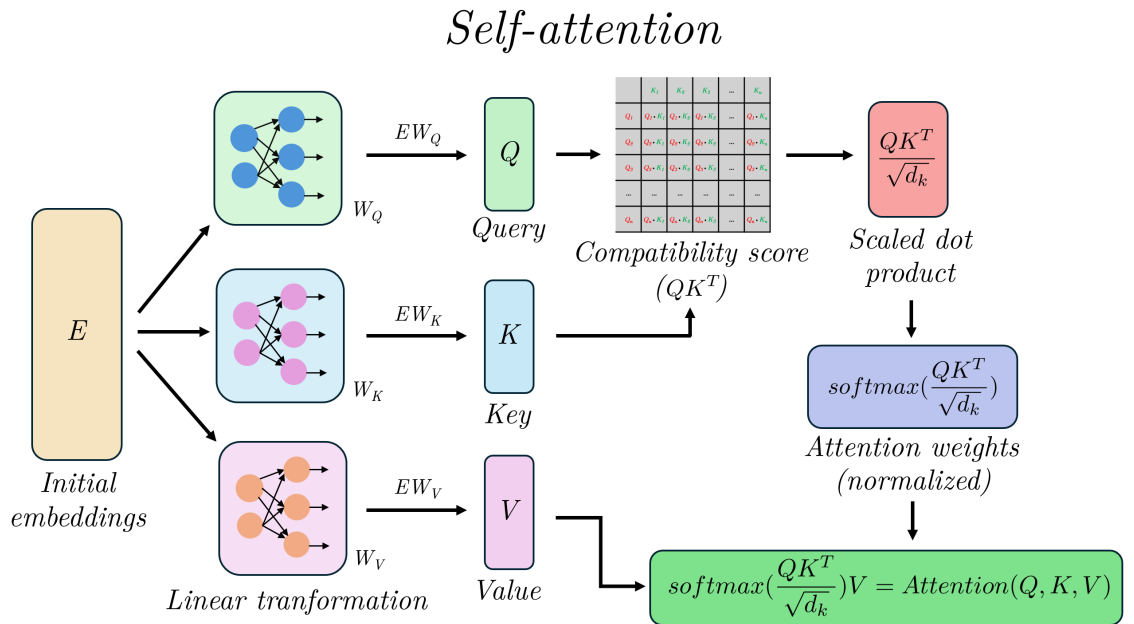


Figure 2.9. The diagram about self-attention mechanism to calculate attention.

In the self-attention mechanism, the embedding vector for each token is transformed into query, key, and value vectors. This is done by a linear transformation, i.e., multiplying the embedding vector with weight matrices W_q , W_k , and W_v assigned for the query, key and

value respectively. The weights of these matrices are learnable parameters³ that are the same for every embedding vector. [91] The role of the weight matrices in the attention mechanism is to extract relevant features from the embedding that are important for their tasks. For W_q and W_k , it is search, and for W_v , it is retrieval. [92, 93] These linear transformations for all embeddings in the sequence can be expressed as:

$$Q = EW_Q, \quad K = EW_K, \quad V = EW_V, \quad (2.7)$$

where E is the embedding matrix and Q , K and V are the query, key, and value in their matrix form. [91] Once these matrices are calculated, the query and key matrices are used to create a compatibility score, which is the dot product between the query and key matrices (QK^T). This dot product is then scaled by dividing it by the square root of the dimension of the key vector d_k . Scaling is done to prevent the dot product from becoming too large, which could lead to vanishing gradients. In the next step, attention weights (also called attention scores) are computed by normalizing with softmax. [94] The computed attention weights are then multiplied with the value matrix:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V, \quad (2.8)$$

giving attention values that show how much each embedding attends to other embeddings. In transformers, multi-head attention is usually used, where multiple self-attention layers run in parallel. In the multi-head attention, each layer has its own distinct weight matrices, producing distinct attention values that are concatenated at the end. [47]

Cross-attention is very similar to the self-attention mechanism. The most important difference between these two is that, in the cross-attention mechanism, input is taken from two different sources instead of one. In practice, in cross-attention, the key and value come from one dataset and the query comes from another dataset. This makes it possible to align two different modalities, such as image and text, speech and image, or EEG signal and speech. [95–97]

2.3 Synthetic Data in Object Detection

The motivation to use synthetic data is mainly related to the difficulties of collecting large and diverse datasets for training object detection models [98]. Common reasons for supplementing a dataset with synthetic data include an insufficient amount of real data, lack of public datasets, or the absence of pretrained models for similar use cases. In addition, capturing real data can have several challenges, such as high costs, time-consuming

³A single-layer neural network performs a linear transformation $y = Wx$ (purely linear transformation) or $y = Wx + b$ (affine linear transformation) when an activation function is not used.

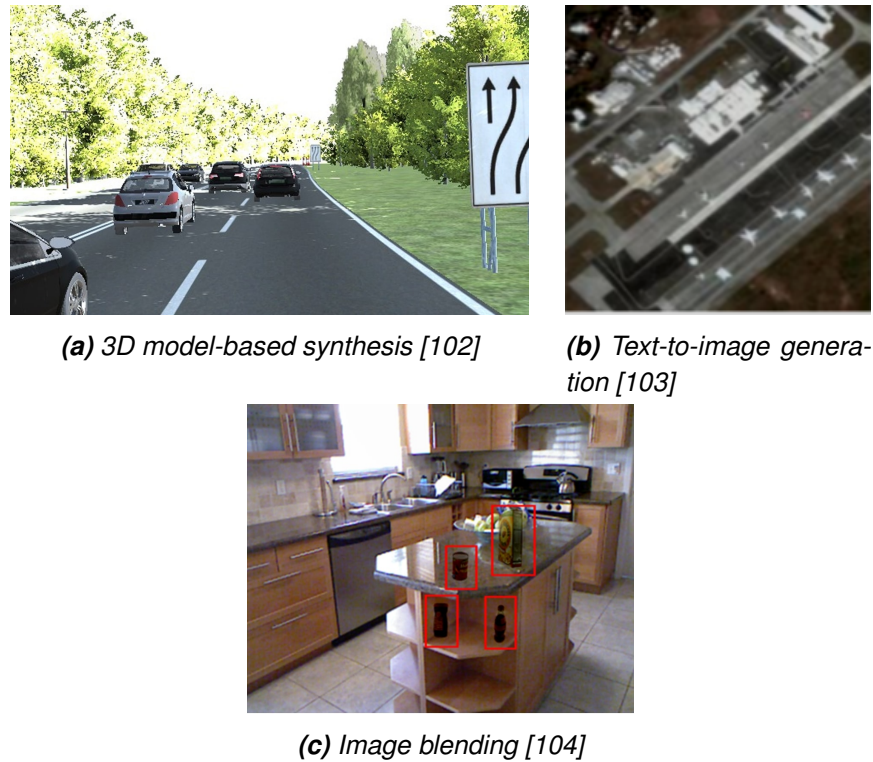


Figure 2.10. Different techniques to synthesize data for object detection training.

annotating, and privacy concerns. Synthetic data can help alleviate these challenges in training object detection models. [99–101]

The most common techniques to generate synthetic data are 3D model-based synthesis, text-to-image generation and image blending. For example, depending on the method used, synthetic data generation can quickly and cheaply produce large amounts of data and enable auto-labeling [99, 101]. Synthetic data generation methods also have the controllability to simulate rare scenarios and objects that do not appear in real datasets, making it possible to train models to detect edge cases [105, 106]. However, when only simulated or synthetic data is used to train object detection models for real-world applications, model performance is usually lower than when using real-world data for training. This happens because the input distribution has shifted between synthetic and real data. In the literature, this shift is commonly referred to as the synthetic-to-real (syn2real) or simulation-to-real (sim2real) gap. [107, 108] To close this gap, there are different approaches that can be used to improve synthetic data generation. Next, reviewed papers are introduced in Table 2.1 that are related to the syn2real problem in the object detection field, where different techniques are used to generate synthetic data.

From Table 2.1, it can be seen that most techniques achieved promising results even when using only synthetic data for training. However, some approaches performed poorly when synthetic data was used alone, likely due to a large gap between the synthetic and real domains. Despite this, all reviewed studies reported improved performance when

Table 2.1. Summary of studies focusing on training object detection with synthetic data.

Technique	Study Focus	Synthetic Image Generation Method	Performed well with synthetic data only ¹	Performed better with synthetic and real data than only using real data ²	Reference
3D simulation	Arbitrary objects for robot perception	iPhone 3D scans and NVIDIA Omniverse Isaac Sim	Yes	-	[109]
3D simulation	Traffic vehicle detection	Domain randomization using 3D models	Yes	Yes	[102]
3D simulation and Text2img	Aerial ship detection	Unity and Stable Diffusion with different domain adaptation algorithms	-	-	[6]
Image blending	Arbitrary objects in indoor scenes	Object placement and blending into real photos	No	Yes	[104]
Image blending	Milk package detection in refrigerator	Object placement and blending into real photos	No	Yes	[110]
Img2img	Aerial transmission tower and tower component detection	Synthesizing images using the AirSim simulator and Unreal Engine and adapting those to be more realistic using CycleGAN, CUT, and TSIT	Yes	Yes	[111]
Img2img	Industrial object detection	CycleGAN-based model with two generators and two discriminators	Yes	Yes	[7]
Text2img	Aerial object detection	Fine-tuning Stable Diffusion model	No	Yes	[112]
Text2img	Downstream road detection	Fine-tuning Stable Diffusion model	Yes	Yes	[103]
Text2img	Heavy machinery detection and classification	DreamBooth framework with Stable Diffusion XL (SDXL)	No	Yes	[43]
Text2img	Motorcyclists' helmet violation detection	Convolutional generative adversarial networks (DCGANs)	-	Yes	[113]
Text2img	Skin disease classification	Stable Diffusion	Yes	Yes	[4]
Text2img	Traffic vehicle detection and classification in urban applications	Kandinsky 2.2 diffusion model	-	-	[114]
Text2img	Underwater mine-like object detection	Comparison of synthetic data generation methods, including GANs and diffusion models (DDPM and DDIM)	Yes	Yes	[115]

¹ “**Performed well with synthetic data only**” indicates the performance of a model trained solely on synthetic data. (“Yes”) = comparable performance; (“No”) = poor performance with mAP typically below 10%; (“-”) = only trained with real or mixed data.

² “**Performed better with synthetic and real data than only using real data**” indicates whether mixed data improved performance compared with using real data alone. (“-”) = only trained with synthetic or mixed data.

mixed datasets were used compared to real data alone. This indicates that useful features can be extracted from synthetic data for training, even when the visual domains differ, as long as real data is also included in the training process. For example, [104] found that a

mixed dataset of 400 real and 20000 synthetic images outperformed a model trained with 5000 real images alone, while a fully synthetic training dataset produced poor results.

Across the reviewed papers, there were mainly two approaches to using synthetic data: (1) training the model only with synthetic data and then fine-tuning it with real data, or (2) training the model with both synthetic and real data simultaneously. In [116], no significant difference was found between these approaches. It was estimated that using a mixed dataset with 5%-20% real and 80%-95% synthetic data can reduce the amount of real data required without sacrificing performance [116]. However, based on the literature, the optimal ratio appears to be case-dependent, as the effectiveness of synthetic data varies by task domain and generation technique.

Among the reviewed studies, synthetic data generation techniques can be broadly grouped into four categories, each with distinct advantages and limitations.

3D simulation techniques (e.g., NVIDIA Omniverse Isaac Sim, Unity) use simulation or game engines to create synthetic environments with controllable objects, cameras, and scene configurations. The realism of the simulation and the required time depend on various factors such as the quality of 3D models and rendering techniques. Different settings, such as object positions, viewpoints, and lighting, can be randomized to automatically produce diverse images. The generated images are usually labeled or segmented during the simulation process. Creating high-quality 3D assets can be time-consuming, especially if no pre-existing models are available.

Image blending techniques create synthetic images by inserting or blending object images into real backgrounds. Objects are typically cut from other images and placed into new backgrounds after defining suitable background surfaces and scaling the objects accordingly. Similar to 3D simulation, image blending methods can also use randomization to create diverse datasets and enable automatic labeling. A common limitation is that objects do not always visually blend well with the background. This may lead to poorer model performance when trained solely on blended images likely due to visual domain gaps (e.g., lighting inconsistencies) between synthetic and real components.

Image-to-image (img2img) translation techniques use deep learning-based generative models, such as CycleGAN or diffusion models, to translate images from a synthetic domain to a more realistic one. For example, images generated using 3D simulation or image blending can be adapted to appear more realistic through these methods. Well-trained img2img models can produce high-quality and diverse images, typically improving object detection performance compared to using only synthetic data. The required workload depends on whether suitable synthetic images already exist or need to be created first.

Text-to-image (txt2img) generation techniques are similar to img2img methods, but

instead of using synthetic images as input, they are trained on datasets containing images paired with textual descriptions. This allows the generation of a wide variety of images using text prompts, enabling control over factors such as object type, lighting, and quantity. By fine-tuning a pre-trained base model trained on large-scale datasets, it is possible to generate images that closely resemble a target real-world dataset. A drawback is that the generated images usually require manual annotation to label objects for object detection training.

Overall, text-to-image methods, such as Generative Adversarial Networks (GANs) and diffusion models, have shown the greatest potential for generating realistic and diverse data for object detection. A systematic review by [117] similarly concluded that these models have strong potential to overcome sim2real problems.

For this work, it was decided to use diffusion-based text-to-image generation, specifically Stable Diffusion XL (SDXL), to generate synthetic training data for object detection model development, for the following reasons:

- **High-fidelity image generation:** SDXL can effectively learn the data distribution of real datasets and generate images with similar visual characteristics.
- **Controllability:** The model enables control of image content through text prompts, allowing targeted dataset creation. Prompts can also be randomized to increase diversity in the generated data.
- **Practical advantages:** SDXL is an open-source text-to-image model based on an improved version of Stable Diffusion v1.5 and is supported by fine-tuning tools. It also offers relatively fast image generation, depending on the GPU used, and has a large community providing numerous guides and tips.
- **Rapid model evolution:** Text-to-image models have evolved significantly in a short time and continue to improve, making them strong candidates for future synthetic data generation.
- **Integration with image-to-image tools:** SDXL can be adapted with techniques such as ControlNet (see Section 2.5.2) to modify or guide generation according to input images. This can be useful for future works such as transforming standard photos into sensor-like images.

In following sections provide more information about text-to-image models, diffusion models, Stable Diffusion, and fine-tuning with Low-Rank Adaptation (LoRA).

2.4 Text-To-Image Models

Text-to-image (T2I) models generate images from textual descriptions, combining the fields of computer vision and natural language processing [118, 119]. The main goal

of these models is to create new, high-quality images, rather than retrieving existing ones using textual conditioning. Modern T2I models are capable of producing images with strong text-image consistency and can handle complex prompts by large-scale training on millions of image-text pairs. [119]

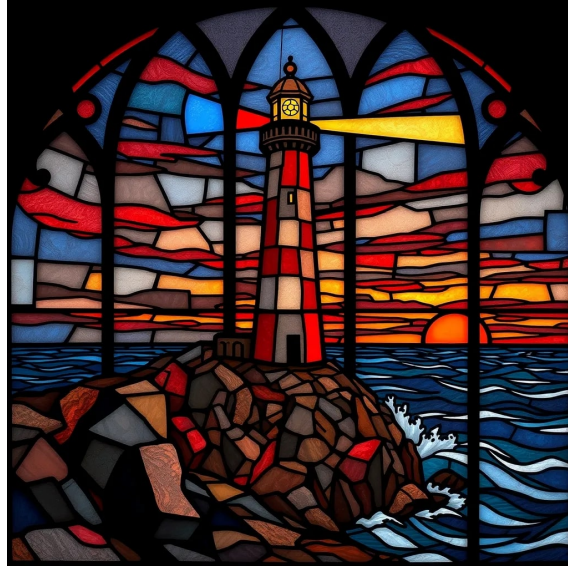


Figure 2.11. Generated image from FLUX.1 [schnell] using prompt "Gothic stained-glass of a lighthouse standing at the end of a rocky coast beside a wavy ocean under a red dawn sky."

The modern development of T2I models began in 2015 with AlignDRAW, which demonstrated the relationship between text input and generated images using attention mechanisms, although the results were not very realistic [118, 119]. After 2015, T2I made a great leap forward with Generative Adversarial Networks (GANs), leading to better image generation. In 2021, OpenAI introduced DALL-E, an autoregressive (AR) model that used transformer architecture and large-scale datasets to generate diverse, high-quality images, even though it suffered from high computational costs.[119] In recent years, diffusion models (DM), such as DALL-E 2 and FLUX.1, have become the leading approach in T2I generation. Latent diffusion models (LDM), which are used for example in the Stable Diffusion models, not only improve the quality of generated images but also reduce computational costs by operating in latent space instead of pixel space.[118, 119] There are still many challenges in T2I models, such as fine-grained control, contextual understanding, or accurately capturing spatial relationships described in text prompts [119].

This work will next focus on diffusion-based T2I models including latent diffusion models and stable diffusion XL. Further details on GANs or AR models can be found in [119].

2.5 Diffusion Models and Stable Diffusion

This section is divided into two main parts. The first two subsections present the theoretical foundations of diffusion-based text-to-image models, while the later subsections focus on Stable Diffusion, a latent diffusion model, and its fine-tuning with Low-Rank Adaptation. Since the training of latent diffusion models is heavily based on Denoising Diffusion Probabilistic Models (DDPM), the first subsection focuses on this perspective, based mainly on [120, 121], while excluding other diffusion models such as Denoising Diffusion Implicit Models (DDIM).

2.5.1 Diffusion Models

Diffusion models (DM), also known as diffusion probabilistic models, are a type of generative model that uses a Markov chain to convert a distribution into the target data distribution through a diffusion process. In this context, that means converting an original complex image distribution to a simple standard Gaussian distribution and vice versa. In the forward process (also known as the diffusion process), Gaussian noise is added iteratively to the data at each step until the data is completely noisy. The goal of diffusion models is to learn how to reverse the forward process for sample generation, gradually reducing noise with each step. [121, 122] The illustration of forward and reverse processes are shown in Figure 2.12.

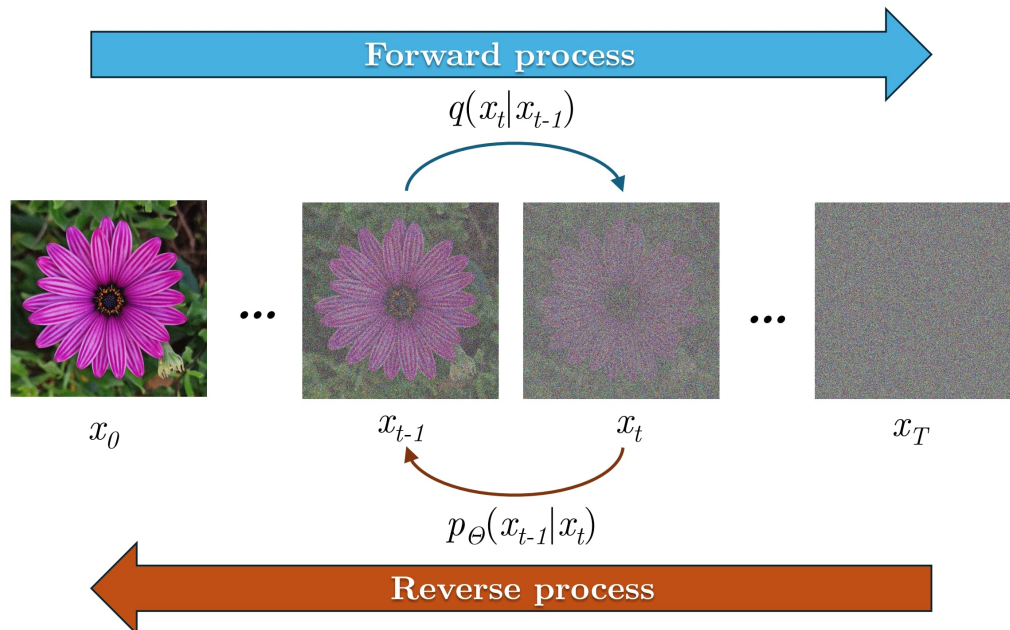


Figure 2.12. Forward and reverse processes in a diffusion model.

Forward process

The forward process starts by providing a clean input data sample x_0 . This continues

by adding standard Gaussian noise $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ at each time step, resulting in a noised sample $\mathbf{x}_t$ at time step t . The forward process ends at the last time step T , where the data is completely Gaussian noise. [121] The amount of Gaussian noise added to the sample at each step is determined by the variance schedule $\beta_1, \dots, \beta_T$, where the forward process variance β_t (also referred to as the diffusion rate) can be precomputed, for example, by using a cosine noise scheduler, or can be learned [78, 123]. Entire forward process from $\mathbf{x}_0$ to $\mathbf{x}_T$ can be written as joint probability

$$q(\mathbf{x}_{1:T} | \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1}), \quad (2.9)$$

where q denotes the probability distribution of the forward process. The noise added gradually at each step in 2.9 can be written as

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = K(\mathbf{x}_t | \mathbf{x}_{t-1}; \beta_t), \quad (2.10)$$

where K is the Markov diffusion kernel, i.e. the sample $\mathbf{x}_t$ is generated from the previous sample $\mathbf{x}_{t-1}$. This also shows that as each diffusion step depends only on the previous step, the process is Markovian. [121]

When a Gaussian Markov diffusion kernel is used, the process is referred to as Gaussian diffusion. In this case, each forward process step is defined as a Gaussian distribution, and 2.10 becomes

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}), \quad (2.11)$$

where in the Gaussian distribution $\mathcal{N}$ the output is $\mathbf{x}_t$, the mean is $\sqrt{1 - \beta_t} \mathbf{x}_{t-1}$, and the covariance matrix is $\beta_t \mathbf{I}$. [121, 122]

To obtain the closed-form of equation 2.11, let's introduce the reparametrization trick, where a Gaussian distribution $\mathbf{z} \sim \mathcal{N}(\mathbf{z}; \mu, \Sigma)$ can be defined as:

$$\mathbf{z} = \mu + \mathbf{A}\epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}), \quad (2.12)$$

where $\mathbf{A}$ is such that $\Sigma = \mathbf{A}\mathbf{A}^T$. By using the reparametrization trick, equation 2.11 can be written as

$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \epsilon_t. \quad (2.13)$$

By defining signal retention rate $\alpha_t := 1 - \beta_t$ and $\bar{\alpha}_t := \prod_{s=1}^t \alpha_s$, equation 2.13 can be derived to the following closed-form:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad (2.14)$$

which can also be presented in a conditional Gaussian distribution form as

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}). \quad (2.15)$$

As seen in equation 2.14, $\mathbf{x}_t$ depends only on $\mathbf{x}_0$, so the sample $\mathbf{x}_t$ can be calculated directly, which makes the forward process fast. The detailed derivation from equation 2.13 to equation 2.14, as well as other steps of the forward process, can be found in [121].

Reverse Process

To sample a clean image $\mathbf{x}_0$ from noisy data, the reverse process is required to gradually remove noise from a sample $\mathbf{x}_t$, producing reconstructed data samples. Like the forward process, the reverse process is a Markov chain, and the joint distribution of the whole reverse process can be written as

$$p_{\theta}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t), \quad (2.16)$$

where p is the probability distribution of the reverse process and θ are the parameters of the neural network, as the reverse process needs to be learned. This is because $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ cannot be used for the noise reversal process guiding $\mathbf{x}_T$ back to $\mathbf{x}_0$, as it is intractable. This can be seen by using Bayes' rule to calculate $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ from $q(\mathbf{x}_t | \mathbf{x}_{t-1})$.⁴ As $q(\mathbf{x}_{t-1})$ and $q(\mathbf{x}_t)$ are not accessible, expanding these terms requires representing and integrating over the distribution $q(\mathbf{x}_0)$. That is not feasible, as the distribution of the original sample $\mathbf{x}_0$ is not assumed to be Gaussian but a complex distribution. This concludes that $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ cannot be calculated directly. [121]

Nonetheless, it is possible to approximate $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ by defining a new distribution to represent the reverse process. As the reverse step is also assumed to be Gaussian like the forward step, it can be written as

$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t), \boldsymbol{\Sigma}_{\theta}(\mathbf{x}_t, t)). \quad (2.17)$$

Based on this, to learn equation 2.17 and thus produce the reverse step, it is necessary to estimate the mean and variance. In practice, as this work follows the implementation of DDPM from paper [120], where the forward process variance β_t is fixed, only the mean

⁴ $q(\mathbf{x}_{t-1} | \mathbf{x}_t) = \frac{q(\mathbf{x}_t | \mathbf{x}_{t-1})q(\mathbf{x}_{t-1})}{q(\mathbf{x}_t)}$

$\mu_\theta(\mathbf{x}_t, t)$ needs to be learned. In the referenced paper, it was empirically found that fixing the reverse-process variance as $\Sigma_\theta(\mathbf{x}_t, t) = \sigma_t^2 \mathbf{I}$ gives similar results whether the forward process variance $\sigma_t^2 = \beta_t$ or the forward process posterior variance $\sigma_t^2 = \tilde{\beta}_t$ is used (the latter will be derived later from the forward-process posterior $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$). Because of this, it was not specified separately whether variance was used for sampling algorithm at the end of this sections.

The next objective is to define the loss function to train the neural network to estimate the mean and then use that to derive the equation to sample $\mathbf{x}_{t-1}$ from $\mathbf{x}_t$. To define the loss function, it is intuitive to maximize the likelihood of $\mathbf{x}_0$, as generative models usually learn data distributions from training data. However, it is not possible to maximize the likelihood $p_\theta(\mathbf{x}_0)$ directly, as calculating it involves integrating over all trajectories from $\mathbf{x}_T$ to $\mathbf{x}_0$, which is intractable. To circumvent this problem, it is possible to optimize an evidence lower bound (ELBO, also known as the variational lower bound) of the log likelihood⁵, which is tractable. [121]

Equations after this point will follow paper [120], which minimizes the ELBO of negative log likelihood instead of maximizing it.⁶

The loss function L is defined by minimizing the ELBO of the log likelihood as

$$L := \mathbb{E}[-\log p_\theta(\mathbf{x}_0)] \leq \mathbb{E}_q \left[-\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right], \quad (2.18)$$

where $\mathbb{E}$ is the expectation operator. Next, the full derivation of the loss presented in [120] is omitted due to its length, and only the resulting loss function is presented:

$$L = \mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T | \mathbf{x}_0) \| p(\mathbf{x}_T))}_{L_T} + \sum_{t>1} \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t))}_{L_{t-1}} - \underbrace{\log p_\theta(\mathbf{x}_0 | \mathbf{x}_1)}_{L_0} \right], \quad (2.19)$$

⁵It is usually easier to work with log likelihood instead of maximizing likelihood, as it has various good properties such as being a monotonically increasing function, and maximizing the log also means maximizing the likelihood. Also, log likelihood turns multiplications into additions, making derivatives and sums easier to calculate.

⁶While this work uses negative log likelihood, some papers may use positive log likelihood instead, like in paper [122]. This is because maximizing log likelihood is equivalent to minimizing negative log likelihood. In that case, equation 2.18 becomes $L := \mathbb{E}[\log p_\theta(\mathbf{x}_0)] \geq \mathbb{E}_q \left[\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right]$

where D_{KL} is the Kullback-Leibler divergence⁷. In expectation, the loss is divided into three terms: the prior matching term L_T , the consistency term L_{t-1} , and the reconstruction term L_0 . In [120], the term L_T is treated as a constant because $p(\mathbf{x}_T)$ is just Gaussian noise and a fixed forward process variance β_t is used. As a result, the forward process distribution q has no learnable parameters. The term L_0 is also ignored, as empirical testing showed that leaving it out not only simplifies the implementation of the loss function but also improves sample quality. This leaves only L_{t-1} to be optimized.

To expand L_{t-1} , the forward process posterior $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ needs to be defined as a Gaussian distribution. It can be calculated using Bayes' rule:

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{x}_0) q(\mathbf{x}_{t-1} | \mathbf{x}_0)}{q(\mathbf{x}_t | \mathbf{x}_0)}. \quad (2.20)$$

By utilizing the Markov property of the entire forward process $q(\mathbf{x}_{1:T}|\mathbf{x}_0)$ ⁸, it is obtained that $q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) = q(\mathbf{x}_t|\mathbf{x}_{t-1})$, which is expressed in Gaussian form in equation 2.11. Additionally, the Gaussian forms of $q(\mathbf{x}_{t-1} | \mathbf{x}_0)$ and $q(\mathbf{x}_t | \mathbf{x}_0)$ can be defined from equation 2.15, which now gives

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \frac{\mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}) \mathcal{N}(\mathbf{x}_{t-1}; \sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0, (1 - \bar{\alpha}_{t-1})\mathbf{I})}{\mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})}. \quad (2.21)$$

Thus, $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ can be expressed in Gaussian form as

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t\mathbf{I}), \quad (2.22)$$

with

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}\mathbf{x}_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\mathbf{x}_t; \quad \tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\beta_t, \quad (2.23)$$

where $\tilde{\boldsymbol{\mu}}_t$ is the forward process posterior mean and $\tilde{\beta}_t$ is the posterior variance.

Since now both $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ in equation 2.17 and $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ in equation 2.22 have Gaussian forms, the KL divergence between two multivariate Gaussian distributions can be calculated in L_{t-1} , resulting in

$$L_{t-1} = \mathbb{E}_q \left[\frac{1}{2\sigma_t^2} \|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_\theta(\mathbf{x}_t, t)\|^2 \right] + C, \quad (2.24)$$

⁷ $D_{\text{KL}}(P\|Q) = \sum P(x) \log \frac{P(x)}{Q(x)}$

⁸ $q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1}) = q(\mathbf{x}_T|\mathbf{x}_0) \prod_{t=2}^T q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$

where C is a constant that does not depend on learnable parameters.

By using $\mathbf{x}_t(\mathbf{x}_0, \epsilon)$ as the parametrized form of $\mathbf{x}_t$ from equation 2.14 and the posterior mean $\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0)$, L_{t-1} can be expanded to

$$L_{t-1} = \mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{1}{2\sigma_t^2} \left\| \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t(\mathbf{x}_0, \epsilon) - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon \right) - \mu_\theta(\mathbf{x}_t(\mathbf{x}_0, \epsilon), t) \right\|^2 \right] + C. \quad (2.25)$$

From this equation, it can be seen that the learnable mean μ_θ needs to predict $\frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t(\mathbf{x}_0, \epsilon) - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon \right)$ given $\mathbf{x}_t$ to be close to zero. Now, the parametrized μ_θ with known $\mathbf{x}_t$ can be defined as

$$\begin{aligned} \mu_\theta(\mathbf{x}_t, t) &= \tilde{\mu}_t \left(\mathbf{x}_t, \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t} \epsilon_\theta(\mathbf{x}_t) \right) \right) \\ &= \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right). \end{aligned} \quad (2.26)$$

By substituting equation 2.26 into 2.25, the loss can be simplified to

$$\mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1-\bar{\alpha}_t)} \left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t} \epsilon, t) \right\|^2 \right]. \quad (2.27)$$

In [120], it was shown empirically that leaving the weighting term in the loss improves results. This gives the most simplified training objective function of the reverse process:

$$\begin{aligned} L(\theta) &= \mathbb{E}_{\mathbf{x}, \epsilon \sim \mathcal{N}(0,1), t} \left[\left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t} \epsilon, t) \right\|^2 \right] \\ &= \mathbb{E}_{\mathbf{x}, \epsilon \sim \mathcal{N}(0,1), t} \left[\left\| \epsilon - \epsilon_\theta(\mathbf{x}_t, t) \right\|^2 \right] \end{aligned} \quad (2.28)$$

which is the mean squared error (MSE) between the true noise ϵ and the model-predicted noise $\epsilon_\theta(\mathbf{x}_t, t)$, conditioned on the noisy sample $\mathbf{x}_t$ and time step t .

Training and sampling DDPM

By using the simplified loss function 2.28, it is possible to train a neural network to predict noise $\epsilon_\theta(\mathbf{x}_t, t)$. In the DDPM training algorithm, this is done by sampling a random image $\mathbf{x}_0$ from the training dataset, adding noise to it via the forward process, and then training the model to predict the noise added to the image by optimizing the loss function 2.28 with gradient descent. This training procedure is shown in Algorithm 1. [120]

Algorithm 1 Training DDPM [120]

-
- ```

1: repeat
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ ▷ Sample image $\mathbf{x}_0$ from training dataset
3: $t \sim \text{Uniform}(\{1, \dots, T\})$ ▷ Sample random timestep t
4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ▷ Sample Gaussian noise ϵ
5: Take gradient descent step on

```

$$\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta} \left( \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t \right) \right\|^2$$

▷ Compute loss to optimize model

- ```

6: until converged

```
-

In practice, the neural network used in diffusion models is usually a U-Net architecture including residual convolution layers, self-attention layers, and cross-attention layers. This U-Net architecture is suitable for diffusion models as it is capable of accurate segmentation, context-aware processing, feature extraction, and preserving image size for input and output.[124]

To sample $x_{t-1} \sim p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ with a trained model, it is necessary to define the sampling step from equation 2.17. As the mean is defined in equation 2.26 and the variance is set to a fixed value, it is possible to define the equation for 2.17 by using the reparametrization trick, resulting in the following equation:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}, \quad (2.29)$$

where $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. [120]

In the DDPM sampling algorithm, $\epsilon_{\theta}(\mathbf{x}_t, t)$, the total noise between $\mathbf{x}_0$ and $\mathbf{x}_t$, is predicted, and then a fraction of the predicted noise is subtracted from $\mathbf{x}_t$. Before sampling $\mathbf{x}_{t-1}$, noise $\sigma_t \mathbf{z}$ is added for stochasticity, where $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ [78]. These steps are performed for each time step t from T to 1, and at the final step, Gaussian noise $\mathbf{z}$ is set to zero, returning $\mathbf{x}_0$ as the clean sample. [120]

Algorithm 2 Sampling DDPM [120]

```

1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  ▷ Sample noisy image
2: for  $t = T$  to 1 do ▷ Go through all time steps from  $T, T - 1, \dots, 1$ 
3:   if  $t > 1$  then
4:      $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:   else
6:      $\mathbf{z} = \mathbf{0}$ 
7:   end if
8:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$  ▷ Do noise prediction and return less  
noisy image  $\mathbf{x}_{t-1}$ 
9: end for
10: return  $\mathbf{x}_0$  ▷ Return clean sample image  $\mathbf{x}_0$ 

```

In the next section, it will be explained how textual conditioning is added to the diffusion model to control the image generation process by textual input.

2.5.2 Conditioning

Conditioning in machine learning is a process in which the computation of the model is conditioned or modulated by additional information [125]. In image synthesis tasks such as text-to-image synthesis, image restoration, or image composition, the process is guided by conditional input. A typical conditioning process in conditional image synthesis proceeds as follows: conditional inputs (e.g., text, depth maps, segmentation masks) are encoded into feature embeddings using a condition encoder, and then the encoded conditional inputs are injected into the model through an injection mechanism. [124]

In text-to-image generation, the main approach to implementing the conditioning mechanism is to use guidance methods. The two main guidance methods are classifier guidance and classifier-free guidance, with the latter having become the main approach in many image generation models. [124] Next, it will be explained how these two guidance methods affect the training and sampling of diffusion models. The CLIP model is also introduced to show how conditioning input is encoded and injected in guidance, and T2I-Adapter ControlNet demonstrates how additional conditioning inputs can be added to pretrained text-to-image diffusion models.

Guidance methods The main idea of classifier guidance is to use a separately trained classifier that nudges the image generation in the direction that matches the class labels [121]. As the classifier is separately trained, it can be used for pretrained diffusion model image sampling without joint training. In the diffusion sampling process, the classifier estimates the probability $p_\phi(\mathbf{y} \mid \mathbf{x}_t)$ of how much the noisy image $\mathbf{x}_t$ matches class $\mathbf{y}$ and provides the gradient $\nabla_{\mathbf{x}_t} \log p_\phi(\mathbf{y} \mid \mathbf{x}_t)$ to guide sampling. The effect of the gradient

can be amplified or attenuated by the gradient scale s , where a larger value gives higher fidelity but less diverse samples. [126]

In classifier-free guidance, the conditioning is trained inside the U-Net. The idea is that instead of learning the estimator $\epsilon_\theta(\mathbf{x}_t, t)$ in equation 2.28, the model learns the conditional model $\epsilon_\theta(\mathbf{x}_t, \mathbf{y}, t)$, where it receives the semantic embedding of the captions $\mathbf{y}$. During training, the semantic caption is sometimes randomly replaced with an empty caption $\emptyset$ with a fixed probability, making the model also learn the unconditional estimator $\epsilon_\theta(\mathbf{x}_t, \emptyset, t)$. In the sampling process, instead of using $\epsilon_\theta(\mathbf{x}_t, t)$ in noise prediction of Algorithm 2, it uses the guided estimator $\tilde{\epsilon}_\theta(\mathbf{x}_t, \mathbf{y}, t)$ defined as:

$$\tilde{\epsilon}_\theta(\mathbf{x}_t, \mathbf{y}, t) = \epsilon_\theta(\mathbf{x}_t, \mathbf{y}, t) + s(\epsilon_\theta(\mathbf{x}_t, \mathbf{y}, t) - \epsilon_\theta(\mathbf{x}_t, \emptyset, t)) \quad (2.30)$$

where s is the gradient scale, similar to classifier guidance. [121, 127] To enable more guided sampling, negative prompts can be used to avoid generating unwanted features in the image. This changes the sampling process so that instead of the unconditional estimator $\epsilon_\theta(\mathbf{x}_t, \emptyset, t)$ in 2.30, the estimator of the negative prompt $\epsilon_\theta(\mathbf{x}_t, \mathbf{y}_{neg}, t)$ is used, where $\mathbf{y}_{neg}$ is the embedding of the negative prompt [128].

CLIP To encode text captions and prompts for conditioning, a text encoder is required. Many modern text-to-image models use the CLIP (Contrastive Language-Image Pretraining) text encoder or a similar architecture to encode prompts into semantic embeddings. CLIP is a pretrained multimodal vision and language model by OpenAI that enables capturing the relationship between images and their corresponding textual descriptions. It is composed of an image encoder and a text encoder (both typically transformer-based), which can be used together or separately. [95]

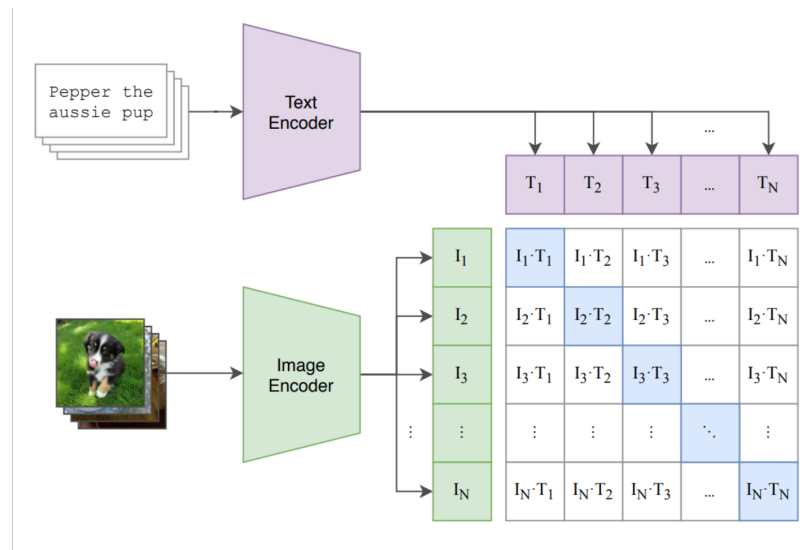


Figure 2.13. Illustration of the contrastive pretraining process, where corresponding batches of images and their textual descriptions are jointly trained to predict correct pairs [95].

In Figure 2.13, batches of corresponding images and their textual descriptions are encoded into embedding vectors. During training, text-image embedding pairs are arranged in a matrix such that the corresponding pairs are located on the diagonal elements, while the incorrect pairs are placed in the off-diagonal elements. The training objective to predict correct text-image pairings is to maximize the cosine similarity of the diagonal elements while minimizing the similarity of the off-diagonal elements. As a result, training creates a multi-modal embedding space for images and texts, making it possible to create applications that can describe textually what is in the image or control image generation with text prompts. [95]

In text-to-image models, the CLIP text encoder operates similarly as transformers described in section 2.2.3. The text encoder encodes the whole caption into a sequence of tokens and then embeddings. Embeddings in the sequence are updated via self-attention to have contextualised information between other embeddings. These contextualized embeddings are then injected into the U-net architecture of the diffusion model via cross-attention layers, allowing image generation to be conditioned by textual information from the caption. Additionally, the U-Net might contain self-attention layers that preserve spatial information, as it helps each pixel to see the relevance of other pixels in the image [129, 130].

ControlNet To add various types of conditioning to already trained text-to-image diffusion models, there are T2I-Adapters like ControlNet. ControlNet is a neural network architecture that can adapt the T2I backbone to sample images with an additional control signal. The main idea of training ControlNet is to copy all deep encoding layers in the pretrained model's U-Net and connect them with "zero convolution" layers to prevent noise that can disturb training. ControlNet can be extended to pretrained text-to-image diffusion models to add extra conditioning like edge maps, sketches, or depth maps alongside the text prompt. [131]

2.5.3 Stable Diffusion

Stable Diffusion is an open-source text-to-image model developed by the CompVis group at Ludwig Maximilian University of Munich, in collaboration with Runway and Stability AI, released in 2022. Stable Diffusion is also known as a latent diffusion model (LDM), which is a type of diffusion model that operates on images in latent space rather than pixel space. [132, 133] Latent space is a compressed representation of data that retains important features while discarding irrelevant details [134]. This enables faster and more efficient model training while focusing on important data features compared to implementations that operate directly in pixel space [132].

The first models of Stable Diffusion, also known as the 1.x series (versions 1.1–1.5, released in 2022), were trained on cropped images at a resolution of 512x512 from subsets

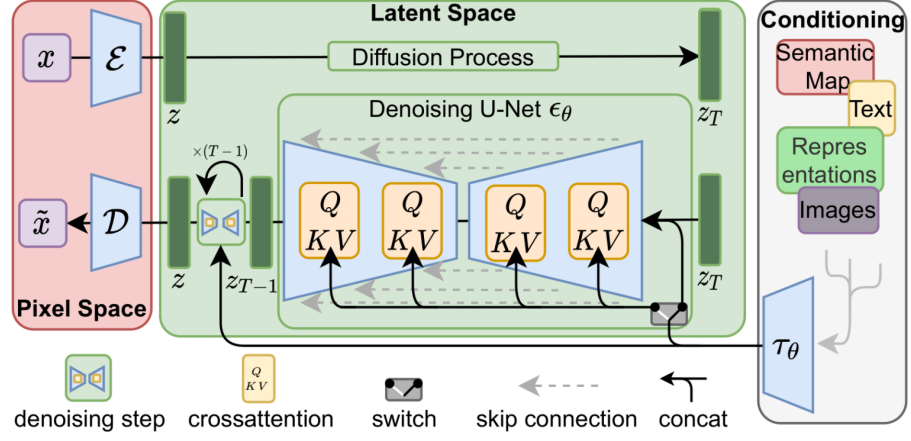


Figure 2.14. Model architecture of latent diffusion model introduced in [132], illustrating the main components' roles in both training and sampling.

of the LAION-5B database [135] and used a frozen CLIP ViT-L/14 text encoder [136] for conditioning. The network architecture integrates a U-Net with 860M parameters and a text encoder with 123M parameters, making it lightweight and capable of running on a single GPU with 10GB of VRAM. [133] Later generations, including the 2.x series (end of 2022), SDXL (July 2023), Cascade (February 2024), and Stable Diffusion 3 (June 2024), have made several improvements compared to the first releases [137]. However, as the implementation and measurements for this work were completed in the middle of 2024, the work was implemented using the SDXL model, as it was one of the most recent models. Therefore, this section will focus on the architecture of the latent diffusion model used in the 1.x series, while the next section will discuss how SDXL is improved from the 1.x series.

The latent diffusion model has three main components: the variational autoencoder (VAE), U-Net, and a text encoder [132]. The VAE is a CNN that is trained to encode RGB images to a lower-dimensional latent space with an encoder and to decode the latent space back to RGB images with a decoder $\mathcal{D}$ [138]. During training, the input image x is encoded to latent space z using the VAE encoder $\mathcal{E}$:

$$z = \mathcal{E}(x). \quad (2.31)$$

In the paper [132], $x \in \mathbb{R}^{H \times W \times 3}$, a three-channel RGB image with height H and width W , is encoded to $z \in \mathbb{R}^{H/8 \times W/8 \times 4}$ latent image. As the latent space is 48 times smaller than the pixel space, it is more efficient for processing.

Latent images are noised with T steps in a fixed diffusion process. The noised latent image z_T is passed to the U-Net, which is trained to predict noise ϵ_θ . [132]

For conditioning the latent diffusion model, the domain-specific encoder τ_θ encodes the

text caption y of the training image to an embedding $\tau_\theta(y)$, which is injected into the U-Net's intermediate layers via cross-attention. Cross-attention is calculated using the following query, key, and value matrices:

$$Q = W_Q^{(i)} \cdot \varphi_i(\mathbf{z}_t), \quad K = W_K^{(i)} \cdot \tau_\theta(y), \quad V = W_V^{(i)} \cdot \tau_\theta(y), \quad (2.32)$$

where $\varphi_i(\mathbf{z}_t)$ is a flattened intermediate representation of the U-Net. [132]

The conditional model is learned by minimizing the following loss function:

$$L_{LDM} := \mathbb{E}_{\mathcal{E}(\mathbf{x}), y, \epsilon \sim \mathcal{N}(0,1), t} [\|\epsilon - \epsilon_\theta(\mathbf{z}_t, t, \tau_\theta(y))\|_2^2]. \quad (2.33)$$

As seen from equation 2.33, $\tau_\theta(y)$ can be optimized together with ϵ_θ , making the conditioning mechanism flexible and allowing it to be parametrized with different domain-specific experts [132]. However, in Stable Diffusion 1.x, the text encoder is kept frozen during training [133].

Training proceeds similarly to the DDPM algorithm but uses the latent image $\mathbf{z}$ instead of the image $\mathbf{x}$ and equation 2.33 in the gradient descent step. [132]

In the sampling process, the encoded text prompt τ is used to generate a latent image from pure noise $\mathbf{z}_T$ using $\epsilon_\theta(\mathbf{z}_t, t, \tau_\theta(y))$ in the noise prediction step. This denoising is processed in $T - 1$ steps until it produces the latent image $\mathbf{z}$. However, compared to training where Stable Diffusion uses DDPM, for sampling it is possible to select a different algorithm like DDPM, DDIM, or PLMS [133]. The latent space is decoded to the image space using the decoder $\mathcal{D}$, which produces the reconstructed image $\tilde{\mathbf{x}}$ as follows:

$$\tilde{\mathbf{x}} = \mathcal{D}(\mathbf{z}), \quad (2.34)$$

where $\tilde{\mathbf{x}}$ is the reconstructed RGB image. [132]

2.5.4 SDXL

SDXL is an improved version of the previous Stable Diffusion models, released in 2023. Its main characteristics are generating images with higher resolution and better quality, as well as improved prompt adherence and composition. Compared to the architecture of Stable Diffusion 1.x and 2.x, SDXL has three times more parameters in the U-Net (2.6B), a heterogeneous distribution of transformer blocks, and uses two different text encoders (CLIP ViT-L and OpenCLIP ViT-bigG) instead of one. To generate higher-resolution images, SDXL uses a latent space that is twice as large (128x128) compared to the 1.x series, resulting in a native resolution of 1024x1024 pixels instead of 512x512 pixels. In

addition, while the pretrained SDXL model was trained on 1024x1024 images, the base model was further fine-tuned with multi-aspect training. For fine-tuning, data buckets with different aspect ratios (e.g., 1024x1024, 1152x896, 1216x832, 1344x768, 1536x640) were used, each having approximately the same pixel count as 1024x1024 with height and width constrained to multiples of 64. [139]

SDXL has single- and two-stage versions: the SDXL base model and SDXL with refiner. In the single-stage version, the SDXL base model is used as a stand-alone module to generate images, whereas the two-stage version uses the refiner for fine-tuning details of the base model's generated latents before decoding them to the final image. The visualization of SDXL with refiner can be seen in Figure 2.15. [139]

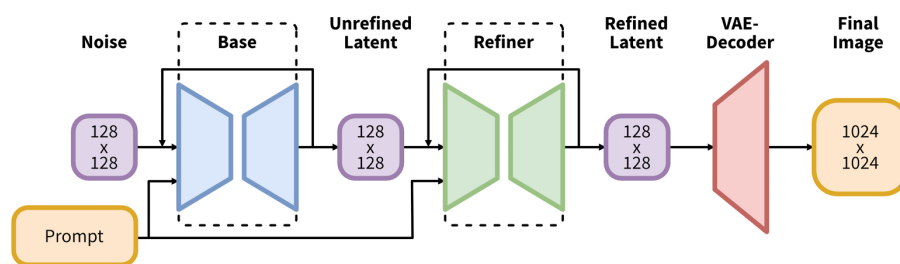


Figure 2.15. Illustration of the two-stage SDXL pipeline. The one-stage model is the same, but without the refiner and its output refined latent. [139]

There are some limitations to SDXL. These include synthesizing intricate structures like human hands and not attaining the nuances that some photorealistic images have, such as lighting effects. There are also sometimes prompt-related issues like concept bleeding and difficulties when generating images from long, legible text. It is also mentioned that training the model with large-scale datasets can introduce biases, which may be reflected in the generated images. [139]

2.5.5 LoRA

Low-Rank Adaptation (LoRA) is a Parameter-Efficient Fine-Tuning (PEFT) method that can adapt large pretrained models by training and inserting only a small number of parameters [140–142]. As is typical for PEFT methods, LoRA provides a low-memory solution with no additional inference latency, allowing models to be fine-tuned quickly with significantly less GPU memory and computational power during training compared to full fine-tuning [141, 143]. In image generation, LoRA offers a good balance between fidelity, controllability, and diversity compared to some other fine-tuning methods like DreamBooth [144]. Trained LoRA weights in Stable Diffusion are loaded into the U-Net, text encoder, or both, and multiple LoRA weights can be used together at the same time, for example, for concept blending [142, 145].

In LoRA fine-tuning, pretrained weights are frozen so they do not receive gradient up-

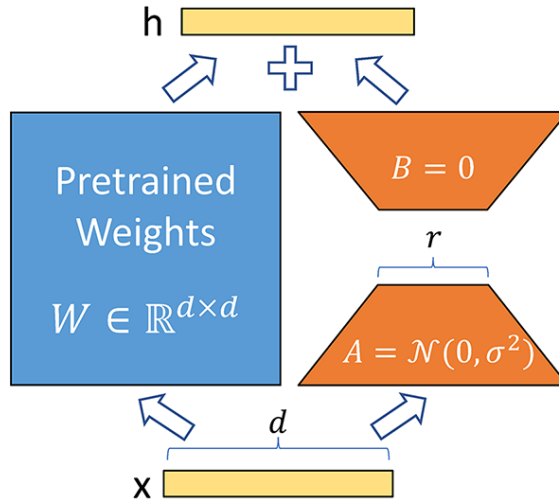


Figure 2.16. LoRA fine-tuning process [140]

dates. The weight updating process can be represented by the following equation:

$$\mathbf{h} = W_0 \mathbf{x} + \Delta W \mathbf{x}, \quad (2.35)$$

where $\mathbf{h}$ is the output, $\mathbf{x}$ is the input, $W_0 \in \mathbb{R}^{d \times k}$ is the pretrained weight matrix, and $\Delta W \in \mathbb{R}^{d \times k}$ is the weight update. During training, the weight update ΔW utilizes a low-rank decomposition:

$$\Delta W = BA, \quad (2.36)$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ are low-rank matrices and the rank $r \ll \min(d, k)$. This is illustrated in Figure 2.16. In the illustration, it can also be seen that at the beginning of training, A is randomly initialized with a Gaussian distribution and B is a zero matrix, which makes ΔW zero based on equation 2.36. The $\Delta W \mathbf{x}$ term is then scaled by $\frac{\alpha}{r}$, where α is a scaling factor. By scaling, it is possible to reduce the need to retune other hyperparameters like the learning rate when varying r . [140]

3. IMPLEMENTATION

This chapter presents the methodologies and tools used to implement the end-to-end pipeline for synthetic image generation and object detection model training. The first three sections introduce the overall workflow, scoping, and used datasets of the work. After that, the implementation of the pipeline is described. The implementation is divided into three main parts: LoRA training, synthetic image generation, and object detection model training. Figure 3.1 shows how the custom LoRA model is trained, how it is integrated into the SDXL base model to generate synthetic images, and how the generated images are processed for object detection model training.

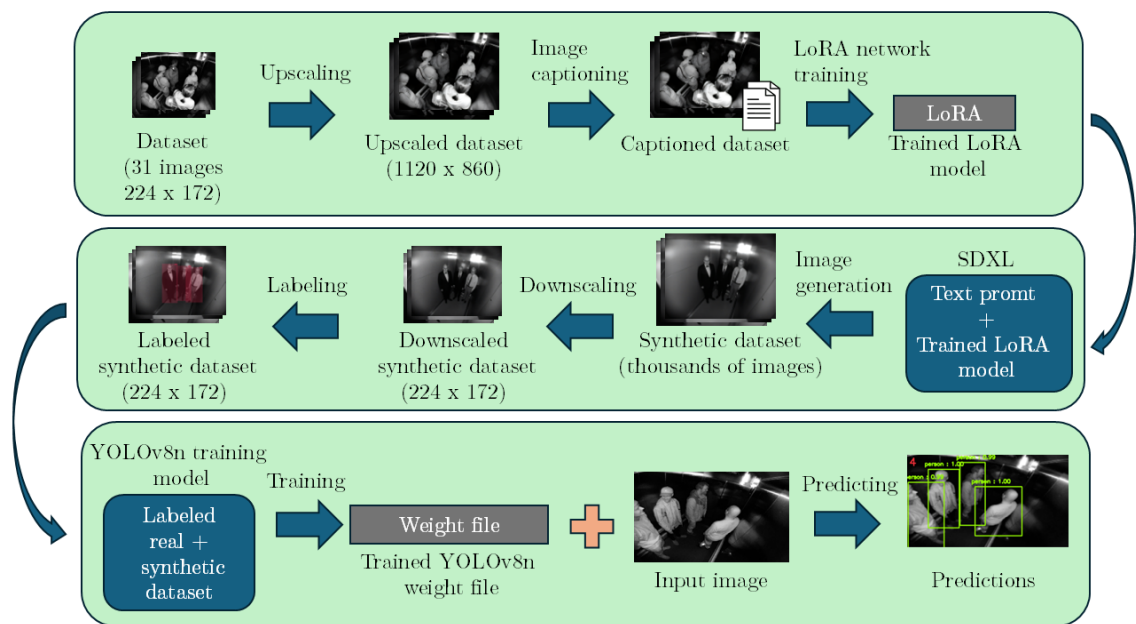


Figure 3.1. Overview of the synthetic image generation and object detection training workflow.

The first row of Figure 3.1 illustrates the preparation of the dataset for LoRA training. The process begins by selecting 31 different images captured from near-infrared (NIR) sensor, which are then upsampled using Upscayl from 224 x 172 to 1120 x 860 pixels, as the pretrained SDXL model requires high-resolution images for training. The upsampled images are captioned using Taggui for efficient captioning. The captioned dataset are used to train the LoRA model with Kohya-ss.

The second row shows how the pretrained SDXL base model is fine-tuned by interating the trained LoRA to generate synthetic images resembling those captured by the NIR sensor. The generation process is performed using Stable Diffusion WebUI Forge. Approximately 4000 images were initially generated, after which the synthetic dataset was cleaned to 2500 images. The cleaned dataset was downscaled to match the original sensor image resolution and then annotated together with sensor images using Darkmark for YOLOv8n model training. From the annotated data there was created 23 different training datasets with varying combinations of real and synthetic images.

The last row represents the training phase of 23 YOLOv8n models, each fine-tuned using one of the dataset variations. The training was performed using the Ultralytics Python library. The final outcome consists of 23 fine-tuned YOLOv8n models capable of detecting people in NIR images

3.1 Workflow

The development of the object detection model follows generally the same steps as machine learning workflows. Typically these workflows are divided into five main categories: scoping, data management, model development, verification, and deployment. These categories, along with their associated phases and example tasks, are listed in Table 3.1.

In the following sections scoping and data management phases are described in Sections 3.2 and 3.3. Model development for synthetic image generation is presented in Section 3.4, and object detection model training is detailed in Section 3.5. Verification is implemented in Chapter 4 to evaluate the object detection models. The deployment phase is not covered in this work, as the focus is on evaluating the effect of synthetic images on object detection model performance.

3.2 Scoping

The central question addressed in this work is whether synthetic data can be used to improve the performance of an object detection model. To answer this question, three main goals were defined:

1. Review existing methods for generating synthetic images and their effectiveness in improving object detection model performance. Based on this review, one approach is selected for implementation: a diffusion-based text-to-image model.
2. Implement a complete synthetic data generation and object detection pipeline from scratch.
3. Analyse the effect of different combinations of real and synthetic data on object detection model performance.

Table 3.1. Overview of the basic machine learning workflow grouped by categories and phases, with example tasks [146–148].

Workflow Category	Workflow Phase	Example Tasks
Scoping	Problem Identification	Defining the problem, defining requirements and constraints, conducting literature review on similar problems
	Objectives and Resources	Setting project goals, determining data requirements, examining available tools, models, and other resources
Data Management	Data Collection	Discovering existing data, recording new data
	Data Preparation	Annotating, cleaning, transforming, reducing, and augmenting data; generating synthetic data; splitting data into training, validation, and testing sets
Model Development	Model Selection	Choosing model architecture based on requirements, Selecting pretrained model for fine-tuning or fully training new model from scratch
	Model Training	Configuring hyperparameters and training the model
Verification	Model Evaluation	Calculating evaluation metrics, comparing results against criteria, iteratively tuning data and model as needed
	Test-based Verification	Performing real-world testing or simulated tests, evaluating model on production hardware
Deployment	Model Deployment	Exporting and integrating the model into applications, deploying to edge devices or cloud infrastructure
	Monitoring	Tracking model performance, updating the model with new data, retraining as required

The object detection task in this work focuses on detecting people in images captured by a Near-Infrared (NIR) sensor in small indoor environments. This domain was chosen because preliminary tests with the YOLOv8n base model showed that people could be detected reasonably well, yet there remained significant room for improvement. This makes it a suitable and challenging test case for exploring the potential of synthetic data augmentation in object detection.

The following requirements and constraints were defined for this work:

- The implementation must use only open-source tools and models.
- The text-to-image model must generate images that closely resemble the characteristics of real NIR sensor images.
- The text-to-image generation process must be automated to produce images across varied scenarios.
- The object detection model must be trained with hyperparameters that improve performance compared to the base model trained with real images only.
- Multiple object detection models must be trained with different combinations of real and synthetic images to analyse their effect on model performance.
- The best-performing object detection model's confidence threshold must be optimized to achieve the highest possible F1-score.

This work adopts the concept of *data-centric AI* as its conceptual framework for training the object detection model. Data-centric AI focuses on improving model performance primarily through refining and expanding the training dataset, while keeping the model architecture and hyperparameters largely unchanged [149]. In this work, this approach is applied by maintaining consistent hyperparameters for the object detection model as long as the results meet the defined requirements. The framework also supports the core idea of evaluating the effect of synthetic data by training multiple object detection models with different combinations of real and synthetic images.

Table 3.2 lists the tools and models used in this work, along with the motivation for their selection. All tools and models meet the requirement of being open-source, ensuring accessibility and reproducibility of the research.

3.3 Data Management

The initial dataset consists of near-infrared (NIR) images recorded inside elevators. These recordings were obtained from a student project dataset for reflection recognition research, where videos were captured in elevators using an NIR sensor. Additional recordings were collected with the same sensor to increase dataset diversity for training and testing. The recordings were taken from a tilted top-view perspective across elevators

Table 3.2. Tools used in the workflow and motivation for their selection.

Task	Tool/Model	Motivation for choice
Upscaling	Upscayl [150]	Upscaling and enhancing multiple images to the desired resolution
Captioning	Taggui [151]	Enables fast captioning of multiple images.
Text-to-image model	SDXL [152]	Open-source, high-performance, and flexible diffusion model suitable for fine-tuning.
LoRA training	Kohya SS [153]	Provides a user-friendly interface for LoRA training.
Text-to-image generation	Stable Diffusion WebUI Forge [154]	User-friendly interface for Stable Diffusion-based image generation; includes useful automation extensions for generating diverse image scenarios.
Annotation	Darkmark [155]	Efficient annotation tool with built-in analytics for annotation quality control.
Object detection model	YOLOv8n [156]	Lightweight and fast object detector, widely used in object detection studies.
Object detection model training	Ultralytics [157]	Python library for YOLO models providing extensive functionality for training and evaluation.

with varying backgrounds, lighting conditions, and occupancy. Since the focus of this work is on people detection, the dataset includes various human-related scenarios, such as individuals standing very close to the sensor or lying on the floor. Example images from the dataset are shown in Figure 3.2.

Captured images have a resolution of 224 x 172 pixels and are 1-channel grayscale. Since most tools used in this work require 3-channel images, each image was converted to RGB by replicating the grayscale values across all channels. The dataset was improved by removing adjacent frames of the recordings to prevent redundancy. Frames containing reflections from mirrors or reflective surfaces were removed, as they could cause false positive detections. Reflection detection itself was scoped out for future research.

After preprocessing, the dataset contained 1598 images for object detection (568 training, 284 validation, 746 testing) and 31 images for LoRA training. Additionally, 2500 synthetic images were later generated for object detection model training.

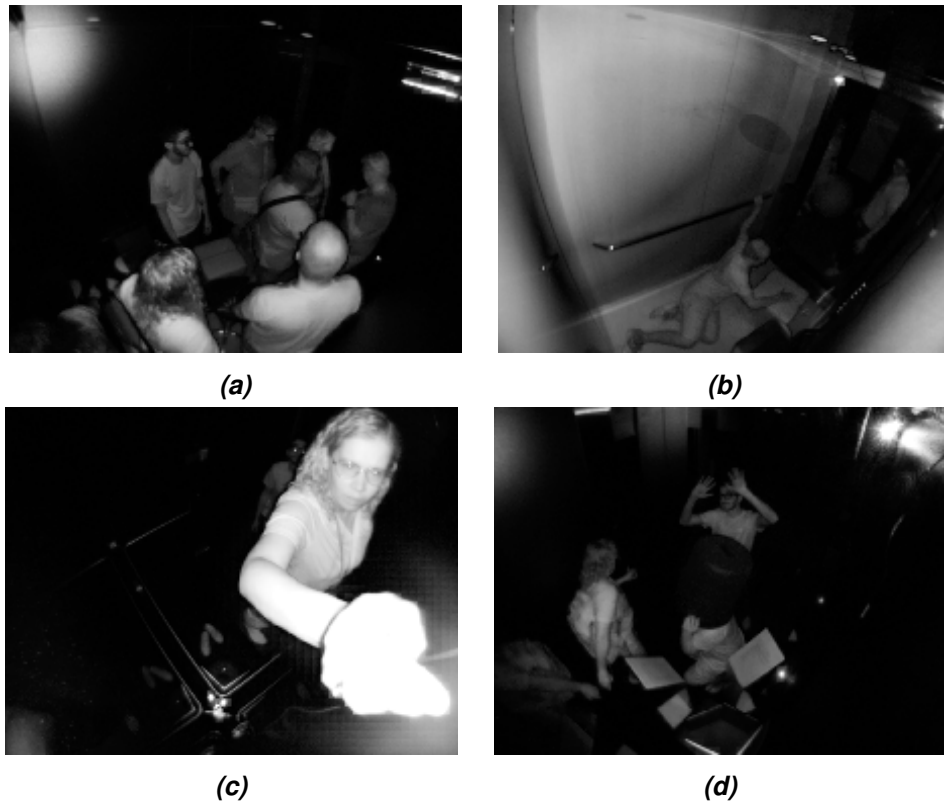


Figure 3.2. Example images from the NIR sensor: (a) crowded elevator, (b) person lying on the floor, (c) person very close to the sensor, (d) various objects and obscured people.

3.4 Synthetic Image Generation

3.4.1 Upscaling

The SDXL base model is trained with high-resolution images of varying aspect ratios, close to a pixel count of 1024 x 1024 [139]. To ensure compatibility with the base model, the LoRA should also be trained on images of similar resolution. Since the NIR sensor records images at 224 x 172 pixels, they must be upscaled before training. Based on the resolutions used for mixed-aspect-ratio fine-tuning in [139], the images were upscaled to 1120 x 860 pixels, maintaining the original aspect ratio as closely as possible.

Two upscaling methods were evaluated:

- **Bilinear interpolation (OpenCV):** A baseline approach where images were re-sized from 224 x 172 to 1120 x 860 pixels using OpenCV's `cv2.INTER_LINEAR` function. This method was expected to preserve sensor-specific characteristics such as noise and texture patterns, but may result in reduced structural coherence in the generated images.
- **AI-enhanced upscaling (Upscayl):** Utilizes AI-driven super-resolution techniques such as Real-ESRGAN, to enlarge images to 1120 x 860 pixels. This approach

aims to improve structural coherence in image generation while retaining the essential characteristics of the NIR images.

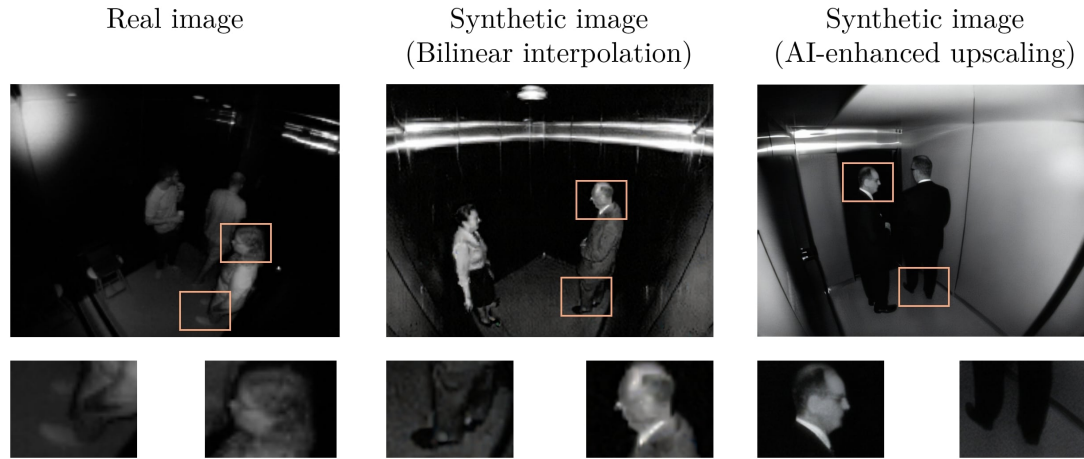


Figure 3.3. Real image and synthetic images generated using different upscaling methods for LoRA training. From left to right: (a) Real image, (b) Synthetic image using bilinear interpolation, and (c) Synthetic image using AI-enhanced upscaling.

Synthetic images in Figure 3.3 were generated using identical LoRA configurations (same hyperparameters, image-caption pairs, and random seed), except that the training datasets used different upscaling methods. Visual inspection indicates that both methods can mimic the monochromatic characteristics of the original images. However, compared to the original dataset, images generated using bilinear interpolation appear to have rougher noise patterns, while those generated using Upscayl exhibit smoother textures. Without any quantitative evaluation, both approaches appear to be visually realistic enough for object detection model training.

The decision to use Upscayl in this work was primarily based on early experiments where captioning was unintentionally left out during LoRA training with the bilinear interpolation dataset, leading to less consistent generation performance. Since the focus of this work was to design a complete pipeline from scratch, it was decided to proceed with Upscayl as it produced sufficiently realistic results.

3.4.2 Image Captioning

Captions are textual descriptions that provide information about the content of an image. As discussed in Section 2.5.2, the text encoder captures visual characteristics such as style, objects, and actions within a shared embedding space, making it possible to guide

image generation. In LoRA training, even though only the weights of the U-Net are fine-tuned, the model remains conditioned on the outputs of the text encoder. Therefore, the information contained in captions indirectly guides the optimization of the U-Net weights. When a model is trained without captions, it still learns the features of the training dataset, but the resulting images exhibit limited flexibility and controllability in prompt-based image generation. This is the main motivation for using captions in training.

Since there was a lack of published research papers at the time of implementation, the primary guidelines for captioning were derived from the experience of the Stable Diffusion community [158–162]. The main principle can be summarized as follows: *"Features that we want to control later in image generation should be mentioned in the captions."*

Within the community, there are mainly used four different captioning strategies: no captions, natural language, tags, or a combination of both natural language and tags. Table 3.3 provides an example of captioning using both natural language and tags.

Table 3.3. Example captioning of the bottom-right image from Figure 2.1 using natural language and tags.

Natural language	Tags
A group of three people walk together in the rain under umbrellas. They are wearing coats and carrying bags on their shoulders. In the background, there is a grassy area with a wooden bench and a brick building with an open vent.	three people, rain, umbrellas, coats, bags, grassy area, wooden bench, brick building, open vent

Natural language captions provide contextual descriptions using full sentences, while tags describe content of the image through short keywords. In this work, a combination of both was used: natural language for elements requiring fine-grained control (e.g., describing clothing or actions of people), and tags for simpler attributes (e.g., presence of mirrors or lighting condition in the elevator). This approach increase caption diversity while remaining compact and providing good control over important features.¹

For captioning, Taggui was used, as it allows fast captioning of multiple images simultaneously—for example, assigning shared tags to similar images. Taggui also includes an option to generate captions using the BLIP captioner, but it did not recognize content of NIR images correctly and was therefore not used. Figure 3.4 shows an example of an image–caption pair used in LoRA training.

¹Some community guidelines also recommend using unique tags to activate specific features, such as new object types or one of the multiple styles from the training dataset. Unique tags are not presented in pretrained text encoder, and letters are sometimes replaced with numbers or special characters to ensure uniqueness. For example, a good unique tag for NIR images could be "N1r1m4g3" as it is not likely presented in pretrained text encoder. In this work, unique tags were not used for feature activation, as the LoRA was expected to learn the characteristics of NIR from the training images itself even without captions.



Caption: ir-image, top-view perspective, dark elevator, a man is going out from an elevator carrying a box on his shoulder

Figure 3.4. Example image-caption pair used in LoRA training.

In Section 3.4.4, Figure 3.5 demonstrates how prompts were used in image generation, while Figure 3.6 shows different scenarios generated by different prompts.

3.4.3 LoRA Training

For training the LoRA to fine-tune the SDXL base model, Kohya's GUI (also known as *kohya-ss*) was used. Tables 3.4 and 3.5 summarize the most important hyperparameters and their values used in LoRA training process.

The hyperparameter values in Table 3.5 were chosen based on experimentation and community experience. Training was performed using Stability AI's official SDXL checkpoint. The training dataset contained 31 captioned images, each repeated 20 times, resulting in 620 steps per epoch. Training ran for 20 epochs, and the LoRA weights were saved after every second epoch, allowing evaluation and comparison of LoRAs at different training stages. The batch size was set to 1 due to GPU memory limitations.

Adafactor was chosen as the optimizer because of its adaptive learning rate scaling and high memory efficiency [164]. The maximum resolution was set high enough to ensure

Table 3.4. Description of the most important hyperparameters for LoRA training [163].

Hyperparameter	Description
Learning Steps	Total steps = number of images $\times$ repeats $\times$ epochs. Each step shows the model one image during training.
Repeats	Number of times each image is shown per epoch.
Epochs	Number of complete passes through the dataset (images and repeats). The model state can be saved after every N epochs.
Batch Size	Number of images processed simultaneously during training. Larger values increase VRAM usage and may slightly affect model performance.
Learning Rate (LR)	Controls how much the weights are adjusted during optimization. Too high a value can cause the model to overfit, while too low a value can lead to underfitting. There are two types of LR in LoRA training: U-Net LR is for fine-tuning attention and other blocks in the U-Net, and Text Encoder LR determines the sensitivity to captions. The Text Encoder LR is usually lower than U-net LR, as it also affects the whole U-net.
Learning Rate Scheduler	Adjusts the LR during training. Common schedulers are constant (constant LR), linear (decreasing LR linearly), and cosine (decreasing LR following a cosine curve).
Optimizer	Algorithm used to update network weights during training. Common optimizers are AdamW, AdamW8bit and Adafactor.
Max Resolution	Sets the maximum training image resolution. Images exceeding this resolution are downscaled.
Buckets	Groups training images of different sizes into resolution "buckets." Images are resized to standard size within the selected bucket resolution limits (if 'Don't upscale bucket resolution' is false).
Network Rank	Defines the amount of information stored in the LoRA. A higher rank captures more detail but risks overfitting, while a lower rank has more consistency but less detail.
Network Alpha	Scaling factor for storing LoRA weights. The alpha/rank ratio affects the learning rate.

that all training images (1120 x 860) were below this threshold. Buckets were enabled to resize images to the nearest multiple of 64 while preserving the aspect ratio as closely as possible. As a result, training images were resized during training to 1088 x 832 pixels (aspect ratio ≈ 1.308) instead of 1120 x 860 (aspect ratio ≈ 1.302). For the network parameters, the rank and alpha were set correspondingly to 32 and 16. The resulting LoRA models reproduced the visual characteristics of the original NIR training data when applied to the SDXL base model, as shown in Figure 3.5.

Table 3.5. Hyperparameter values used for LoRA training.

Hyperparameter	Value
Pretrained model	stabilityai/stable-diffusion-xl-base-1.0
Save precision	BF16
LoRA type	Standard
Train batch size	1
Epoch	20
Repeats	20
LR Scheduler	Constant
Optimizer	Adafactor
Max resolution	1536 x 1536
Enable buckets	True
LR (U-Net and Text Encoder)	0.0004
Learning rate warmup steps	0
Network Rank	32
Network Alpha	16

3.4.4 Image Generation

Image generation was performed using Stable Diffusion WebUI Forge, which is a platform built on top of Stable Diffusion WebUI (also known as *AUTOMATIC1111*) [165], designed to optimize resource management. The parameters used for the image generation are described in Table 3.6.

Table 3.6. Description of parameters used for image generation.

Parameter	Description
Sampler	Defines the algorithm used for sampling (e.g., DDIM, PLMS, DPM++ 2M, Euler a).
Sampling step	Number of denoising iterations performed from random noise to the final image.
CFG scale	Classifier-Free Guidance (CFG) scale controls how strongly the text prompt will influence the diffusion process. There is a trade-off between diversity and accuracy: Higher value makes the sampling process follow the text prompt more strictly, while a lower value reduces the influence of the text prompt.

Parameter sensitivity was analyzed by changing one parameter at a time while keeping the others fixed and repeating the process with several random seeds. This was also

done for different LoRA models and strengths to determine which combination of LoRA model, strength, and parameter values produce the best results. An example comparing different LoRA strengths is shown in Figure 3.5.



Figure 3.5. Effect of different LoRA strengths on image generation using the following prompts: **Prompt:** *ir-image, top-view perspective, two construction workers waving hands inside an elevator*

Negative prompt: *mutated body parts, disfigured, bad anatomy, deformed body features, metallic wall, mirror, (white clothes:1.2), beanie*

From Figure 3.5, it can be seen how different LoRA strengths affect image generation. The SDXL base model itself was unable to generate images inside an elevator, as described in the prompt. After applying a LoRA strength to 0.4, the similar characteristics captured from the NIR sensor started to appear, although images still appeared more defined and less noisy compared to the training images. Strengths between 0.6 and 0.8 provided a good balance between diversity and similarity. Beyond a strength of 0.8, generated images began to overfit, occasionally reproducing recognizable individuals from the training dataset. Based on these observations, a LoRA strength of 0.7 was selected for image generation. Similar examples for Sampler, Sampling step, and CFG scale are illustrated in Appendix A.

Based on these experiments, the parameters used for image generation are summarized in Table 3.7.

To generate a diverse set of synthetic images, the wildcard extension tool was used to randomize prompts. Wildcards allow defining collections of words under wildcard variable that are randomly selected during image generation. For example, a custom wildcard

Table 3.7. Values of the parameters used for image generation.

Parameter	Value
Height and width	1088 x 832
Sampler	Euler a
Sampling step	20
CFG scale	7

variable `__amount_of_people__` could randomly be replaced by "one", "two", "three", "a few", "many" or "large group of". In this work, wildcards were used to control various attributes such as environment, action, and number of people, as shown in Figure 3.6.

**Figure 3.6.** Example of images generated using wildcards in prompts.

A total of over 4000 images were generated, after which images with visible color artifacts, overrepresented attributes or unrealistic distortions were removed. From cleaned synthetic dataset, 2500 images remained for training object detection models. The average generation time with the chosen parameters was approximately 6 seconds per image using an RTX 3080 10GB GPU.

3.5 Object Detection Model Training

The cleaned synthetic dataset of 2500 images was resized to 224 x 172 pixels to match the resolution of the original NIR sensor images. Both synthetic and real images were annotated using Darkmark, a GUI tool for annotating images, in which people were labeled by drawing bounding boxes around them. To reduce manual labour, a pretrained YOLOv8l model was used to automatically pre-annotate the dataset. This approach was able to correctly detect some people and provided a good starting point for manual an-

notation, although many cases were missed. As the annotation process progressed, a YOLOv8l model was fine-tuned using already annotated images to improve detection accuracy and assist even better in pre-annotating the remaining synthetic images.

To evaluate how synthetic images affect fine-tuned model performance, 23 separate models were trained using different combinations of real and synthetic images. The real image subsets included 0, 200, 400, and 568 images, while the synthetic image subsets included 0, 500, 1000, 1500, 2000, and 2500 images; each larger subset contained all images from the previous, smaller subset. This resulted in 24 possible combinations, with the combination of 0 real and 0 synthetic images representing the base model without any fine-tuning. For validation, 284 real images were used for all combinations.

For training, a pretrained YOLOv8n model was fine-tuned using the Ultralytics Python library. Hyperparameters were selected based on [43] and are presented in Table 3.8. Additionally, the first 10 layers of the model were frozen, as the base model was capable of detecting people.

Table 3.8. Hyperparameters used for YOLOv8n fine-tuning.

Hyperparameter	Value
Epochs	60
Optimizer	AdamW
Momentum	0.9
Image size	640
Initial learning rate	0.0027
Learning rate factor	0.5
Batch size	64
Cosine learning rate	True
Warmup epochs	4
Single class	True
Freeze layers	10
Seed	0

The detection settings and the measured evaluation metrics of the fine-tuned models are presented in Chapter 4.

4. RESULTS

This chapter presents the results of the object detection experiments. In the first part of the chapter, the testing setup and the average precision (AP) of the models are presented. Additionally, a few representative models were examined to assess how much performance can vary, to explain the observed irregularities, and to further illustrate the effect of synthetic and real images. In the second part of the chapter, the confidence threshold of the YOLOv8n base model and the best-performing fine-tuned model is optimized to maximize the F1-score, and the detections of both models are qualitatively inspected.

4.1 Testing setup and Impact of Synthetic Images

The testing dataset consisted of 746 images collected from scenarios different from those used in training and validation to ensure that model performance was evaluated on unseen data. The models were evaluated using the Ultralytics Python library to generate predictions on the test images. The evaluation metric $AP_{IoU:0.50}$ was computed with the Object Detection Metrics toolkit [59], following the Pascal VOC evaluation standard. Predictions were made using an IoU threshold of 0.5 and a confidence threshold of 0.001 to ensure that confidence filtering would not distort the AP computation. Figure 4.1 shows the AP values for the YOLOv8n base model and all models trained with different combinations of real and synthetic images.

As a general trend, model performance increased with the number of synthetic training images and when real images were added on top of them. Synthetic data alone improved the base model's ability to detect people in real images. The best-performing models were r400s1000 (400 real + 1000 synthetic) and r568s2500, both achieving an AP of 0.938.¹ The YOLOv8n base model performed the weakest, with an AP of 0.815.

Although models trained solely with synthetic data improved their performance, adding real images had a strong positive impact on detection accuracy. For instance, increasing the dataset with 200 real images improved and stabilized the AP of models trained with at least 1500 synthetic images (real images making up approximately 10% of the dataset).

¹ Although both achieve the same AP, the text refers to r568s2500 as the best performing model because it achieved the highest F1-score at the optimized confidence threshold described later (Table B.1).

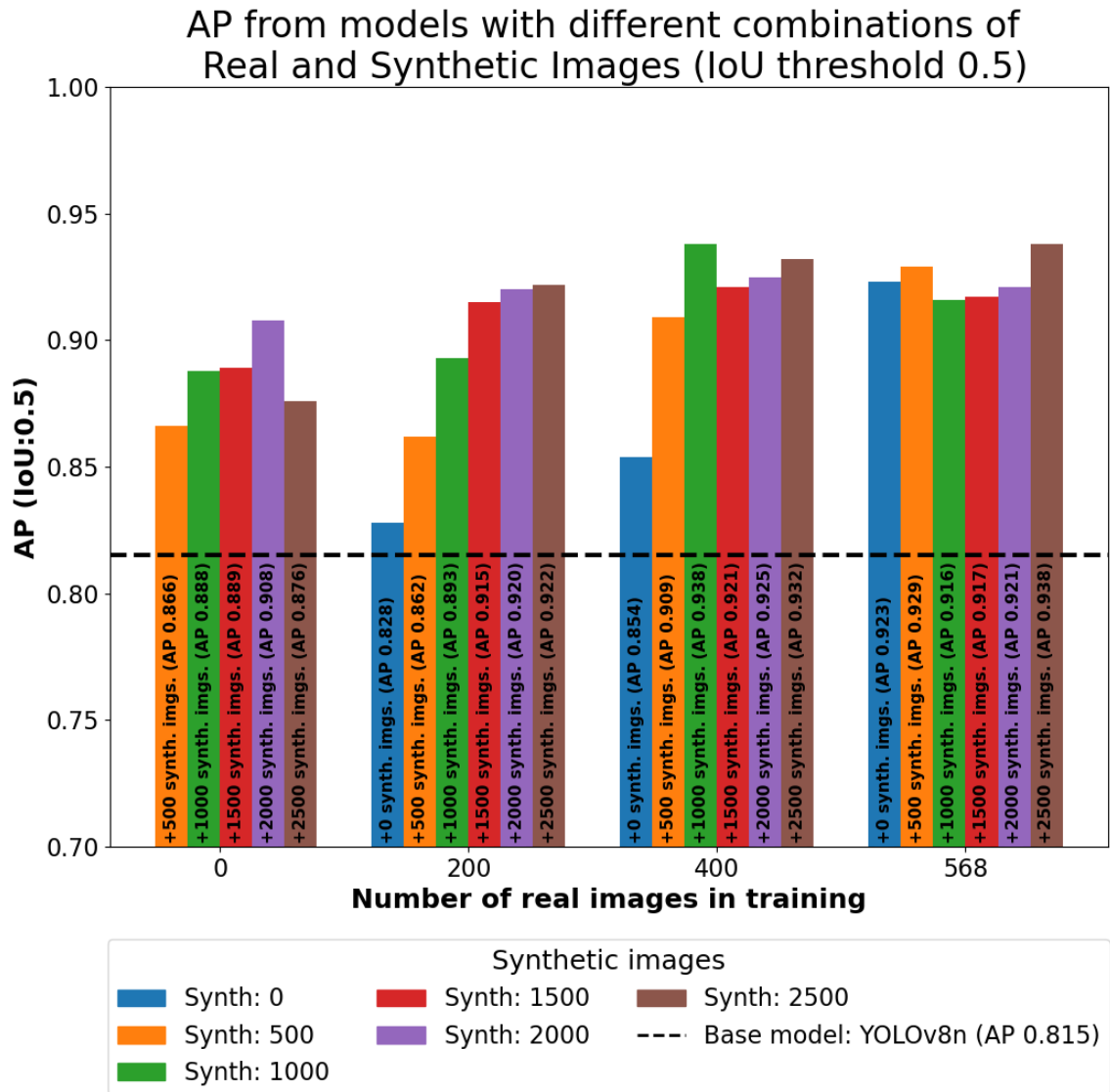


Figure 4.1. Average Precision ($AP_{IoU:0.50}$) for the YOLOv8n base model and fine-tuned models trained with different real/synthetic ratios. Exact values are presented inside the bars.

However, this effect was less clear for models trained exclusively on real images. The relatively weak performance of r200 is somewhat surprising, given that adding 200 real images improved and stabilized the models with many synthetic images. Moreover, because the performance of r200 and r400 appeared to follow a logarithmic trend as the dataset size increased, which is typical in deep learning, r568 was expected to achieve an AP similar to s500. Instead, r568 achieved an irregularly high AP. Due to these irregularities, it is difficult to precisely quantify this effect.

The following irregularities were observed across model configurations:

- The s2500 model performed second worst among the synthetic-only models, but improved greatly in mixed datasets.

- r200s500 performed worse than s500.
- r400s1000 achieved a highest AP together with r568s2500.
- r568 showed a sudden increase compared to r200 and r400 models.
- r568s500 was the second-best performer in the r568 group, contrary to the general trend.

While hyperparameters were kept constant across all trainings, even small changes in dataset composition affected the learning process, as seen in Figure 4.1. To better understand the underlying causes of the observed performance variations, an additional experiment was conducted after obtaining the main results to assess the effect of stochasticity in deep learning training. First, the training process was verified to be deterministic when using the same dataset and seed, as retraining produced identical weights. Then, three models, namely s2500, r568, and r568s2500, were each retrained twice using different but shared random seeds. The following AP means and standard deviations in Table 4.1 were obtained across the original and retrained models:

Table 4.1. $AP_{IoU:0.50}$ mean and standard deviation across three runs of models trained with different shared seeds.

Model	$AP_{IoU:0.50}$	
	Mean	Standard deviation
s2500	0.886	0.010
r568	0.900	0.027
r568s2500	0.932	0.010

These results indicate that random initialization can have a notable effect on model performance, which partly explains why some models outperform or underperform relative to expectations. This is particularly evident in the case of the r568 model, where the standard deviation is high and the mean AP is considerably lower than that shown in Figure 4.1. Similar irregularities related to random seed selection were also observed when retraining once the r400s1000 and r400s1500 models. In the original results (Figure 4.1), r400s1000 (AP 0.938) outperformed r400s1500 (AP 0.921), whereas after retraining with a new shared seed, the relationship was reversed (r400s1000: AP 0.930, r400s1500: AP 0.932). This further confirms that stochastic variations during training can significantly alter the relative ranking of models with otherwise similar data compositions (especially for small datasets).

Beyond the stochastic effects, the results in Table 4.1 also highlight how dataset itself influence both the stability and performance of the model. The s2500 model achieved a lower mean AP than r568, while the r568 model exhibited nearly three times higher standard deviation. When these datasets were combined in the r568s2500 model, both

the mean AP improved significantly and the standard deviation decreased to the level of s2500.

4.2 Confidence Threshold Optimization and Qualitative Analysis

To find the optimal confidence threshold that produces the highest F1-score, scikit-learn's `precision_recall_curve` function was used. Before providing inputs to the function, predictions were filtered with an IoU threshold of 0.5. As described in Section 2.1.2, the precision-recall curve is obtained by plotting precision and recall for each confidence threshold. To identify the optimal confidence threshold, F1-scores were computed using Equation 2.6 and plotted against the corresponding confidence thresholds. The threshold that yielded the highest F1-score was selected. Figures 4.2 show F1-scores for different confidence thresholds for the YOLOv8n base model and the r568s2500 model:

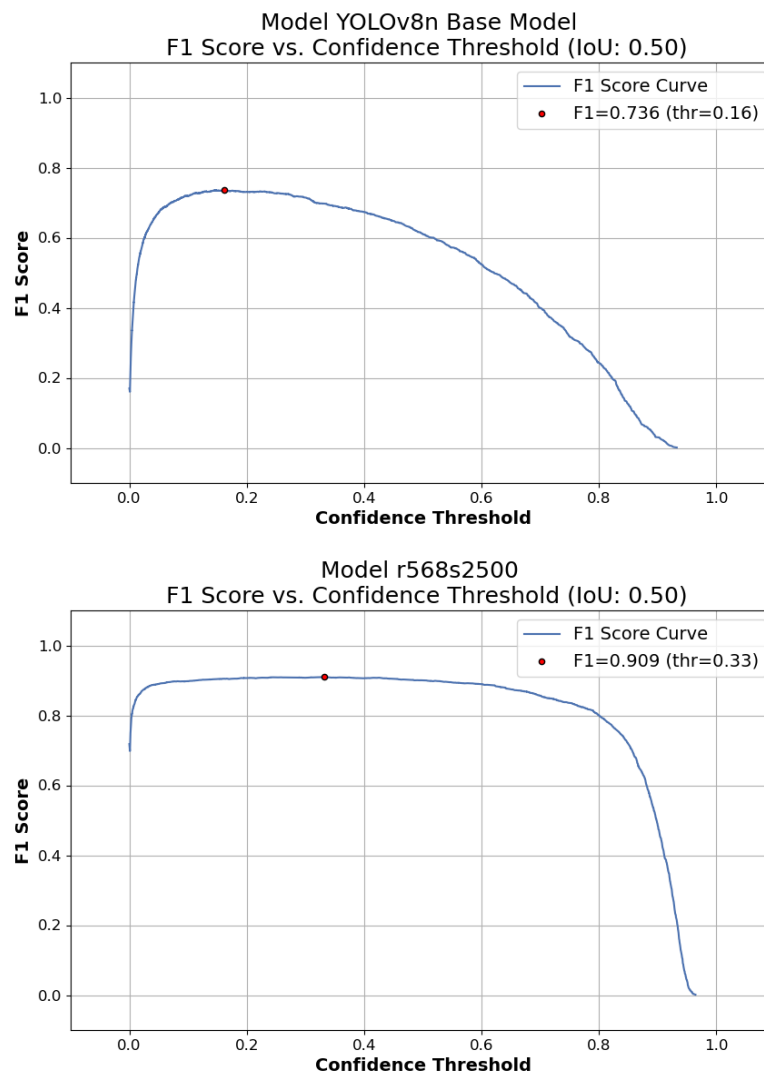


Figure 4.2. F1-scores over all confidence thresholds for the YOLOv8n base model and the r568s2500 model. The highest F1-score on the curve is marked with a dot.

From the plots in Figure 4.2, it can be seen that the YOLOv8n base model reaches its highest F1-score ($F1 = 0.736$) at a confidence threshold of 0.16, while the r568s2500 model reaches its maximum ($F1 = 0.909$) at a threshold of 0.33. A lower F1-score indicates weaker overall performance, which can result from low precision, low recall or both. The figures also show that the YOLOv8n model maintains a high F1-score within the range of 0.10-0.20 before it starts to decline relatively steadily, whereas the r568s2500 model maintains high performance between 0.10-0.50 and drops sharply after 0.80. As mentioned in Section 2.1.2, a higher confidence threshold results in more false negatives (and fewer false positives) as predictions are discarded under stricter criteria. Based on this, it can be concluded that the YOLOv8n base model detects people across a broad range of confidence levels, as its F1-score starts to decline steadily after a confidence threshold of 0.20. In contrast, the r568s2500 model produces consistently high-confidence detections, maintaining a strong F1-score until a sharp drop occurs beyond 0.80. This indicates that discarding predictions above this threshold rapidly disrupts the balance between precision and recall. The s2500 model, trained solely on synthetic data, also exhibited a similar F1-score curve to the r568s2500 model, as shown in Appendix B.1.

A more detailed quantitative overview of the model predictions is presented in Table 4.2, which summarizes the number of ground-truth objects, total predictions, true positives (TP), false positives (FP), and false negatives (FN) for the selected models using optimized confidence thresholds and an IoU threshold of 0.5.

Table 4.2. Ground truth, predictions, true positives (TP), false positives (FP), and false negatives (FN) for the YOLOv8n base model and r568s2500 model using optimized confidence thresholds.

Model	Ground-truth	Predictions	TP	FP	FN
YOLOv8n	1785	1496	1207	289	578
r568s2500	1785	1714	1591	123	194

From the Table 4.2, it can be observed that the YOLOv8n base model produced the largest number of false detections, both FP and FN. After fine-tuning with real and synthetic data, the number of false negatives decreased by approximately two-thirds compared to the base model, and false positives were reduced by more than half. For all models presented in Figure 4.1, the optimized confidence thresholds and related evaluation metrics are provided in Table B.1. An interesting observation from the previously mentioned table is that the r200 and r400 models had false positive values in the range of around 400 at their selected confidence thresholds, which is notably higher than that of the YOLOv8n base model.

YOLOv8n base model: A qualitative inspection of the YOLOv8n predictions (Figure 4.3 and 4.4) showed that the model performs reliably in straightforward scenarios where a

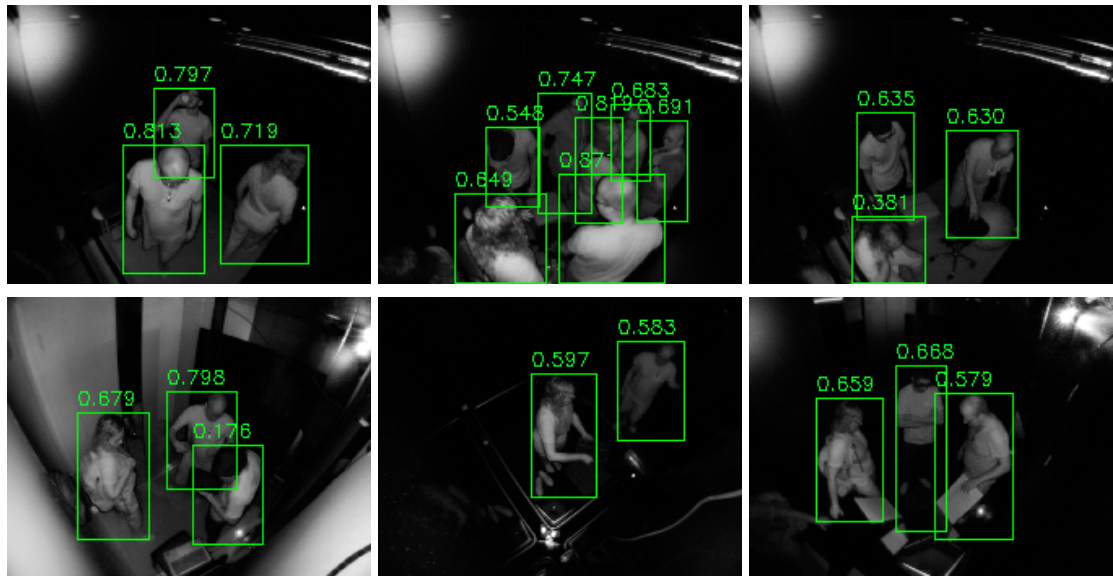


Figure 4.3. Examples of successful detections by the YOLOv8n base model. Green bounding boxes indicate detected persons, with confidence scores shown above each box.

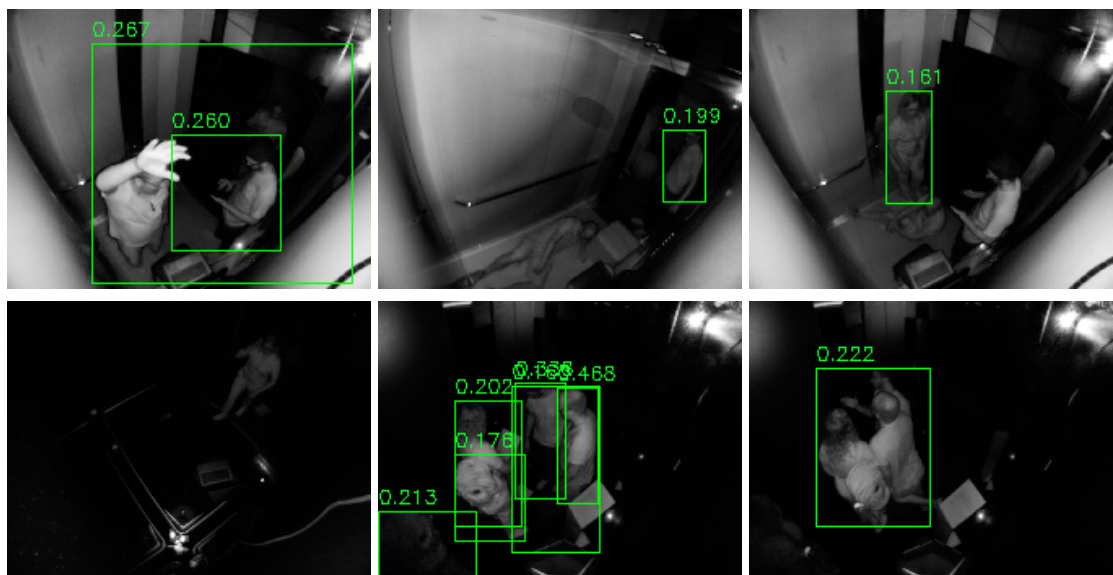


Figure 4.4. Examples of detections by the YOLOv8n base model, showing both false negatives and false positives.

person is clearly visible and upright. The model was also capable of detecting close-range human figures without major difficulties. However, it often failed to detect people in more complex settings, such as partially occluded individuals, those positioned near doorways, or persons lying on the floor. Especially in images with bright lighting, strong reflections, or shadows (as seen in the upper images of Figure 4.4), people were frequently not detected at all. These undetected cases constituted the majority of false negatives. False positives were frequently caused by the model interpreting parts of the environment (e.g., furniture or bags) as humans, especially when these objects overlapped or resembled human silhouettes. In several cases, large regions of the environment were incorrectly classified as people.

Fine-tuned model (r568s2500): The fine-tuned model r568s2500 demonstrated a clear improvement in both recall and precision. Visual inspection confirmed that the fine-tuned model was able to detect nearly all standard cases where a person was visible, even under challenging lighting and viewing conditions. The model also avoided mistaking environmental structures for humans. By comparing Figures 4.3 and 4.5, it can be seen that the fine-tuned model successfully detected people in corresponding cases while the confidence levels were generally higher. False negatives occasionally occurred in extreme cases, such as individuals lying down, standing behind a door in a dark area, or being partially occluded. The improved performance can be clearly seen by comparing Figures 4.4 and 4.6. However, unlike the YOLOv8n base model, in cases where individuals interacted closely with the camera or prominently raised their hands, the model often detected the hands as separate persons, as shown in Figure 4.7. This accounted for most of the false positive cases. Notably, this phenomenon of misclassifying hands as people occurred across all fine-tuned models, including those trained purely on real or synthetic data. This strongly suggests that the cause of this issue lies in the way training data were labeled.

In summary, the qualitative and quantitative analyses consistently show that fine-tuning the YOLOv8n base model with both real and synthetic images greatly improved its robustness across diverse indoor scenarios. When comparing the YOLOv8n base model to the best-performing r568s2500 model, AP increased from 0.815 to 0.938 (+0.123, 15%). The peak F1-score rose from 0.736 to 0.909 (+0.173, 24%) and at the optimized thresholds false negatives decreased from 578 to 194 (66% reduction) while false positives dropped from 289 to 123 (57% reduction).

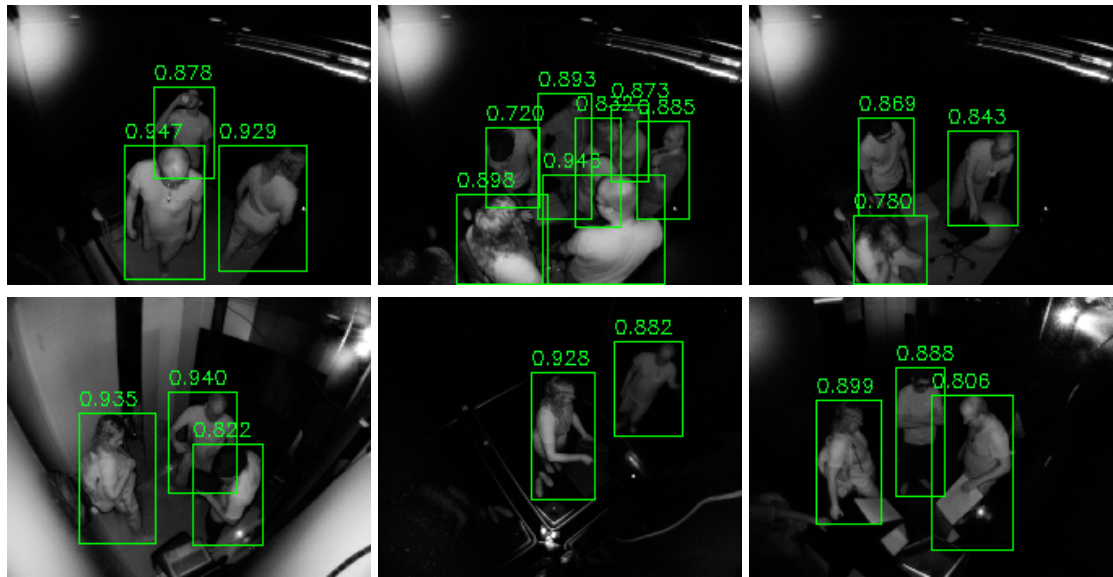


Figure 4.5. Detections of the *r568s2500* model for the corresponding images shown in Figure 4.3.

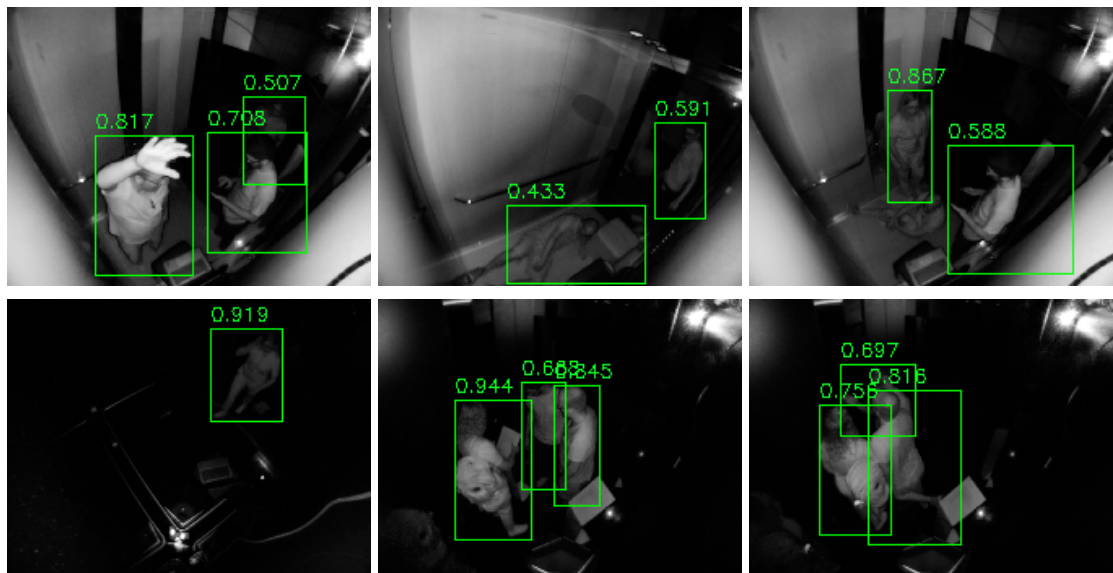


Figure 4.6. Detections of the *r568s2500* model for the corresponding images shown in Figure 4.4.

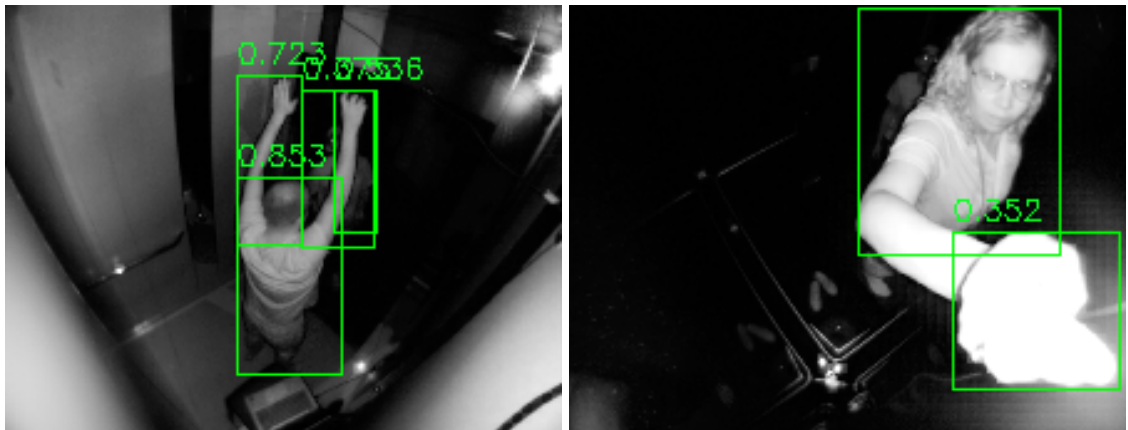


Figure 4.7. Examples of false positive cases detected by the *r568s2500* model, where hands were occasionally detected as separate persons.

5. DISCUSSION

This chapter presents reflections on the observations made during the work and, based on these, provides suggestions for practical implementation as well as proposals for future research.

5.1 Discussion of Implementation and Results

The implementation of the synthetic data generation and object detection pipeline was successful based on the results and objectives set for this work. Using the selected methods, tools, and parameters, it was possible to automate the generation of images that visually resembled the real images used for training. It is noteworthy that the chosen method, which produces synthetic images using the Stable Diffusion SDXL model, proved particularly suitable for recognizing people in indoor environments. This is likely because the base model has been trained primarily on a large collection of photographs featuring people, objects, and diverse environments. Therefore, the implementations where the base model is fine-tuned with only a few dozen images may be more difficult to apply to concepts that are very different from the original training data of the base model. Such concepts could include, for example, medical images or complex industrial components. Although numerous recent studies have investigated diffusion-based text-to-image models for these types of use cases, a detailed exploration of such applications is left for future research.

The results demonstrated that synthetic images, both alone and in combination with real images, can be effectively used to train object detection models for real-world data. This suggests that the generated synthetic images successfully captured the essential visual characteristics of the real domain. Relatively good and stable performance was achieved when a sufficient amount of synthetic data was combined with real images. When retraining object detection models multiple times while changing only the random seed, a higher mean AP indicates overall better performance, and a lower standard deviation indicates greater stability. As seen in Table 4.1, the s2500 model showed greater stability, the r568 model showed better overall performance, and the r568s2500 model combined both high performance and stability. This indicates that a larger dataset generally produces more stable results, while real images can provide a stronger performance boost even with

fewer samples compared to synthetic images.

Several factors can explain the irregularities observed in the results, in addition to the stochasticity of deep learning model training. For example, the underperformance of the s2500 model could be attributed to overfitting, as the training loss decreased steadily while the validation loss began to increase during the last ten epochs. Conversely, the sudden high performance of the r568 model could be related to the quality of the added images, where even small improvements may have a stronger effect in a smaller dataset. This may also explain the low performance of r200, while performance and stability improved when a small number of real images were added to larger synthetic datasets. The added real images likely helped the model generalize better to the real domain, whereas r200 alone was too small to meaningfully improve the base model. The other irregularities can likely be attributed to the stochastic nature of training, where minor changes in the dataset, hyperparameters, or weight initialization can lead to significantly different outcomes. This is understandable considering that convolutional neural networks must learn to extract relevant features from images and classify them through high-dimensional fully connected layers, forming a highly complex system with many local minima in the loss landscape.

From the qualitative analysis, it can be observed that the YOLOv8n base model was already somewhat capable of detecting human figures, as its pretrained weights are likely familiar with monochromatic images. However, NIR images have their own specific noise characteristics, and in combination with difficult lighting conditions, some basic cases and especially occlusions and unusual poses proved challenging for the base model. The r568s2500 model showed a clear improvement in adapting to these conditions and was able to detect people in more challenging scenarios, even though some room for improvement remained in edge cases. It is understandable that more challenging cases still have room for improvement when training datasets are relatively small, but since the objective of this work was to test the effectiveness of the chosen methods and synthetic images, the use of much larger datasets is left to a possible full scale implementation.

5.2 Practical Recommendations

For practical implementation of the methods presented in this work, the following recommendations are made based on the observations and results obtained:

Diversity of LoRA training dataset: During image generation, it was sometimes necessary to use stronger prompt weights to reduce the repetition of certain visual elements originating from the training images or to make specific elements appear more prominently. Although a small number of images was sufficient for LoRA training in this work, greater diversity among them would be beneficial. For example, including different environments, people, and other varying factors in the training data would enable the gener-

ation of a wider variety of images, reducing the repetition of recurring features. Furthermore, although the generated images in this work primarily represented elevator spaces from a tilted top-view perspective, it is reasonable to assume that the model's performance could further improve if the training data included a variety of perspectives, such as close-ups, straight-on views, or entirely different environments like corridors or lobbies. In such cases, the importance of accurate and descriptive captioning becomes even more critical, as it helps distinguish these variations from one another.

Recommended real-synthetic ratio: The results of this work show that a sufficiently large mixed dataset can provide both good stability and performance, even with a relatively small number of real images. The suggested ratio of 5%-20% real images to 80%-95% synthetic images in training, as presented in Section 2.3, is supported by the results where the r200s1500 and r200s2500 models contained 11% and 7% real images respectively.

Confidence threshold selection: When the model maintains high F1-scores across a wide range of confidence thresholds, there is flexibility in selecting more suitable threshold depending on the needs of the application. For example, slightly increasing the confidence threshold can be useful if false positives are more problematic than false negatives in a given context. However, it is still recommended to select a threshold close to the one that maximizes the F1-score to achieve the best overall balance.

Edge cases: Although the fine-tuned models achieved good results in quantitative analysis, there were still some edge cases where detection performance was weaker, such as when people were lying on the floor. This small number of similar undetected cases could potentially be reduced by adding more images of such situations and by making people lying on the floor appear even more "blurred," as they did in the real images. Therefore, it is recommended to include at least a few representative images of each edge case in the LoRA training dataset if possible. Alternatively, such variations could be introduced through advanced prompt engineering.

Labeling: It was observed across all fine-tuned models that they were vulnerable to classifying arms, hands, or partially visible limbs as separate persons. This is likely related to the labeling guidelines for occluded individuals, where the entire person was annotated even if only a small part, such as an arm, was visible or when significant overlap occurred. Even if this was not the original cause of the issue, it is recommended to carefully reconsider labeling guidelines to detect more precisely with less false detections.

5.3 Future Works

One of the most important directions for future research would be to investigate whether synthetic images could be automatically annotated during their generation phase. This

would eliminate the need for manual annotation, allowing the images to be used immediately for training. For instance, the Diffusion Attentive Attribution Maps (DAAM) [166] approach can produce heatmaps that visualize which parts of the generated image the diffusion model focused on with respect to a specific prompt, such as "person." Combining this approach with models like Segment Anything [167] could enable highly accurate automatic generation of bounding boxes for most synthetic images. Based on the author's experiments, the Segment Anything model performed well with the synthetic images used in this work, suggesting that it is a promising candidate for further investigation.

Another potential research avenue would be to identify a suitable simulator or game engine capable of modeling the desired environments, generating images from them, and then applying an image-to-image technique to make these images more realistic. This could produce data that is both precise (as in simulator-based imagery) and visually realistic, while simultaneously addressing the challenge of manual annotation. The image-to-image approach could be implemented similarly to this work within the SDXL framework by incorporating the previously mentioned ControlNet method. ControlNet would allow simulator-generated images to be transformed into the LoRA-trained visual style.

Additional research topics could include optimizing individual stages of the workflow, such as comparing different captioning methods (e.g., long descriptive captions versus concise summaries) or evaluating different upscaling approaches. Furthermore, future work could quantitatively assess the similarity between synthetic and real images using established metrics such as Fréchet Inception Distance (FID), Kernel Inception Distance (KID), or Inception Score (IS).

6. CONCLUSION

This work investigated how effectively synthetic images can be used to improve the performance of an object detection model. The object detection task of the work was to detect people in small indoor environment using images captured with a near-infrared sensor. Based on a literature review, different methods for generating synthetic images were compared, including 3D simulators, image blending, image-to-image and text-to-image models, for enhancing the performance of object detection models. From this review, a diffusion-based text-to-image model, Stable Diffusion XL (SDXL), was selected for synthetic image generation due to its ability to produce visually similar images to the real training set by fine-tuning and to generate diverse images using textual prompts.

For implementation, a pipeline was developed for synthetic image generation and object detection model training. In the first stage, a LoRA model was trained to fine-tune SDXL using 31 images captured with a near-infrared sensor. For LoRA training, the images were resized to match the SDXL model's requirements, and descriptive captions were created for each image. By combining the trained LoRA with the SDXL model, it was possible to generate synthetic images resembling the near-infrared images. The generation of diverse images was automated using the Wildcard extension tool, which allowed controlled randomization of prompts. After filtering and cleaning the outputs, a total of 2500 synthetic images were obtained and used for training the object detection model.

To evaluate the impact of synthetic images, a total of 23 YOLOv8n object detection models were trained with identical training parameters using different combinations of real and synthetic images. Quantitatively, models fine-tuned solely on synthetic images achieved $AP_{IoU:0.50}$ between 0.866 and 0.908, outperforming the base model at 0.815. Adding real images on top of a large synthetic set further improved both accuracy and stability. The best performing model combined 568 real images with 2500 synthetic images and achieved an $AP_{IoU:0.50}$ of 0.938.

In the qualitative evaluation, where the detections of the base model were compared with best-performing fine-tuned model trained on both synthetic and real images, it was observed that the mixed-dataset model had become substantially better at detecting people in near-infrared images. This improvement was seen not only in simple cases but also in more challenging ones, including varying lighting, partial occlusions and unusual poses

of individuals.

The methods implemented in this work can be considered an effective way to expand the training dataset for object detection models while also improving model performance. The approach reduces the need to collect large amounts of real data and enables the creation of more diverse training datasets. To make the method even more efficient and time-saving, future research should focus on automating the annotation process of synthetic images, enabling the generated images to be used directly for model training.

REFERENCES

- [1] Zhengxia Zou et al. "Object Detection in 20 Years: A Survey". In: *Proceedings of the IEEE* 111.3 (2023), pp. 1–20.
- [2] Jun-Ho Moon et al. "How much deep learning is enough for automatic identification to be reliable?" In: *The Angle Orthodontist* 90 (July 2020). DOI: 10.2319/021920-116.1.
- [3] Alina Kuznetsova et al. "The Open Images Dataset V4: Unified Image Classification, Object Detection, and Visual Relationship Detection at Scale". In: *International Journal of Computer Vision* 128.7 (Mar. 2020), pp. 1956–1981. DOI: 10.1007/s11263-020-01316-z.
- [4] Mohamed Akrouf et al. *Diffusion-based Data Augmentation for Skin Disease Classification: Impact Across Original Medical Datasets to Fully Synthetic Images*. 2023. URL: <https://arxiv.org/abs/2301.04802>.
- [5] Oğuzhan Can, Özlem Er, and Yusuf Kunt. "Use of Synthetic Data on Object Detection Applications". In: *2021 29th Signal Processing and Communications Applications Conference (SIU)*. 2021, pp. 1–4. DOI: 10.1109/SIU53274.2021.9477977.
- [6] Yaoyuan Wu et al. "Syn2Real Detection in the Sky: Generation and Adaptation of Synthetic Aerial Ship Images". In: *Applied Sciences* 14.11 (2024). DOI: 10.3390/app14114558. URL: <https://www.mdpi.com/2076-3417/14/11/4558>.
- [7] Jiwei Hu et al. "Synthetic Data Generation Based on RDB-CycleGAN for Industrial Object Detection". In: *Mathematics* 11.22 (2023). DOI: 10.3390/math11224588.
- [8] K. Nanthini et al. "A Survey on Data Augmentation Techniques". In: *2023 7th International Conference on Computing Methodologies and Communication (ICCMC)*. 2023, pp. 913–920. DOI: 10.1109/ICCMC56507.2023.10084010.
- [9] Su-Yen Chen. "Generative AI, learning and new literacies". In: *Journal of Educational Technology Development and Exchange* (2023). URL: <https://api.semanticscholar.org/CorpusID:265174809>.
- [10] Alexander Reben. *The weird and wonderful art created when AI and humans unite*. Nov. 2022. URL: <https://www.bbc.com/future/article/20221123-the-weird-and-wonderful-art-created-when-ai-and-humans-unite> (visited on 10/30/2025).
- [11] Negar Kamali et al. *Characterizing Photorealism and Artifacts in Diffusion Model-Generated Images*. 2025. URL: <https://arxiv.org/abs/2502.11989>.
- [12] Charlotte Bird, Eddie L. Ungless, and Atoosa Kasirzadeh. *Typology of Risks of Generative Text-to-Image Models*. 2023. URL: <https://arxiv.org/abs/2307.05543>.

- [13] Lakshmanan Sudha et al. "Semantic Image Synthesis from Text: Current Trends and Future Horizons in Text-to-Image Generation". In: *EAI Endorsed Transactions on Internet of Things* 11 (Nov. 2024). DOI: 10.4108/eetiot.5336. URL: <https://publications.eai.eu/index.php/IoT/article/view/5336>.
- [14] Mrs. Arjoo Pandey. "Computer Vision". In: *International Journal for Research in Applied Science and Engineering Technology* 11.7 (July 2023), pp. 510–514. DOI: 10.22214/ijraset.2023.54701.
- [15] Kari Pulli et al. "Realtime Computer Vision with OpenCV: Mobile computer-vision technology will soon become as ubiquitous as touch interfaces." In: *Queue* 10.4 (Apr. 2012), pp. 40–56. DOI: 10.1145/2181796.2206309.
- [16] Xuan Wang and Zhigang Zhu. "Context understanding in computer vision: A survey". In: *Computer Vision and Image Understanding* 229 (Mar. 2023), p. 103646. DOI: 10.1016/j.cviu.2023.103646.
- [17] Jaskirat Kaur and Williamjeet Singh. "Tools, techniques, datasets and application areas for object detection in an image: a review". In: *Multimedia tools and applications* 81.27 (2022), pp. 38297–38351.
- [18] Yin Zhou and Oncel Tuzel. *VoxelNet: End-to-End Learning for Point Cloud Based 3D Object Detection*. 2017. URL: <https://arxiv.org/abs/1711.06396>.
- [19] Stanley Cohen. "Chapter 2 - The basics of machine learning: strategies and techniques". In: *Artificial Intelligence and Deep Learning in Pathology*. Ed. by Stanley Cohen. Elsevier, 2021, pp. 13–40. DOI: 10.1016/B978-0-323-67538-3.00002-6.
- [20] Christian Janiesch, Patrick Zschech, and Kai Heinrich. "Machine learning and deep learning". In: *Electronic Markets* 31.3 (Apr. 2021), pp. 685–695. DOI: 10.1007/s12525-021-00475-2.
- [21] Cathaoir Agnew et al. "Annotation Quality Versus Quantity for Object Detection and Instance Segmentation". In: *IEEE Access* 12 (2024), pp. 140958–140977. DOI: 10.1109/ACCESS.2024.3467008.
- [22] Chen Sun et al. *Revisiting Unreasonable Effectiveness of Data in Deep Learning Era*. 2017. URL: <https://arxiv.org/abs/1707.02968>.
- [23] Thomas Wiatowski and Helmut Bölcskei. *A Mathematical Theory of Deep Convolutional Neural Networks for Feature Extraction*. 2017. URL: <https://arxiv.org/abs/1512.06293>.
- [24] Ilmars Apeinans. "Optimal Size of Agricultural Dataset for YOLOv8 Training". In: June 2024. DOI: 10.17770/etr2024vol2.8041.
- [25] Xiangxin Zhu et al. "Do We Need More Training Data?" In: *International Journal of Computer Vision* 119.1 (Mar. 2015), pp. 76–92. DOI: 10.1007/s11263-015-0812-2.
- [26] Nikola Djukic, Markus Vincze, and Walter G. Kropatsch. "The Difficulties of Detecting Deformable Objects Using Deep Neural Networks". In: 2020. URL: <https://api.semanticscholar.org/CorpusID:235329977>.

- [27] Annika Mütze et al. *On the Influence of Shape, Texture and Color for Learning Semantic Segmentation*. 2024. URL: <https://arxiv.org/abs/2410.14878>.
- [28] Kaziwa Saleh and Zoltán Vámosy. “BBBD: Bounding Box Based Detector for Occlusion Detection and Order Recovery”. In: *Proceedings of the 2nd International Conference on Image Processing and Vision Engineering - IMPROVE*. INSTICC. 2022, pp. 78–84. URL: <https://arxiv.org/abs/2204.12841>.
- [29] Nithish Muthuchamy Selvaraj, Ilyas Muhammad, and Chien Chern Cheah. “Towards Dependable Object Detection”. In: *IECON 2020 The 46th Annual Conference of the IEEE Industrial Electronics Society*. 2020, pp. 523–528. DOI: 10.1109/IECON43393.2020.9255076.
- [30] Imran Khan Mirani et al. *Object Recognition in Different Lighting Conditions at Various Angles by Deep Learning Method*. 2022. URL: <https://arxiv.org/abs/2210.09618>.
- [31] Dara Molloy et al. “Analysis of the Impact of Lens Blur on Safety-Critical Automotive Object Detection”. In: *IEEE Access* 12 (2024), pp. 3554–3569. DOI: 10.1109/access.2023.3348663.
- [32] Donggil You et al. “Enhancing Depth Image with Richer Convolution Feature for Improved Perception”. In: *2023 23rd International Conference on Control, Automation and Systems (ICCAS)*. IEEE, Oct. 2023, pp. 1866–1869. DOI: 10.23919/iccas59377.2023.10316901.
- [33] Abhishek Mukhopadhyay and Pradipta Biswas. “Advancements in Deep-Learning-Based Object Detection in Challenging Environments”. In: *Wireless World Research and Trends Magazine* (Mar. 2024), pp. 1–6. DOI: 10.13052/2794-7254.001.
- [34] Zhongtian Wang et al. *Towards Reflected Object Detection: A Benchmark*. 2024. URL: <https://arxiv.org/abs/2407.05575>.
- [35] Daehee Park and Yong-Hwa Park. “Identifying Reflected Images From Object Detector in Indoor Environment Utilizing Depth Information”. In: *IEEE Robotics and Automation Letters* 6.2 (2021), pp. 635–642. DOI: 10.1109/LRA.2020.3047796.
- [36] Samed Yalçın and Hazım Kemal Ekenel. *Impact of Surface Reflections in Maritime Obstacle Detection*. 2024. URL: <https://arxiv.org/abs/2410.08713>.
- [37] *Crowd counting dataset*. Kaggle, 2023. URL: <https://www.kaggle.com/datasets/trainingdatapro/crowd-counting-dataset> (visited on 06/03/2025).
- [38] *TIMo Dataset - Overview*. VIZTA Time of Flight Dataset, 2022. URL: <https://vizta-tf.of.kl.dfki.de/timo-dataset-overview/> (visited on 06/03/2025).
- [39] Fares Elmenshawii. *Human dataset*. Kaggle, 2023. URL: <https://www.kaggle.com/datasets/fareselmenshawii/human-dataset> (visited on 06/03/2025).
- [40] Tsung-Yi Lin et al. *Microsoft COCO: Common Objects in Context*. 2015. URL: <https://arxiv.org/abs/1405.0312>.

- [41] Mukhriddin Arabboev et al. "A Comparative Study of Various Object Detection Algorithms". In: *International Research Journal of Modernization in Engineering Technology and Science* 6 (June 2024), pp. 2072–2079. DOI: 10.56726/IRJMET59147.
- [42] *Ultralytics YOLO11*. Ultralytics. URL: <https://docs.ultralytics.com/models/yolo11/> (visited on 07/10/2025).
- [43] Chen Xin, Andreas Hartel, and Enkelejda Kasneci. *DART: An Automated End-to-End Object Detection Pipeline with Data Diversification, Open-Vocabulary Bounding Box Annotation, Pseudo-Label Review, and Model Training*. 2024. URL: <https://arxiv.org/abs/2407.09174>.
- [44] Yibo Sun, Zhe Sun, and Weitong Chen. "The evolution of object detection methods". In: *Engineering Applications of Artificial Intelligence* 133 (2024), p. 108458. DOI: 10.1016/j.engappai.2024.108458.
- [45] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Ed. by F. Pereira et al. Vol. 25. Curran Associates, Inc., 2012. URL: https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf.
- [46] Ross Girshick et al. *Rich feature hierarchies for accurate object detection and semantic segmentation*. 2014. URL: <https://arxiv.org/abs/1311.2524>.
- [47] Ashish Vaswani et al. *Attention Is All You Need*. 2023. URL: <https://arxiv.org/abs/1706.03762>.
- [48] Hanqing Bi, Vincent Wen, and Zhenyu Xu. "Comparing one-stage and two-stage learning strategy in object detection". In: *Applied and Computational Engineering* 5 (June 2023), pp. 171–177. DOI: 10.54254/2755-2721/5/20230556.
- [49] Akhilesh Sharma, Vipin Kumar, and Louis Longchamps. "Comparative performance of YOLOv8, YOLOv9, YOLOv10, YOLOv11 and Faster R-CNN models for detection of multiple weed species". In: *Smart Agricultural Technology* 9 (2024), p. 100648. DOI: 10.1016/j.atech.2024.100648.
- [50] Nicolas Carion et al. *End-to-End Object Detection with Transformers*. 2020. URL: <https://arxiv.org/abs/2005.12872>.
- [51] Alexey Dosovitskiy et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021. URL: <https://arxiv.org/abs/2010.11929>.
- [52] Alexander Mattern et al. "A comparison of transformer and CNN-based object detection models for surface defects on Li-Ion Battery Electrodes". In: *Journal of Energy Storage* 105 (Jan. 2025), p. 114378. DOI: 10.1016/j.est.2024.114378.
- [53] Md Sajol and A S M Jahid Hasan. "Benchmarking CNN and Cutting-Edge Transformer Models for Brain Tumor Classification Through Transfer Learning". In: Aug. 2024, pp. 1–6. DOI: 10.1109/IS61756.2024.10705175.

- [54] Haodong Ouyang. *DEYOv2: Rank Feature with Greedy Matching for End-to-End Object Detection*. 2023. URL: <https://arxiv.org/abs/2306.09165>.
- [55] Álvaro Arcos-García, Juan Alvarez-Garcia, and Luis Soria Morillo. "Evaluation of Deep Neural Networks for traffic sign detection systems". In: *Neurocomputing* 316 (Aug. 2018). DOI: 10.1016/j.neucom.2018.08.009.
- [56] Rafael Padilla, Sergio L. Netto, and Eduardo A. B. da Silva. "A Survey on Performance Metrics for Object-Detection Algorithms". In: *2020 International Conference on Systems, Signals and Image Processing (IWSSIP)*. 2020, pp. 237–242. DOI: 10.1109/IWSSIP48289.2020.9145130.
- [57] Hyeonjin Lee, Jeong-Sik Lee, and Hyun-Chul Choi. "Parallelization of Non-Maximum Suppression". In: *IEEE Access* 9 (2021), pp. 166579–166587. DOI: 10.1109/ACCESS.2021.3134639.
- [58] Juan Terven et al. "A comprehensive survey of loss functions and metrics in deep learning". In: *Artificial Intelligence Review* 58.7 (Apr. 2025). DOI: 10.1007/s10462-025-11198-7.
- [59] Rafael Padilla et al. "A Comparative Analysis of Object Detection Metrics with a Companion Open-Source Toolkit". In: *Electronics* 10.3 (2021). DOI: 10.3390/electronics10030279.
- [60] Ovidiu L. Calin. *Deep learning architectures: a mathematical approach*. Springer series in the data sciences. Cham, Switzerland: Springer, 2020. DOI: 10.1007/978-3-030-36721-3.
- [61] Marcos Eduardo Valle, Wington L. Vital, and Guilherme Vieira. *Universal Approximation Theorem for Vector- and Hypercomplex-Valued Neural Networks*. 2024. URL: <https://arxiv.org/abs/2401.02277>.
- [62] Zhou Lu et al. *The Expressive Power of Neural Networks: A View from the Width*. 2017. URL: <https://arxiv.org/abs/1709.02540>.
- [63] Teofilo F. Gonzalez. *Handbook of Approximation Algorithms and Metaheuristics (Chapman & Hall/Crc Computer & Information Science Series)*. Chapman & Hall/CRC, 2007.
- [64] Evelyn Herberg. *Lecture Notes: Neural Network Architectures*. 2023. URL: <https://arxiv.org/abs/2304.05133>.
- [65] Abolfazl Younesi et al. "A Comprehensive Survey of Convolutions in Deep Learning: Applications, Challenges, and Future Trends". In: *IEEE Access* 12 (2024), pp. 41180–41218. DOI: 10.1109/ACCESS.2024.3376441.
- [66] Muhammad Asif Saleem et al. "Comparative Analysis of Recent Architecture of Convolutional Neural Network". In: *Mathematical Problems in Engineering* 2022.1 (2022), p. 7313612. DOI: 10.1155/2022/7313612.
- [67] Rikiya Yamashita et al. "Convolutional neural networks: an overview and application in radiology". In: *Insights into Imaging* 9 (June 2018). DOI: 10.1007/s13244-018-0639-9.

- [68] Mohammad Mustafa Taye. "Theoretical Understanding of Convolutional Neural Network: Concepts, Architectures, Applications, Future Directions". In: *Computation* 11.3 (2023). DOI: 10.3390/computation11030052. URL: <https://www.mdpi.com/2079-3197/11/3/52>.
- [69] Joseph Redmon et al. *You Only Look Once: Unified, Real-Time Object Detection*. 2016. URL: <https://arxiv.org/abs/1506.02640>.
- [70] Sobhan Soleymani et al. *Multi-Level Feature Abstraction from Convolutional Neural Networks for Multimodal Biometric Identification*. 2018. URL: <https://arxiv.org/abs/1807.01332>.
- [71] Matthew D Zeiler and Rob Fergus. *Visualizing and Understanding Convolutional Networks*. 2013. URL: <https://arxiv.org/abs/1311.2901>.
- [72] Ramprasaath R. Selvaraju et al. "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization". In: *International Journal of Computer Vision* 128.2 (Oct. 2019), pp. 336–359. DOI: 10.1007/s11263-019-01228-7.
- [73] Frincy Clement, Ji Yang, and Irene Cheng. *Feature CAM: Interpretable AI in Image Classification*. 2024. URL: <https://arxiv.org/abs/2403.05658>.
- [74] Aniket K. Singh et al. *Integrative CAM: Adaptive Layer Fusion for Comprehensive Interpretation of CNNs*. 2024. URL: <https://arxiv.org/abs/2412.01354>.
- [75] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. 2015. URL: <https://arxiv.org/abs/1505.04597>.
- [76] Xue-Cheng Tai et al. *A Mathematical Explanation of UNet*. 2024. URL: <https://arxiv.org/abs/2410.04434>.
- [77] Lizhou Liu and Yong Zhou. "A closer look at U-net for road detection". In: *International Conference on Digital Image Processing*. 2018. URL: <https://api.semanticscholar.org/CorpusID:173181204>.
- [78] Jaaneet Shah, Michael Gromis, and Rickston Pinto. *Enhancing Diffusion Models for High-Quality Image Generation*. 2024. URL: <https://arxiv.org/abs/2412.14422>.
- [79] Jianbo Liu et al. *A Holistically-Guided Decoder for Deep Representation Learning with Applications to Semantic Segmentation and Object Detection*. 2020. URL: <https://arxiv.org/abs/2012.10162>.
- [80] Saidul Islam et al. "A comprehensive survey on applications of transformers for deep learning tasks". In: *Expert Systems with Applications* 241 (2024), p. 122666. DOI: 10.1016/j.eswa.2023.122666.
- [81] Zihao Fu et al. *Decoder-Only or Encoder-Decoder? Interpreting Language Model as a Regularized Encoder-Decoder*. 2023. URL: <https://arxiv.org/abs/2304.04052>.
- [82] Paul Suganthan et al. *Adapting Decoder-Based Language Models for Diverse Encoder Downstream Tasks*. 2025. URL: <https://arxiv.org/abs/2503.02656>.

- [83] Dhar Susnato. *Unit 3. Transformer architectures for audio*. July 2023. URL: <https://github.com/huggingface/audio-transformers-course/blob/main/chapters/en/chapter3/introduction.mdx> (visited on 07/27/2025).
- [84] Fadi Gattoussi. *Image Tokenization*. Nov. 2024. URL: https://github.com/canyon289/GenAiGuidebook/blob/main/genaiguidebook/image/image_tokenization.md (visited on 07/27/2025).
- [85] Alina Arseniev-Koehler. *Theoretical foundations and limits of word embeddings: what types of meaning can they capture?* 2021. URL: <https://arxiv.org/abs/2107.10413>.
- [86] Xin Zhou, Zhaohui Ren, and Tianzhuang Yu. *Rethinking position embedding methods in the Transformer architecture*. Jan. 2023. DOI: 10.21203/rs.3.rs-2525471/v1.
- [87] Amirhossein Kazemnejad. "Transformer Architecture: The Positional Encoding". In: (2019). URL: https://kazemnejad.com/blog/transformer_architecture_positional_encoding/ (visited on 07/27/2025).
- [88] Dave Bergmann and Cole Stryker. *What is an attention mechanism?* 2024. URL: <https://www.ibm.com/think/topics/attention-mechanism> (visited on 07/24/2025).
- [89] Nan Zhang and Junyeong Kim. "A Survey on Attention mechanism in NLP". In: *2023 International Conference on Electronics, Information, and Communication (ICEIC)*. 2023, pp. 1–4. DOI: 10.1109/ICEIC57457.2023.10049971.
- [90] Md Kowsher et al. *Does Self-Attention Need Separate Weights in Transformers?* 2025. URL: <https://arxiv.org/abs/2412.00359>.
- [91] Benyamin Ghogh and Ali Ghodsi. *Attention Mechanism, Transformers, BERT, and GPT: Tutorial and Survey*. Dec. 2020. DOI: 10.31219/osf.io/m6gcn.
- [92] Sarthak Mittal et al. *Compositional Attention: Disentangling Search and Retrieval*. 2022. URL: <https://arxiv.org/abs/2110.09419>.
- [93] Rohan K. Mehta. "The Attention Mechanism Demystified". In: 2023. URL: <https://api.semanticscholar.org/CorpusID:259319157>.
- [94] James Bernhard. *Alternatives to the Scaled Dot Product for Attention in the Transformer Neural Network Architecture*. 2023. URL: <https://arxiv.org/abs/2311.09406>.
- [95] Alec Radford et al. *Learning Transferable Visual Models From Natural Language Supervision*. 2021. URL: <https://arxiv.org/abs/2103.00020>.
- [96] Yiming Xu. "Multihead Attention-based Audio Image Generation with Cross-Modal Shared Weight Classifier". In: *2023 International Joint Conference on Neural Networks (IJCNN)*. 2023, pp. 1–7. DOI: 10.1109/IJCNN54540.2023.10191393.
- [97] Dashanka De Silva et al. *NeuroSpex: Neuro-Guided Speaker Extraction with Cross-Modal Attention*. 2024. URL: <https://arxiv.org/abs/2409.02489>.
- [98] Roy Voetman, Maya Aghaei, and Klaas Dijkstra. *The Big Data Myth: Using Diffusion Models for Dataset Generation to Train Deep Detection Models*. 2023. URL: <https://arxiv.org/abs/2306.09762>.

- [99] Benjamin Kiefer, David Ott, and Andreas Zell. *Leveraging Synthetic Data in Object Detection on Unmanned Aerial Vehicles*. 2021. URL: <https://arxiv.org/abs/2112.12252>.
- [100] Rita Delussu, Putzu Lorenzo, and Giorgio Fumera. "Synthetic Data for Video Surveillance Applications of Computer Vision: A Review". In: *International Journal of Computer Vision* 132 (2024), pp. 4473–4509. DOI: 10.1007/s11263-024-02102-x.
- [101] Huu Long Nguyen, Duc Toan Le, and Hong Hai Hoang. "Application of Synthetic Data on Object Detection Tasks". In: *Engineering, Technology & Applied Science Research* 14.4 (2024), pp. 15695–15699. DOI: 10.48084/etasr.7929.
- [102] Jonathan Tremblay et al. *Training Deep Networks with Synthetic Data: Bridging the Reality Gap by Domain Randomization*. 2018. URL: <https://arxiv.org/abs/1804.06516>.
- [103] Datao Tang et al. *CRS-Diff: Controllable Remote Sensing Image Generation with Diffusion Model*. 2024. URL: <https://arxiv.org/abs/2403.11614>.
- [104] Georgios Georgakis et al. *Synthesizing Training Data for Object Detection in Indoor Scenes*. 2017. URL: <https://arxiv.org/abs/1702.07836>.
- [105] Hancheng Zhang et al. "Enhancing Rare Object Detection on Roadways Through Conditional Diffusion Models for Data Augmentation". In: *IEEE Transactions on Intelligent Transportation Systems* 25.11 (Nov. 2024), pp. 19018–19029. DOI: 10.1109/tits.2024.3438263.
- [106] J  r  my Jaspar et al. "Using Synthetic Images to Improve and Test Object Detection in the Context of the Autonomous Vehicle". In: *2024 12th European Workshop on Visual Information Processing (EUVIP)*. 2024, pp. 1–6. DOI: 10.1109/EUVIP61797.2024.10772859.
- [107] Bram Vanherle et al. *Analysis of Training Object Detection Models with Synthetic Data*. 2022. URL: <https://arxiv.org/abs/2211.16066>.
- [108] Nikhil Mishra et al. *Closing the Visual Sim-to-Real Gap with Object-Composable NeRFs*. 2024. URL: <https://arxiv.org/abs/2403.04114>.
- [109] Ritvik Singh et al. *Synthetica: Large Scale Synthetic Data for Robot Perception*. 2024. URL: <https://arxiv.org/abs/2410.21153>.
- [110] Eduard Bartolovic et al. "From Sources to Solutions: Enhancing Object Detection Models through Synthetic Data". In: *Computer Science Research Notes* (2024). URL: <https://api.semanticscholar.org/CorpusID:271426693>.
- [111] Augusto J. Peterlevitz et al. "Sim-to-Real Transfer for Object Detection in Aerial Inspections of Transmission Towers". In: *IEEE Access* 11 (2023), pp. 110312–110327. URL: <https://api.semanticscholar.org/CorpusID:263729608>.
- [112] Yanan Jian et al. *Stable Diffusion For Aerial Object Detection*. 2023. URL: <https://arxiv.org/abs/2311.12345>.

- [113] Maged Shoman et al. "Enforcing Traffic Safety: A Deep Learning Approach for Detecting Motorcyclists' Helmet Violations Using YOLOv8 and Deep Convolutional Generative Adversarial Network-Generated Images". In: *Algorithms* 17.5 (2024). DOI: 10.3390/a17050202.
- [114] Ilya Reutov. "Generating of synthetic datasets using diffusion models for solving computer vision tasks in urban applications". In: *Procedia Computer Science* 229 (2023), pp. 335–344. DOI: 10.1016/j.procs.2023.12.036.
- [115] Aayush Agrawal et al. *Syn2Real Domain Generalization for Underwater Mine-like Object Detection Using Side-Scan Sonar*. 2024. URL: <https://arxiv.org/abs/2410.12953>.
- [116] Sven Burdorf, Karoline Plum, and Daniel Hasenklever. *Reducing the Amount of Real World Data for Object Detector Training with Synthetic Data*. 2022. URL: <https://arxiv.org/abs/2202.00632>.
- [117] Vitória Biz Cecchetti, Marcelo Rudek, and Roberto Z. Freire. "In the Search for the Balance Between Real and Synthetic Images in Multi-Class Detection Systems". In: *2024 IEEE 19th Conference on Industrial Electronics and Applications (ICIEA)*. 2024, pp. 1–6. DOI: 10.1109/ICIEA61579.2024.10664977.
- [118] Chenshuang Zhang et al. *Text-to-image Diffusion Models in Generative AI: A Survey*. 2024. URL: <https://arxiv.org/abs/2303.07909>.
- [119] Nonghai Zhang and Hao Tang. *Text-to-Image Synthesis: A Decade Survey*. 2024. URL: <https://arxiv.org/abs/2411.16164>.
- [120] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. 2020. URL: <https://arxiv.org/abs/2006.11239>.
- [121] Inga Strümke and Helge Langseth. *Lecture Notes in Probabilistic Diffusion Models*. 2023. URL: <https://arxiv.org/abs/2312.10393>.
- [122] Jascha Sohl-Dickstein et al. *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*. 2015. URL: <https://arxiv.org/abs/1503.03585>.
- [123] Gabriel della Maggiora et al. *Conditional Variational Diffusion Models*. 2024. URL: <https://arxiv.org/abs/2312.02246>.
- [124] Zheyuan Zhan et al. *Conditional Image Synthesis with Diffusion Models: A Survey*. 2025. URL: <https://arxiv.org/abs/2409.19365>.
- [125] Katerina Margatina, Christos Baziotis, and Alexandros Potamianos. *Attention-based Conditioning Methods for External Knowledge Integration*. 2019. URL: <https://arxiv.org/abs/1906.03674>.
- [126] Prafulla Dhariwal and Alex Nichol. *Diffusion Models Beat GANs on Image Synthesis*. 2021. URL: <https://arxiv.org/abs/2105.05233>.
- [127] Jonathan Ho and Tim Salimans. *Classifier-Free Diffusion Guidance*. 2022. URL: <https://arxiv.org/abs/2207.12598>.
- [128] Yuanhao Ban et al. *Understanding the Impact of Negative Prompts: When and How Do They Take Effect?* 2024. URL: <https://arxiv.org/abs/2406.02965>.

- [129] Bingyan Liu et al. *Towards Understanding Cross and Self-Attention in Stable Diffusion for Text-Guided Image Editing*. 2024. URL: <https://arxiv.org/abs/2403.03431>.
- [130] Bakht Alam Khan and Jin-Woo Jung. "Semantic Segmentation of Aerial Imagery Using U-Net with Self-Attention and Separable Convolutions". In: *Applied Sciences* 14.9 (2024). DOI: 10.3390/app14093712.
- [131] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. *Adding Conditional Control to Text-to-Image Diffusion Models*. 2023. URL: <https://arxiv.org/abs/2302.05543>.
- [132] Robin Rombach et al. *High-Resolution Image Synthesis with Latent Diffusion Models*. 2022. URL: <https://arxiv.org/abs/2112.10752>.
- [133] *Stable Diffusion*. CompVis - Computer Vision and Learning LMU Munich. URL: <https://github.com/CompVis/stable-diffusion/> (visited on 09/08/2025).
- [134] Diederik P Kingma and Max Welling. *Auto-Encoding Variational Bayes*. 2022. URL: <https://arxiv.org/abs/1312.6114>.
- [135] Christoph Schuhmann et al. *LAION-5B: An open large-scale dataset for training next generation image-text models*. 2022. URL: <https://arxiv.org/abs/2210.08402>.
- [136] *clip-vit-large-patch14*. OpenAI. URL: <https://huggingface.co/openai/clip-vit-large-patch14> (visited on 10/02/2025).
- [137] *Comparing stable diffusion models*. Apr. 2024. URL: <https://medium.com/@promptingpixels/comparing-stable-diffusion-models-2c1dc9919ab7> (visited on 09/08/2025).
- [138] Patrick Esser, Robin Rombach, and Björn Ommer. *Taming Transformers for High-Resolution Image Synthesis*. 2021. URL: <https://arxiv.org/abs/2012.09841>.
- [139] Dustin Podell et al. *SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis*. 2023. URL: <https://arxiv.org/abs/2307.01952>.
- [140] Edward J. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. 2021. URL: <https://arxiv.org/abs/2106.09685>.
- [141] Luping Wang et al. *Parameter-Efficient Fine-Tuning in Large Models: A Survey of Methodologies*. 2025. URL: <https://arxiv.org/abs/2410.19878>.
- [142] *LoRA*. Diffusers. URL: <https://huggingface.co/docs/diffusers/api/loaders/lora> (visited on 08/07/2025).
- [143] Ethan Smith et al. *LoRA Diffusion: Zero-Shot LoRA Synthesis for Diffusion Model Personalization*. 2024. URL: <https://arxiv.org/abs/2412.02352>.
- [144] Shih-Ying Yeh et al. *Navigating Text-To-Image Customization: From LyCORIS Fine-Tuning to Model Evaluation*. 2024. URL: <https://arxiv.org/abs/2309.14859>.
- [145] Aniket Roy et al. *MultLFG: Training-free Multi-LoRA composition using Frequency-domain Guidance*. 2025. URL: <https://arxiv.org/abs/2505.20525>.
- [146] Styliani I. Kampezidou et al. "Fundamental Components and Principles of Supervised Machine Learning Workflows with Numerical and Categorical Data". In: *Eng* 5.1 (2024), pp. 384–416. DOI: 10.3390/eng5010021.

- [147] Stefan Studer et al. *Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology*. 2021. URL: <https://arxiv.org/abs/2003.05155>.
- [148] Andrei Paleyes, Raoul-Gabriel Urma, and Neil D. Lawrence. “Challenges in Deploying Machine Learning: A Survey of Case Studies”. In: *ACM Computing Surveys* 55.6 (Dec. 2022), pp. 1–29. DOI: 10.1145/3533378.
- [149] Daochen Zha et al. “Data-centric Artificial Intelligence: A Survey”. In: *arXiv.org* (2023).
- [150] *Upscayl*. URL: <https://github.com/upscayl/upscayl> (visited on 10/10/2025).
- [151] jhc13. *TagGUI*. URL: <https://github.com/jhc13/taggui> (visited on 10/09/2025).
- [152] *SD-XL 1.0-base Model Card*. Stability AI. URL: <https://huggingface.co/stabilityai/stable-diffusion-xl-base-1.0> (visited on 10/10/2025).
- [153] Benoit Maltais. *Kohya SS Repository*. URL: https://github.com/bmaltais/kohya_ss/ (visited on 03/24/2025).
- [154] Illyasviel. *Stable Diffusion WebUI Forge*. URL: <https://github.com/Illyasviel/stable-diffusion-webui-forge> (visited on 03/24/2025).
- [155] Stéphane Charette. *DarkMark: A Tool for Annotating Images for Object Detection*. URL: <https://github.com/stephanecharette/DarkMark> (visited on 03/24/2025).
- [156] *Ultralytics YOLOv8*. Ultralytics. URL: <https://huggingface.co/Ultralytics/YOLOv8> (visited on 10/10/2025).
- [157] *Ultralytics*. URL: <https://github.com/ultralytics/ultralytics> (visited on 10/10/2025).
- [158] FollowfoxAI. *Impact of Image Captions on Style Training for Stable Diffusion - 7 Experiments*. 2023. URL: <https://followfoxai.substack.com/p/impact-of-image-caption-on-style> (visited on 10/09/2025).
- [159] CrasH. *Captions vs No Captions: A Deep Dive into Effects on Flux LoRA Training*. 2024. URL: <https://civitai.com/articles/7203/captions-vs-no-captions-a-deep-dive-into-effects-on-flux-lora-training> (visited on 10/09/2025).
- [160] ts953304. *LoRA Training — How / What to Caption?* 2024. URL: https://www.reddit.com/r/StableDiffusion/comments/1b3ygtz/lora_training_how_what_to_caption/ (visited on 10/09/2025).
- [161] julieroseoff. *Why no captions? LoRA training perform better?* 2024. URL: https://www.reddit.com/r/StableDiffusion/comments/1brdew9/why_no_captions_lora_training_perform_better/ (visited on 10/09/2025).
- [162] terrarium. *Advanced advice for model training / fine-tuning and captioning*. 2023. URL: https://www.reddit.com/r/StableDiffusion/comments/114dxgl/advanced_advice_for_model_training_finetuning_and/ (visited on 10/09/2025).
- [163] Benoit Maltais. *LoRA Training Parameters - Kohya SS Wiki*. URL: https://github.com/bmaltais/kohya_ss/wiki/LoRA-training-parameters (visited on 03/24/2025).
- [164] Noam Shazeer and Mitchell Stern. *Adafactor: Adaptive Learning Rates with Sub-linear Memory Cost*. 2018. URL: <https://arxiv.org/abs/1804.04235>.

- [165] AUTOMATIC1111. *Stable Diffusion WebUI*. URL: <https://github.com/AUTOMATIC1111/stable-diffusion-webui> (visited on 03/24/2025).
- [166] Raphael Tang et al. *What the DAAM: Interpreting Stable Diffusion Using Cross Attention*. 2022. URL: <https://arxiv.org/abs/2210.04885>.
- [167] Alexander Kirillov et al. *Segment Anything*. 2023. URL: <https://arxiv.org/abs/2304.02643>.

APPENDIX A: EFFECTS OF IMAGE GENERATION PARAMETERS

This appendix contains additional figures demonstrating the effects of the sampler type, number of sampling steps, and CFG scale on the image generation process.

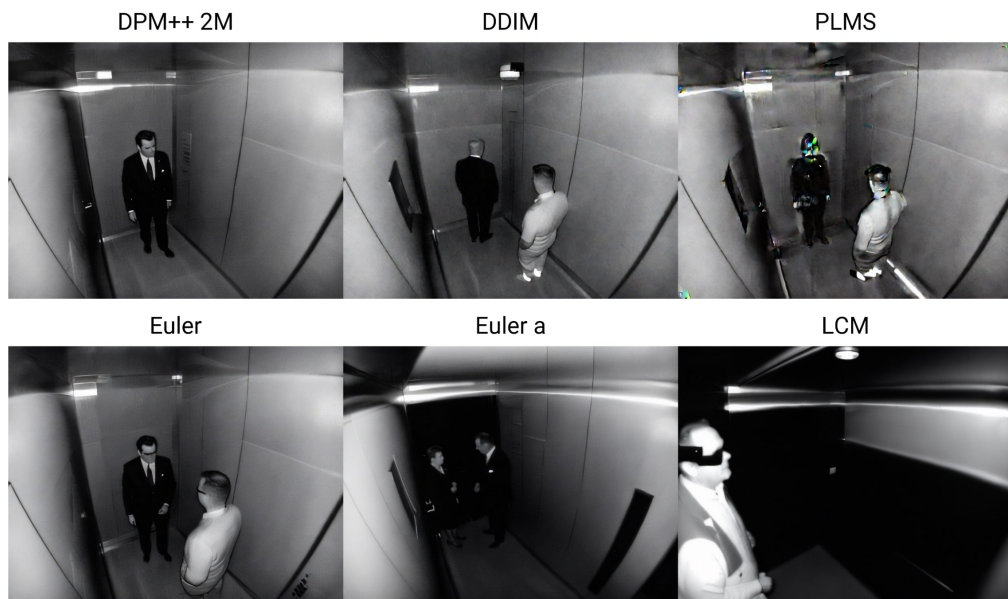


Figure A.1. Effect of changing the sampler while keeping the random seed and other parameters from Table 3.7 constant.

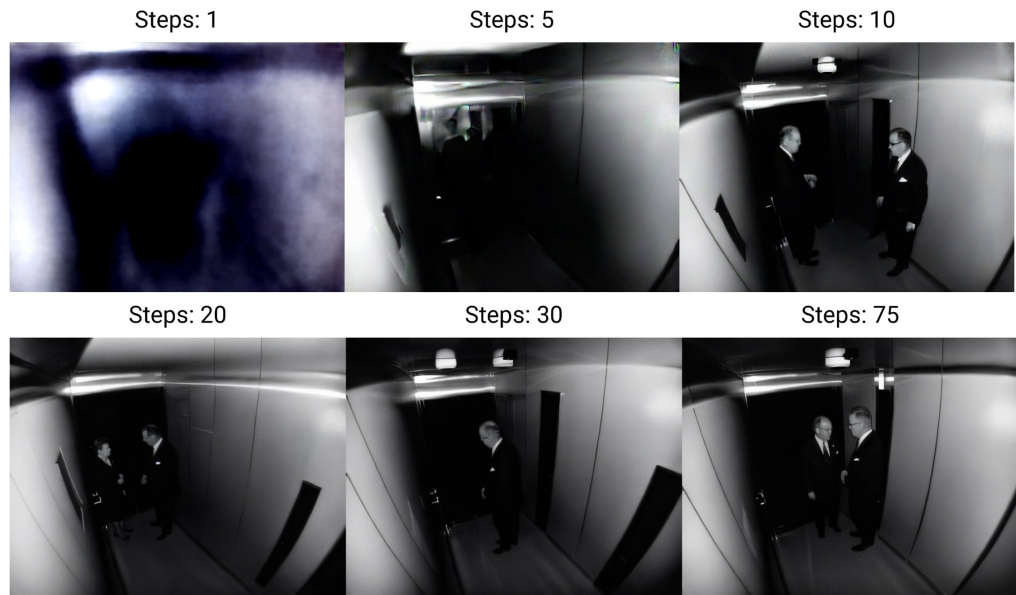


Figure A.2. Effect of changing the sampling step while keeping the random seed and other parameters from Table 3.7 constant.

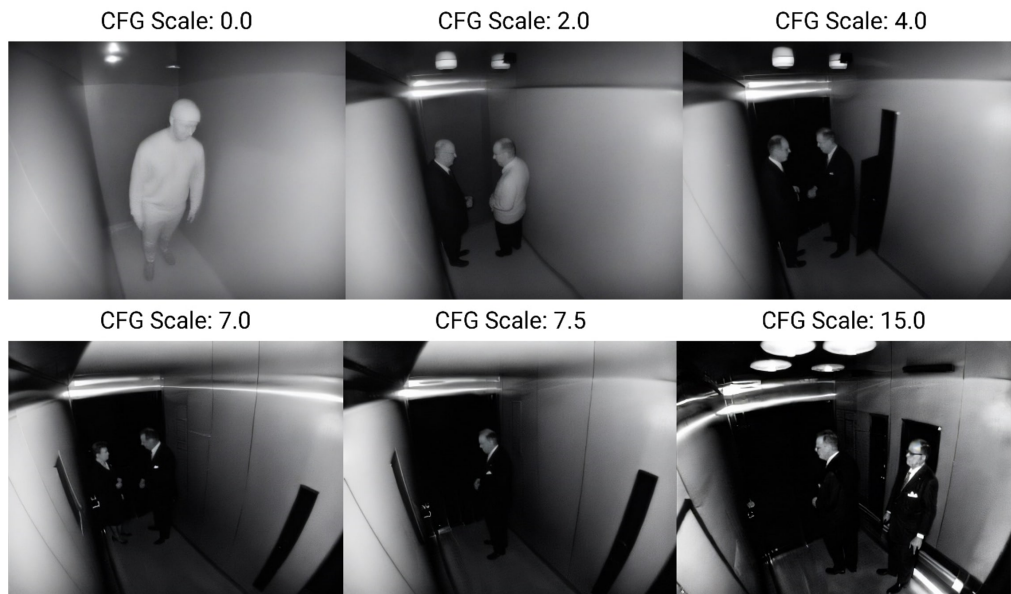


Figure A.3. Effect of changing the CFG while keeping the random seed and other parameters from Table 3.7 constant.

APPENDIX B: ADDITIONAL RESULTS

This appendix contains additional results from the object detection model evaluation.

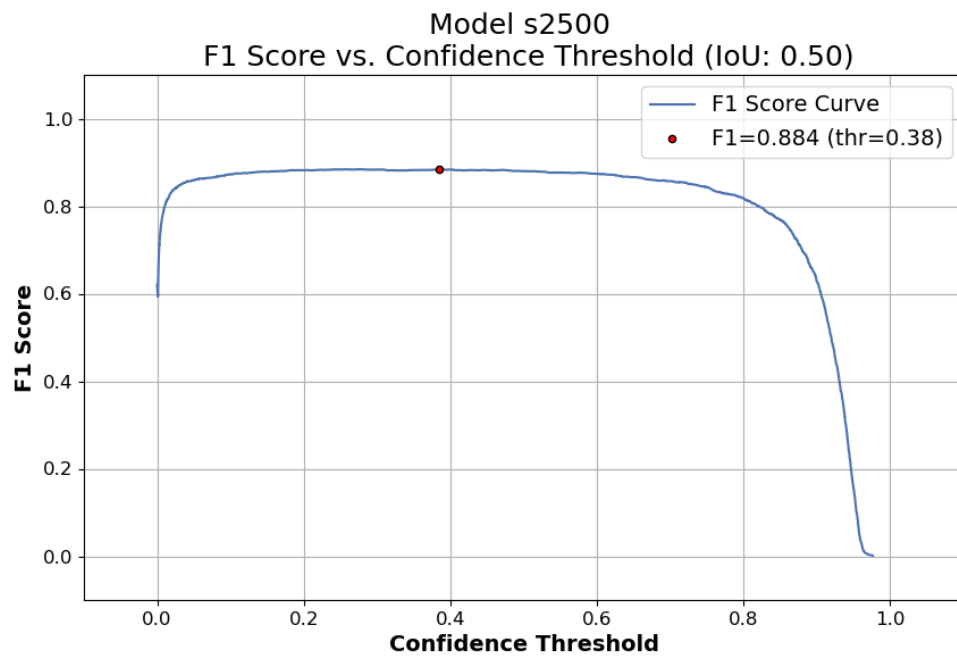


Figure B.1. F1-scores over all confidence thresholds for the s2500 model. The highest F1-score on the curve is marked with a dot.

Table B.1. Comparison between the fine-tuned YOLOv8n models and the base model. The numbers of predicted boxes, true positives (TP), false positives (FP), false negatives (FN), and derived metrics (precision, recall, and F1) are reported at each model's optimal confidence threshold. The Average Precision ($AP_{IoU:0.50}$) is calculated as the area under the precision-recall curve across all confidence thresholds (0-1). All results are computed using an IoU threshold of 0.5.

Model (s = synthetic, r = real)	Best confidence threshold	Ground-truth boxes	Predicted boxes	True positives (TP _{IoU:0.50})	False positives (FP _{IoU:0.50})	False negatives (FN _{IoU:0.50})	Precision _{IoU:0.50}	Recall _{IoU:0.50}	F1 _{IoU:0.50}	AP _{IoU:0.50}
s500	0.770	1785	1747	1455	292	330	0.833	0.815	0.824	0.866
s1000	0.593	1785	1748	1527	221	258	0.874	0.855	0.864	0.888
s1500	0.498	1785	1789	1540	249	245	0.861	0.863	0.862	0.889
s2000	0.384	1785	1702	1542	160	243	0.906	0.864	0.884	0.908
s2500	0.369	1785	1654	1485	169	300	0.898	0.832	0.864	0.876
r200	0.818	1785	1780	1384	396	401	0.778	0.775	0.776	0.828
r200s500	0.505	1785	1531	1395	136	390	0.911	0.782	0.841	0.862
r200s1000	0.398	1785	1670	1496	174	289	0.896	0.838	0.866	0.893
r200s1500	0.392	1785	1622	1499	123	286	0.924	0.840	0.880	0.915
r200s2000	0.441	1785	1750	1585	165	200	0.906	0.888	0.897	0.920
r200s2500	0.180	1785	1771	1590	181	195	0.898	0.891	0.894	0.922
r400	0.659	1785	1973	1520	453	265	0.770	0.852	0.809	0.854
r400s500	0.577	1785	1604	1458	146	327	0.909	0.817	0.860	0.909
r400s1000	0.609	1785	1705	1574	131	211	0.923	0.882	0.902	0.938
r400s1500	0.311	1785	1738	1577	161	208	0.907	0.883	0.895	0.921
r400s2000	0.228	1785	1749	1582	167	203	0.905	0.886	0.895	0.925
r400s2500	0.469	1785	1721	1580	141	205	0.918	0.885	0.901	0.932
r568	0.566	1785	1646	1540	106	245	0.936	0.863	0.898	0.923
r568s500	0.195	1785	1746	1587	159	198	0.909	0.889	0.899	0.929
r568s1000	0.350	1785	1723	1564	159	221	0.908	0.876	0.892	0.916
r568s1500	0.338	1785	1694	1555	139	230	0.918	0.871	0.894	0.917
r568s2000	0.218	1785	1734	1591	143	194	0.918	0.891	0.904	0.921
r568s2500	0.332	1785	1714	1591	123	194	0.928	0.891	0.909	0.938
yolov8n	0.161	1785	1496	1207	289	578	0.807	0.676	0.736	0.815